



SAPIENZA
UNIVERSITÀ DI ROMA

Ingegneria del Software

UNIVERSITÀ DEGLI STUDI DI ROMA "LA
SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
PROF. ENRICO TRONCI

1 dicembre 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Questo testo è stato integrato con il libro [1].

Indice

Prefazione	1
1 Introduzione	7
1.1 Sviluppo software professionale	7
1.1.1 Ingegneria del software	9
1.1.2 Diversità nell'ingegneria del software	10
1.1.3 Ingegneria del software e Web	11
1.2 Etica dell'ingegneria del software	11
1.3 Casi di studio	13
1.4 Punti chiave	17
2 Processi software	19
2.1 Modelli di processo software	19
2.1.1 Modello a cascata	20
2.1.2 Sviluppo incrementale	20
2.1.3 Integrazione e configurazione	21
2.2 Attività di processo	22
2.2.1 Specificazione del software	22
2.2.2 Progettazione e implementazione del software	23
2.2.3 Validazione del software	23
2.2.4 Evoluzione del software	24
2.3 Affrontare il cambiamento	25
2.3.1 Prototipazione	25
2.3.2 Consegna incrementale	26
2.4 Miglioramento dei processi	27
2.5 Punti chiave	29
3 Sviluppo software agile	31
3.1 Metodi agili	31
3.2 Tecniche di sviluppo agili	33
3.2.1 Storie degli utenti	33
3.2.2 Refactoring	35
3.2.3 Test-first development	35
3.2.4 Pair programming	36
3.3 Gestione agile dei progetti	36
3.4 Scalabilità dei metodi agili	38
3.4.1 Problemi pratici con i metodi agili	39
3.4.2 Metodi agili e metodi plan-driven	39
3.4.3 Metodi agili per grandi sistemi	41
3.4.4 Metodi agili nelle organizzazioni	42
3.5 Punti chiave	42
4 Ingegneria dei requisiti	43
4.1 Requisiti funzionali e non funzionali	45
4.1.1 Requisiti funzionali	45
4.1.2 Requisiti non funzionali	45
4.2 Processi di ingegneria dei requisiti	47

4.3	Elicitazione dei requisiti	48
4.3.1	Tecniche di elicitazione dei requisiti	49
4.3.2	Storie e scenari	50
4.4	Specifica dei requisiti	51
4.4.1	Specifica in linguaggio naturale	52
4.4.2	Specifiche strutturate	53
4.4.3	Use cases	54
4.4.4	Documento sui requisiti software	55
4.5	Validazione dei requisiti	55
4.6	Requisiti in continua evoluzione	56
4.6.1	Pianificazione della gestione dei requisiti	57
4.6.2	Gestione delle modifiche ai requisiti	57
4.7	Punti chiave	58
5	Modellazione del sistema	61
5.1	Modelli di contesto	62
5.2	Modelli di interazione	63
5.2.1	Modellazione degli use case	63
5.2.2	Diagrammi di sequenza	64
5.3	Modelli strutturali	66
5.3.1	Diagrammi delle classi	66
5.3.2	Generalizzazione	67
5.3.3	Aggregazione	67
5.4	Modelli comportamentali	68
5.4.1	Modelli guidati dai dati	68
5.4.2	Modelli guidati dagli eventi	69
5.5	Ingegneria guidata dai modelli	70
5.5.1	Architettura guidata da modelli	71
5.6	Punti chiave	73
6	Architectural design	75
6.1	Decisioni di architectural design	76
6.2	Architectural views	77
6.3	Architectural patterns	78
6.3.1	Layered architecture	79
6.3.2	Repository architecture	80
6.3.3	Client-server architecture	81
6.3.4	Pipe and filter architecture	82
6.4	Application architectures	83
6.4.1	Sistemi di elaborazione delle transazioni	84
6.4.2	Sistemi informativi	84
6.4.3	Sistemi di elaborazione del linguaggio	85
6.5	Punti chiave	87
7	Test del software	89
7.1	Test di sviluppo	91
7.1.1	Unit testing	91
7.1.2	Scelta dei casi di unit test	92
7.1.3	Testing dei componenti	93
7.1.4	Testing del sistema	95
7.2	Test-driven development	96
7.3	Testing di rilascio	97
7.3.1	Testing basato sui requisiti	97
7.3.2	Testing basato su scenari	98
7.3.3	Testing delle prestazioni	98
7.4	Testing dell'utente	99
7.5	Punti chiave	100

8	Pianificazione del progetto	101
8.1	Prezzi del software	101
8.2	Sviluppo plan-driven	102
8.2.1	Piani di progetto	102
8.2.2	Il processo di pianificazione	103
8.3	Pianificazione del progetto	104
8.3.1	Presentazione della pianificazione	105
8.4	Pianificazione agile	106
8.5	Tecniche di stima	108
8.5.1	Tecniche basate sull'esperienza	108
8.5.2	Modelli algoritmici di costo	109
8.6	Modellazione dei costi COCOMO	109
8.6.1	Modello di composizione dell'applicazione	109
8.6.2	Modello di progettazione preliminare	110
8.6.3	Modello di riuso	111
8.6.4	Livello post-architetturale	111
8.6.5	Durata del progetto e gestione del personale	113
8.7	Punti chiave	113
9	Altri argomenti	115
9.1	Discrete Time Markov Chains	115
9.1.1	Esempi di utilizzo delle DTMC	115
9.1.2	Reti di DTMC	118
	Bibliografia	119

Capitolo 1

Introduzione

Gli obiettivi di questo capitolo sono:

- capire cos'è l'ingegneria del software e perché è importante;
- comprendere che lo sviluppo di diversi tipi di sistemi software può richiedere diverse tecniche di ingegneria del software;
- comprendere alcune questioni etiche e professionali importanti per gli ingegneri del software;
- introduzione di tre sistemi, di diversa tipologia, che verranno utilizzati come esempi in tutto il corso.

L'ingegneria del software è un campo che si occupa dello sviluppo professionale di software utilizzando teorie, metodi e strumenti specifici.

È una disciplina che ha acquisito sempre maggiore importanza in tutte le economie avanzate. Con l'espansione tecnologica e l'automazione di molti sistemi, sempre più operazioni dipendono da software avanzati. Questa dipendenza ha reso l'ingegneria del software un fattore determinante per lo sviluppo economico, influenzando una parte significativa del Prodotto Interno Lordo (PIL) dei paesi sviluppati.

Uno degli aspetti centrali dell'ingegneria del software riguarda la gestione dei costi. Quando si sviluppa un sistema informatico, le spese legate al software spesso superano di gran lunga quelle del hardware. Questo fenomeno è particolarmente evidente nei personal computer, dove il costo di licenze e aggiornamenti del software può superare quello delle componenti fisiche della macchina. Tuttavia, uno degli aspetti più onerosi è rappresentato dai costi di manutenzione del software, che superano quelli di sviluppo iniziale.

Poiché le nuove tecniche di ingegneria del software ci aiutano a costruire sistemi più grandi e complessi, le richieste devono essere costruite e fornite più rapidamente; sono richiesti sistemi più grandi e ancora più complessi. È abbastanza facile scrivere programmi per computer senza utilizzare metodi e tecniche di ingegneria del software. Molte aziende, infatti, si sono spostate verso lo sviluppo di software man mano che i loro prodotti e servizi si sono evoluti. Non utilizzando metodi di ingegneria del software nel loro lavoro quotidiano il loro software è spesso più costoso e meno affidabile di quanto dovrebbe essere.

1.1 Sviluppo software professionale

La stragrande maggioranza dello sviluppo software è un'attività professionale in cui il software viene sviluppato per scopi aziendali specifici, per essere incluso in altri dispositivi o come prodotti software come sistemi informativi. Il software professionale, destinato a essere utilizzato da qualcuno oltre al suo sviluppatore, è solitamente sviluppato da team piuttosto che da singoli individui e viene mantenuto e modificato nel corso della sua vita.

Molte persone pensano che il software sia semplicemente un altro termine per indicare i programmi per computer. Tuttavia, quando parliamo di ingegneria del software, il software non è solo

Domanda	Risposta
Cos'è il software?	Programmi per computer e documentazione associata. I prodotti software possono essere sviluppati per un cliente specifico o per il mercato generale.
Quali sono le caratteristiche di un buon software?	Un buon software deve fornire le funzionalità richieste e le prestazioni attese dall'utente, e deve essere mantenibile, affidabile e facile da usare.
Cos'è l'ingegneria del software?	L'ingegneria del software è una disciplina ingegneristica che si occupa di tutti gli aspetti della produzione di software.
Quali sono le attività fondamentali dell'ingegneria del software?	Specificazione del software, sviluppo del software, validazione del software ed evoluzione del software.
Qual è la differenza tra ingegneria del software e informatica?	L'informatica si concentra sulla teoria e sui fondamenti; l'ingegneria del software si occupa delle questioni pratiche dello sviluppo e della distribuzione di software utile.
Qual è la differenza tra ingegneria del software e ingegneria dei sistemi?	L'ingegneria dei sistemi si occupa di tutti gli aspetti dello sviluppo di sistemi basati su computer, compresi hardware, software e ingegneria dei processi. L'ingegneria del software è parte di questo processo più generale.
Quali sono le principali sfide dell'ingegneria del software?	Affrontare una crescente diversità, richieste di tempi di consegna ridotti e lo sviluppo di software affidabile.
Quali sono i costi dell'ingegneria del software?	Circa il 60% dei costi del software sono costi di sviluppo; il 40% sono costi di test. Per il software su misura, i costi di evoluzione spesso superano i costi di sviluppo.
Quali sono le migliori tecniche e metodi di ingegneria del software?	Anche se tutti i progetti software devono essere gestiti e sviluppati professionalmente, diverse tecniche sono appropriate per diversi tipi di sistema. Ad esempio, i giochi dovrebbero sempre essere sviluppati tramite una serie di prototipi, mentre i sistemi di controllo critici per la sicurezza richiedono una specifica completa e analizzabile. Non si può quindi affermare che un metodo sia migliore di un altro.
Quali differenze ha introdotto il Web nell'ingegneria del software?	Il Web ha portato alla disponibilità di servizi software e alla possibilità di sviluppare sistemi distribuiti basati su servizi. Lo sviluppo di sistemi Web ha portato importanti progressi nei linguaggi di programmazione e nel riutilizzo del software.

Tabella 1.1: Domande frequenti sul software

i programmi stessi, ma anche tutta la documentazione associata e i dati di configurazione necessari per far funzionare correttamente questi programmi.

Gli ingegneri del software si occupano dello sviluppo di prodotti software (cioè, software che possono essere venduti a un cliente). Esistono due tipi di prodotti software:

1. *Prodotti generici*: Questi sono sistemi autonomi prodotti da un'organizzazione di sviluppo e venduti sul mercato aperto a qualsiasi cliente che sia in grado di acquistarli. Esempi di questo tipo di prodotto includono software per PC come database, elaboratori di testi, pacchetti di disegno e strumenti di gestione di progetti. Include anche le cosiddette applicazioni verticali progettate per scopi specifici come i sistemi informativi per biblioteche, i sistemi contabili o i sistemi per la gestione delle cartelle cliniche dentali.
2. *Prodotti personalizzati*: Questi sono sistemi commissionati da un particolare cliente. Un'azienda di sviluppo software sviluppa il software appositamente per quel cliente. Esempi di questo tipo di software includono sistemi di controllo per dispositivi elettronici, sistemi scritti per supportare un particolare processo aziendale e sistemi di controllo del traffico aereo.

Una differenza importante tra questi tipi di software è che, nei prodotti generici, l'organizzazione che sviluppa il software controlla la specifica del software. Per i prodotti personalizzati, la specifica

viene solitamente sviluppata e controllata dall'organizzazione che acquista il software. I sviluppatori devono lavorare su quella specifica.

Caratteristiche del prodotto	Descrizione
Manutenibilità	Il software dovrebbe essere scritto in modo tale da poter evolvere per soddisfare le esigenze in continuo cambiamento dei clienti. Questo è un attributo critico perché il cambiamento del software è un requisito inevitabile in un ambiente aziendale in evoluzione.
Affidabilità e sicurezza	L'affidabilità del software include una serie di caratteristiche come affidabilità, sicurezza e sicurezza operativa. Il software affidabile non dovrebbe causare danni fisici o economici in caso di guasto del sistema. Gli utenti malintenzionati non dovrebbero essere in grado di accedere o danneggiare il sistema.
Efficienza	Il software non dovrebbe fare un uso inutile delle risorse di sistema, come la memoria e i cicli del processore. L'efficienza include quindi la reattività, il tempo di elaborazione, l'utilizzo della memoria, ecc.
Accettabilità	Il software deve essere accettabile per il tipo di utenti a cui è destinato. Ciò significa che deve essere comprensibile, utilizzabile e compatibile con altri sistemi che utilizzano.

Tabella 1.2: Attributi essenziali di un buon software

1.1.1 Ingegneria del software

L'ingegneria del software è una disciplina ingegneristica che si occupa di tutti gli aspetti della produzione di software, dalle prime fasi della specifica del sistema fino alla manutenzione del sistema dopo che è stato messo in uso. In questa definizione, ci sono due frasi chiave:

1. *Disciplina ingegneristica*: Gli ingegneri applicano teorie, metodi e strumenti quando appropriato, ma li usano in modo selettivo e cercano sempre soluzioni ai problemi, anche quando non esistono teorie e metodi applicabili. Inoltre, gli ingegneri riconoscono che devono lavorare con vincoli organizzativi e finanziari, quindi cercano soluzioni all'interno di questi limiti.
2. *Tutti gli aspetti della produzione del software*: L'ingegneria del software non si occupa solo dei processi tecnici di sviluppo software, ma include anche attività come la gestione dei progetti software e lo sviluppo di strumenti, metodi e teorie per supportare la produzione di software.

L'ingegneria del software è importante per due motivi:

1. Sempre di più, individui e società si affidano a sistemi software avanzati. Dobbiamo essere in grado di produrre sistemi affidabili e sicuri in modo economico e rapido.
2. A lungo termine, è di solito più economico utilizzare metodi e tecniche di ingegneria del software per i sistemi software piuttosto che scrivere semplicemente i programmi come fossero progetti di programmazione personali. Per la maggior parte dei tipi di sistemi, i costi maggiori sono quelli legati alle modifiche del software una volta che è stato messo in uso.

L'approccio sistematico utilizzato nell'ingegneria del software è talvolta chiamato processo software. Un processo software è una sequenza di attività che porta alla produzione di un prodotto software. Ci sono quattro attività fondamentali comuni a tutti i processi software:

1. *Specificazione del software*, in cui i clienti e gli ingegneri definiscono il software da produrre e i vincoli sul suo funzionamento.
2. *Sviluppo del software*, in cui il software viene progettato e programmato.
3. *Validazione del software*, in cui il software viene verificato per garantire che corrisponda ai requisiti del cliente.

4. *Evoluzione del software*, in cui il software viene modificato per riflettere le esigenze dei clienti e del mercato in evoluzione.

Ci sono quattro questioni generali che influenzano molti tipi di software:

1. *Eterogeneità*: Sempre più spesso i sistemi devono operare come sistemi distribuiti su reti che includono diversi tipi di computer e dispositivi mobili. Il software deve integrarsi con sistemi legacy scritti in diversi linguaggi di programmazione.
2. *Cambiamento aziendale e sociale*: Il mondo aziendale e sociale sta cambiando rapidamente. È necessario essere in grado di modificare il software esistente e sviluppare rapidamente nuovo software.
3. *Sicurezza e fiducia*: Poiché il software è strettamente connesso a tutti gli aspetti della nostra vita, è essenziale che ci si possa fidare di esso.
4. *Scala*: Il software deve essere sviluppato su una vasta gamma di scale, da sistemi embedded molto piccoli in dispositivi portatili o indossabili fino a sistemi basati su cloud e su scala Internet che servono una comunità globale.

1.1.2 Diversità nell'ingegneria del software

Non esiste un metodo universale di ingegneria del software adatto a tutti i sistemi e le aziende. Piuttosto, nel corso degli ultimi 50 anni si è evoluta una vasta gamma di metodi e strumenti di ingegneria del software, ognuno adatto a differenti contesti.

Uno dei fattori principali che influenzano la scelta dei metodi di ingegneria del software è il tipo di applicazione che si sta sviluppando. Di seguito sono riportati i principali tipi di applicazioni:

1. *Applicazioni stand-alone*: Sistemi che funzionano su un computer locale senza bisogno di una connessione di rete.
2. *Applicazioni interattive basate su transazioni*: Applicazioni che vengono eseguite su un computer remoto e a cui gli utenti accedono da PC o terminali. Come applicazioni web di e-commerce, sistemi aziendali o servizi cloud.
3. *Sistemi di controllo embedded*: Sistemi software che gestiscono dispositivi hardware. Numericamente, probabilmente ci sono più sistemi embedded di qualsiasi altro tipo di sistema.
4. *Sistemi batch*: Sistemi aziendali progettati per elaborare dati in grandi blocchi. Elaborano un gran numero di input individuali per creare output corrispondenti.
5. *Sistemi di intrattenimento*: Sistemi progettati per uso personale e destinati a intrattenere l'utente.
6. *Sistemi di modellazione e simulazione*: Sistemi utilizzati per modellare processi fisici o situazioni.
7. *Sistemi di raccolta dati*: Sistemi che raccolgono dati dall'ambiente attraverso sensori e inviano tali dati ad altri sistemi per l'elaborazione.
8. *Sistemi di sistemi*: Sistemi composti da altri sistemi software.

Nonostante questa diversità, esistono alcuni principi fondamentali dell'ingegneria del software che si applicano a tutti i tipi di sistemi:

1. *Processo di sviluppo gestito*: Il processo di sviluppo deve essere pianificato, e l'organizzazione deve avere un'idea chiara di cosa sarà prodotto e quando.
2. *Affidabilità e prestazioni*: Il software deve comportarsi come previsto senza guasti, essere sicuro e performante.
3. *Gestione delle specifiche*: È essenziale capire e gestire le aspettative dei clienti per fornire un sistema utile entro i limiti di tempo e budget.
4. *Riutilizzo delle risorse esistenti*: Dove possibile, è preferibile riutilizzare software esistente piuttosto che scrivere nuovo codice.

1.1.3 Ingegneria del software e Web

L'evoluzione del World Wide Web ha avuto un impatto significativo sullo sviluppo del software. Inizialmente, il Web fungeva principalmente come archivio di informazioni accessibile a tutti e non influenzava direttamente i sistemi software, che operavano localmente all'interno delle organizzazioni. Tuttavia, intorno all'anno 2000, il Web ha iniziato a cambiare con l'aggiunta di maggiore funzionalità ai browser, permettendo la creazione di sistemi web-based accessibili tramite un browser web anziché un'interfaccia utente specifica. Questo ha favorito la nascita di nuovi prodotti software e servizi innovativi, spesso finanziati dalla pubblicità, senza che gli utenti pagassero direttamente per l'accesso.

La capacità dei browser di eseguire piccoli programmi e gestire alcune elaborazioni locali ha portato all'evoluzione del software aziendale. Invece di distribuire e installare il software sui PC degli utenti, esso veniva implementato su server web, riducendo notevolmente i costi di aggiornamento e distribuzione. Questo cambiamento ha spinto molte aziende a passare all'interazione web-based con i propri sistemi.

Negli ultimi anni si è diffusa l'idea secondo cui il software non viene eseguito localmente sui computer ma su cloud computing accessibili via Internet. I cloud computing sono grandi insiemi di sistemi informatici collegati, condivisi da molti utenti, che pagano in base all'uso del software.

Questi cambiamenti hanno modificato anche il modo in cui i sistemi web-based vengono progettati:

1. Il riutilizzo di componenti preesistenti è diventato il metodo dominante per costruire sistemi web-based. Quando si costruiscono questi sistemi, si pensa a come assemblarli da componenti software e sistemi preesistenti.
2. È ormai riconosciuto che non è pratico definire in anticipo tutte le specifiche per questi sistemi. Pertanto, devono essere sviluppati e rilasciati in modo incrementale.
3. Il software può essere implementato utilizzando l'ingegneria del software orientata ai servizi, in cui i componenti software sono servizi Web autonomi.
4. Sono emerse tecnologie di sviluppo di interfacce come AJAX e HTML5 che supportano la creazione di interfacce avanzate all'interno di un browser web.

Nonostante queste differenze, i concetti fondamentali dell'ingegneria del software rimangono applicabili anche ai sistemi web-based.

1.2 Etica dell'ingegneria del software

L'ingegneria del software comporta responsabilità più ampie della semplice applicazione di competenze tecniche. Gli ingegneri del software devono comportarsi in modo onesto ed eticamente responsabile se vogliono essere rispettati come professionisti. Il comportamento etico è più che semplicemente rispettare la legge, ma implica seguire una serie di principi che sono moralmente corretti. Tuttavia, ci sono aree in cui gli standard di comportamento accettabili non sono vincolati dalla legge, ma dalla più tenue nozione di responsabilità professionale. Alcuni di questi includono:

1. *Riservatezza*: Dovresti normalmente rispettare la riservatezza dei tuoi datori di lavoro o clienti, indipendentemente dal fatto che sia stato firmato un accordo di riservatezza formale o meno.
2. *Competenza*: Non dovresti rappresentare falsamente il tuo livello di competenza. Non dovresti accettare consapevolmente lavori che siano al di fuori delle tue competenze.
3. *Diritti di proprietà intellettuale*: Dovresti essere a conoscenza delle leggi locali che governano l'uso della proprietà intellettuale, come brevetti e diritti d'autore. Dovresti prestare attenzione a garantire che la proprietà intellettuale dei datori di lavoro e dei clienti sia protetta.
4. *Uso improprio dei computer*: Non dovresti utilizzare le tue competenze tecniche per abusare dei computer altrui. L'uso improprio dei computer varia da comportamenti relativamente banali (giocare su una macchina del datore di lavoro) a questioni estremamente serie (diffusione di virus o altri malware).

Le società e le istituzioni professionali hanno un ruolo importante nel definire gli standard etici. Organizzazioni come l'ACM (Association for Computing Machinery), l'IEEE (Institute of Electrical and Electronic Engineers) e la British Computer Society pubblicano un codice di condotta professionale o un codice etico. I membri di queste organizzazioni si impegnano a seguire tale codice quando si iscrivono. Questi codici di condotta riguardano generalmente comportamenti etici fondamentali. Il Codice contiene otto Principi relativi al comportamento e alle decisioni prese dai professionisti dell'ingegneria del software, inclusi praticanti, educatori, manager, supervisori e responsabili politici, oltre a tirocinanti e studenti della professione.

I computer hanno un ruolo centrale e in crescita nel commercio, nell'industria, nel governo, nella medicina, nell'istruzione, nell'intrattenimento e nella società in generale. Gli ingegneri del software sono coloro che contribuiscono, attraverso la partecipazione diretta o l'insegnamento, all'analisi, alla specifica, alla progettazione, allo sviluppo, alla certificazione, alla manutenzione e al collaudo dei sistemi software. A causa dei loro ruoli nello sviluppo dei sistemi software, gli ingegneri del software hanno significative opportunità di fare del bene o causare danno, di abilitare gli altri a fare del bene o causare danno, o di influenzare gli altri a fare del bene o causare danno. Per garantire, per quanto possibile, che i loro sforzi vengano utilizzati per il bene, gli ingegneri del software devono impegnarsi a fare dell'ingegneria del software una professione benefica e rispettata. In conformità a tale impegno, gli ingegneri del software devono aderire al seguente Codice di Etica e Pratica Professionale.

Codice Etico e Pratica Professionale dell'Ingegneria del Software

Gruppo di Lavoro Congiunto ACM/IEEE-CS sull'Etica e le Pratiche Professionali nell'Ingegneria del Software

PREAMBOLO

La versione breve del codice riassume le aspirazioni a un alto livello di astrazione; le clausole incluse nella versione completa forniscono esempi e dettagli su come queste aspirazioni cambiano il nostro modo di agire come professionisti dell'ingegneria del software. Senza le aspirazioni, i dettagli possono diventare legalistici e tediosi; senza i dettagli, le aspirazioni possono risultare altisonanti ma vuote; insieme, aspirazioni e dettagli formano un codice coerente.

Gli ingegneri del software si impegneranno a fare dell'analisi, della specifica, della progettazione, dello sviluppo, del collaudo e della manutenzione del software una professione benefica e rispettata. In conformità con il loro impegno per la salute, la sicurezza e il benessere del pubblico, gli ingegneri del software devono aderire ai seguenti Otto Principi:

1. **PUBBLICO** — Gli ingegneri del software agiranno in modo coerente con l'interesse pubblico.
2. **CLIENTE E DATORE DI LAVORO** — Gli ingegneri del software agiranno in un modo che sia nel miglior interesse del loro cliente e datore di lavoro, in coerenza con l'interesse pubblico.
3. **PRODOTTO** — Gli ingegneri del software garantiranno che i loro prodotti e le modifiche correlate soddisfino i più elevati standard professionali possibili.
4. **GIUDIZIO** — Gli ingegneri del software manterranno integrità e indipendenza nel loro giudizio professionale.
5. **GESTIONE** — I manager e i leader nell'ingegneria del software si iscriveranno e promuoveranno un approccio etico alla gestione dello sviluppo e della manutenzione del software.
6. **PROFESSIONE** — Gli ingegneri del software promuoveranno l'integrità e la reputazione della professione in coerenza con l'interesse pubblico.
7. **COLLEGHI** — Gli ingegneri del software saranno giusti e solidali nei confronti dei loro colleghi.
8. **SE STESSI** — Gli ingegneri del software parteciperanno a un apprendimento continuo riguardante la pratica della loro professione e promuoveranno un approccio etico alla pratica della professione.

1.3 Casi di studio

Per illustrare i concetti dell'ingegneria del software, utilizzo esempi tratti da quattro diverse tipologie di sistema.

1. *Una pompa per insulina personale*: Un sistema incorporato in una pompa per insulina utilizzata dai diabetici per mantenere il controllo della glicemia.
2. *Un sistema di gestione dei casi di salute mentale*: Mentcare. Un sistema utilizzato per conservare i registri delle persone che ricevono cure per problemi di salute mentale.
3. *Una stazione meteorologica per aree selvagge*: Un sistema di raccolta dati che raccoglie dati sulle condizioni meteorologiche in aree remote.
4. *iLearn*: un ambiente di apprendimento digitale. Un sistema per supportare l'apprendimento nelle scuole.

Sistema di controllo della pompa di insulina

Una pompa di insulina è un sistema medico che simula il funzionamento del pancreas. Il software che controlla questo sistema è un sistema embedded che raccoglie informazioni da un sensore di glicemia e calcola la quantità di insulina necessaria da iniettare. Il calcolo basato sulla velocità di variazione dei livelli di glicemia. Quindi una pompa somministra una dose controllata di insulina a un utente.

Un controllo irregolare può portare a livelli molto bassi di glucosio nel sangue (se c'è troppa insulina) o a livelli molto alti di zucchero nel sangue (se c'è troppo poca insulina). Un basso livello di glucosio nel sangue è, a breve termine, una condizione più grave poiché può comportare malfunzionamenti temporanei del cervello e, infine, incoscienza e morte. A lungo termine, tuttavia, livelli costantemente elevati di glucosio nel sangue possono portare a danni agli occhi, danni ai reni e problemi cardiaci.

La figura 1.1 mostra i componenti hardware e l'organizzazione della pompa di insulina. Il sensore del sangue misura la conduttività elettrica del sangue in diverse condizioni, questi valori possono essere correlati al livello di zucchero nel sangue. La pompa di insulina fornisce un'unità di insulina in risposta a un singolo impulso proveniente da un controller. Pertanto, per somministrare 10 unità di insulina, il controller invia 10 impulsi alla pompa. La figura 1.2 è un modello di attività UML (Unified Modeling Language) che illustra come il software trasforma un livello di zucchero nel sangue in una sequenza di comandi che azionano la pompa di insulina.

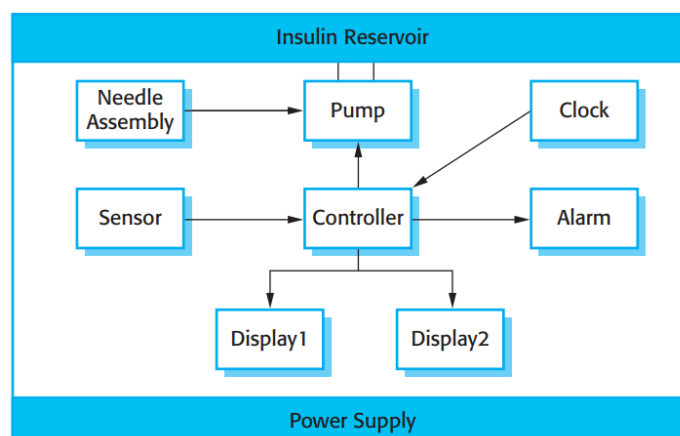


Figura 1.1: Hardware per pompe di insulina

Chiaramente, questo è un sistema critico per la sicurezza. Questo sistema deve quindi soddisfare due requisiti essenziali di alto livello:

1. Il sistema deve essere disponibile per somministrare insulina quando necessario.

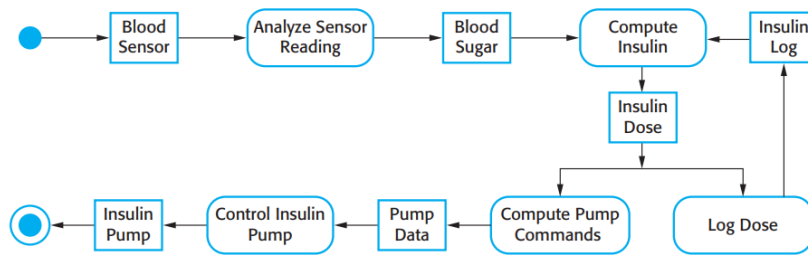


Figura 1.2: Modello di attività della pompa per insulina

2. Il sistema deve funzionare in modo affidabile e fornire la giusta quantità di insulina per controllare il livello attuale di zucchero nel sangue.

Il sistema deve quindi essere progettato e implementato per garantire che soddisfi sempre questi requisiti.

Sistema informativo per i pazienti per l'assistenza sanitaria mentale

Un sistema informativo per pazienti a supporto della cura della salute mentale (il sistema Mentcare) è un sistema informativo medico che gestisce informazioni sui pazienti affetti da problemi di salute mentale e sui trattamenti che hanno ricevuto. La maggior parte dei pazienti con problemi di salute mentale non richiede un trattamento ospedaliero dedicato, ma deve recarsi regolarmente presso cliniche specialistiche dove possono incontrare un medico che ha una conoscenza approfondita dei loro problemi. Per facilitare l'accesso dei pazienti, queste cliniche non sono gestite solo negli ospedali. Possono anche essere tenute presso ambulatori medici locali o centri comunitari.

Il sistema Mentcare (Figura 1.3) è un sistema informativo per pazienti destinato all'uso nelle cliniche. Utilizza un database centralizzato di informazioni sui pazienti, ma è stato anche progettato per funzionare su un laptop, in modo da poter essere accessibile e utilizzato in luoghi che non hanno connettività di rete sicura. Quando i sistemi locali hanno accesso alla rete sicura, utilizzano le informazioni sui pazienti presenti nel database, ma possono scaricare e utilizzare copie locali delle cartelle cliniche quando sono disconnessi.

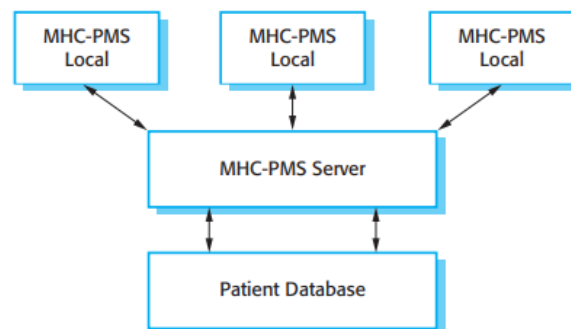


Figura 1.3: L'organizzazione del MHC-PMS

Questo sistema ha due scopi principali:

1. Generare informazioni gestionali che permettano ai manager del servizio sanitario di valutare le prestazioni rispetto agli obiettivi locali e governativi.
2. Fornire al personale medico informazioni tempestive per supportare il trattamento dei pazienti.

Le caratteristiche principali del sistema sono:

1. *Gestione individuale delle cure:* I medici possono creare cartelle cliniche per i pazienti, modificare le informazioni nel sistema, visualizzare la storia clinica dei pazienti, ecc. Il sistema supporta riassunti dei dati in modo che i medici che non hanno mai incontrato un paziente possano rapidamente informarsi sui principali problemi e sui trattamenti prescritti.

2. *Monitoraggio dei pazienti:* Il sistema monitora regolarmente le cartelle dei pazienti coinvolti nel trattamento e invia avvisi se vengono rilevati problemi potenziali.
3. *Rapporti amministrativi:* Il sistema genera rapporti gestionali mensili che mostrano il numero di pazienti trattati in ciascuna clinica, il numero di pazienti che sono entrati e usciti dal sistema di cure, il numero di pazienti sezionati, i farmaci prescritti e i loro costi, ecc.

Come in tutti i sistemi medici, la privacy è un requisito critico del sistema. È essenziale che le informazioni sui pazienti siano riservate e non vengano mai divulgate a nessuno, tranne che al personale medico autorizzato e ai pazienti stessi. Il sistema Mentcare è anche un sistema critico per la sicurezza. Alcune malattie mentali possono rendere i pazienti suicidi o pericolosi per altre persone. Ogni volta che possibile, il sistema dovrebbe avvisare il personale medico riguardo a pazienti potenzialmente suicidi o pericolosi. Il sistema deve essere disponibile quando necessario; in caso contrario, la sicurezza potrebbe essere compromessa e potrebbe essere impossibile prescrivere i farmaci corretti ai pazienti.

Stazione meteorologica in zone selvagge

Per aiutare a monitorare il cambiamento climatico e migliorare l'accuratezza delle previsioni meteorologiche nelle aree remote, il governo di un paese con grandi aree selvagge decide di installare diverse centinaia di stazioni meteorologiche in zone isolate. Queste stazioni meteorologiche raccolgono dati da una serie di strumenti che misurano temperatura, pressione, luce solare, precipitazioni, velocità e direzione del vento.

Le stazioni meteorologiche includono strumenti che misurano parametri meteorologici come la velocità e la direzione del vento, le temperature del suolo e dell'aria, la pressione barometrica e le precipitazioni in un periodo di 24 ore. Ognuno di questi strumenti è controllato da un sistema software che rileva periodicamente i parametri e gestisce i dati raccolti dagli strumenti.

Le stazioni meteorologiche selvagge fanno parte di un sistema più grande (Figura 1.4), che è un sistema di informazioni meteorologiche che raccoglie dati dalle stazioni meteorologiche e li rende disponibili per altri sistemi per l'elaborazione.

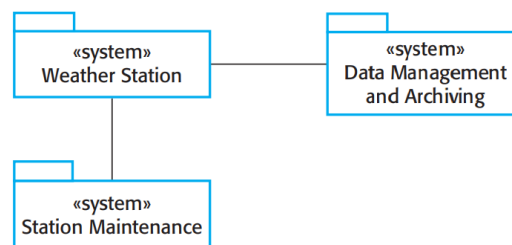


Figura 1.4: L'ambiente della stazione meteorologica

I sistemi nella Figura 1.4 sono:

1. *Il sistema della stazione meteorologica:* Questo sistema è responsabile della raccolta dei dati meteorologici, dell'esecuzione di alcune elaborazioni iniziali dei dati e della trasmissione al sistema di gestione dei dati.
2. *Il sistema di gestione e archiviazione dei dati:* Questo sistema raccoglie i dati da tutte le stazioni meteorologiche selvagge, esegue l'elaborazione e l'analisi dei dati e archivia i dati in una forma recuperabile da altri sistemi.
3. *Il sistema di manutenzione delle stazioni:* Questo sistema può comunicare tramite satellite con tutte le stazioni meteorologiche selvagge per monitorare lo stato di salute di questi sistemi e fornire rapporti sui problemi.

Il software della stazione non si occupa quindi solo della raccolta dei dati, ma deve anche:

1. Monitorare gli strumenti, l'hardware di alimentazione e di comunicazione e segnalare i guasti al sistema di gestione.

2. Gestire l'alimentazione del sistema, assicurandosi che le batterie vengano caricate ogni volta che le condizioni ambientali lo permettono, ma anche che i generatori vengano spenti in condizioni meteorologiche potenzialmente dannose, come venti forti.
3. Consentire una riconfigurazione dinamica, in cui parti del software vengono sostituite con nuove versioni e gli strumenti di riserva vengono integrati nel sistema in caso di guasto del sistema.

Ambiente di apprendimento digitale

Un ambiente di apprendimento digitale è una piattaforma in cui può essere integrato un insieme di strumenti generici e specificamente progettati per l'apprendimento, oltre a una serie di applicazioni orientate ai bisogni degli studenti che utilizzano il sistema. Gli strumenti inclusi in ogni versione dell'ambiente vengono scelti dagli insegnanti e dagli studenti per soddisfare le loro specifiche esigenze. Questi strumenti possono includere applicazioni generali come fogli di calcolo, applicazioni di gestione dell'apprendimento come un ambiente virtuale di apprendimento (VLE) per gestire la consegna dei compiti e le valutazioni, giochi e simulazioni.

La Figura 1.5 è un modello architetturale ad alto livello di un ambiente di apprendimento digitale (iLearn).

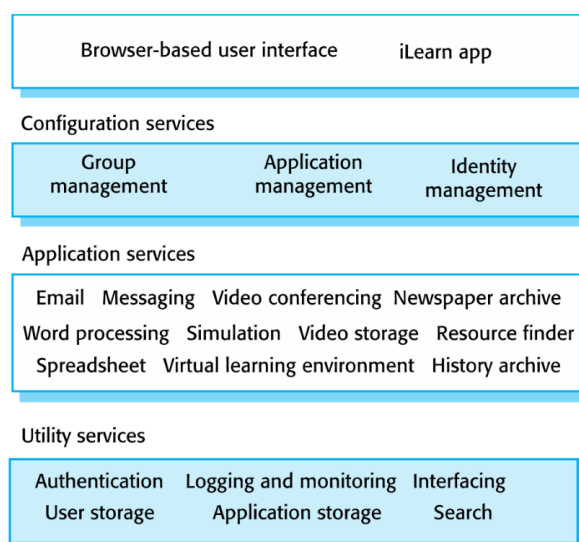


Figura 1.5: L'architettura di un ambiente di apprendimento digitale

Il sistema è un sistema orientato ai servizi, in cui tutti i componenti del sistema sono considerati servizi sostituibili. Ci sono tre tipi di servizio nel sistema:

1. *Servizi di utilità:* Forniscono funzionalità di base indipendenti dalle applicazioni e possono essere utilizzati da altri servizi del sistema.
2. *Servizi applicativi:* Forniscono applicazioni specifiche come email, conferenze, condivisione di foto, ecc., e l'accesso a contenuti educativi specifici come film scientifici o risorse storiche.
3. *Servizi di configurazione:* Sono utilizzati per adattare l'ambiente con un set specifico di servizi applicativi e per definire come i servizi vengono condivisi tra studenti, insegnanti e genitori.

L'ambiente è stato progettato in modo che i servizi possano essere sostituiti man mano che nuovi servizi diventano disponibili e per fornire versioni diverse del sistema adatte all'età degli utenti. Ciò significa che il sistema deve supportare due livelli di integrazione dei servizi:

1. *Servizi integrati:* Sono servizi che offrono un'API (interfaccia di programmazione delle applicazioni) e che possono essere accessibili da altri servizi tramite tale API. È quindi possibile una comunicazione diretta tra i servizi.

2. *Servizi indipendenti*: Sono servizi che vengono semplicemente accessibili tramite un'interfaccia del browser e che operano in modo indipendente dagli altri servizi. Le informazioni possono essere condivise con altri servizi solo tramite azioni esplicite dell'utente, come il copia e incolla; potrebbe essere richiesta una nuova autenticazione per ciascun servizio indipendente.

1.4 Punti chiave

- L'ingegneria del software è una disciplina ingegneristica che si occupa di tutti gli aspetti della produzione del software.
- Il software non è solo un programma o programmi, ma include anche tutta la documentazione elettronica necessaria per gli utenti del sistema, il personale di assicurazione della qualità e gli sviluppatori. Gli attributi essenziali di un prodotto software sono la manutenibilità, l'affidabilità e la sicurezza, l'efficienza e l'accettabilità.
- Il processo di sviluppo del software include tutte le attività coinvolte nello sviluppo del software. Le attività ad alto livello di specifica, sviluppo, validazione ed evoluzione fanno parte di tutti i processi software.
- Esistono molti tipi diversi di sistema, e ciascuno richiede strumenti e tecniche di ingegneria del software appropriati per il proprio sviluppo. Poche, se non nessuna, tecniche specifiche di progettazione e implementazione sono applicabili a tutti i tipi di sistema.
- Le idee fondamentali dell'ingegneria del software sono applicabili a tutti i tipi di sistema software. Questi fondamenti includono processi software gestiti, affidabilità e sicurezza del software, ingegneria dei requisiti e riuso del software.
- Gli ingegneri del software hanno responsabilità nei confronti della professione ingegneristica e della società. Non dovrebbero occuparsi solo di questioni tecniche, ma dovrebbero essere consapevoli delle questioni etiche che influenzano il loro lavoro.
- Le società professionali pubblicano codici di condotta che incorporano standard etici e professionali. Questi stabiliscono gli standard di comportamento attesi dai loro membri.

Capitolo 2

Processi software

Un **processo software** è un insieme di attività correlate che porta alla produzione di un sistema software.

Come già discusso, esistono diversi tipi di sistemi software e non esiste un metodo di ingegneria del software universale applicabile a tutti. Nonostante l'esistenza di molti processi software diversi, tutti devono includere, in qualche forma, le seguenti quattro attività fondamentali dell'ingegneria del software:

1. *Specificazione del software*: Definire la funzionalità del software e le limitazioni operative.
2. *Sviluppo del software*: Produrre il software in base alla specifica.
3. *Validazione del software*: Validare il software per garantire che soddisfi le esigenze del cliente.
4. *Evoluzione del software*: Evolvere il software per soddisfare i bisogni in continua evoluzione del cliente.

Quando descriviamo i processi, parliamo solitamente delle attività, dell'ordine in cui vengono eseguite e del ruolo delle persone coinvolte, dei prodotti generati e delle condizioni che influenzano l'esecuzione delle attività:

1. *Prodotti o deliverable*: Sono i risultati di un'attività di processo.
2. *Ruoli*: Rappresentano le responsabilità delle persone coinvolte nel processo.
3. *Precondizioni e postcondizioni*: Condizioni che devono essere verificate prima e dopo l'esecuzione di un'attività di processo o la produzione di un prodotto.

Nei processi "plan-driven", tutte le attività di processo sono pianificate in anticipo e i progressi vengono misurati rispetto a questo piano. Nei processi agili, la pianificazione è incrementale e continua durante lo sviluppo del software, rendendo più facile adattare il processo a nuovi requisiti. Ogni approccio è adatto a diversi tipi di software; in generale, per grandi sistemi è necessario trovare un equilibrio tra processi pianificati e agili.

2.1 Modelli di processo software

Un modello di processo software (chiamato a volte Ciclo di Vita del Software o SDLC) è una rappresentazione semplificata di un processo software. Ogni modello di processo rappresenta il processo da una prospettiva particolare e fornisce quindi solo informazioni parziali.

I modelli generali di processo che analizzeremo sono:

1. *Modello a cascata (Waterfall model)*: Questo modello suddivide le attività fondamentali del processo (specificazione, sviluppo, validazione ed evoluzione) in fasi di processo separate come specifica dei requisiti, progettazione del software, implementazione e testing.
2. *Sviluppo incrementale*: In questo approccio, le attività di specificazione, sviluppo e validazione si alternano. Il sistema viene sviluppato come una serie di versioni (incrementi), con ogni versione che aggiunge funzionalità alla precedente.

3. *Integrazione e configurazione*: Questo approccio si basa sulla disponibilità di componenti o sistemi riutilizzabili.

La maggior parte dei processi software pratici si basa su un modello generale, ma spesso incorpora le migliori caratteristiche di tutti i modelli.

2.1.1 Modello a cascata

Questo modello presenta il processo di sviluppo software come una serie di fasi, come mostrato nella Figura 2.1. A causa del flusso a cascata da una fase all'altra, questo modello è conosciuto come modello a cascata. È un esempio di processo plan-driven: in teoria, si pianificano e si programmano tutte le attività del processo prima di iniziare lo sviluppo del software.

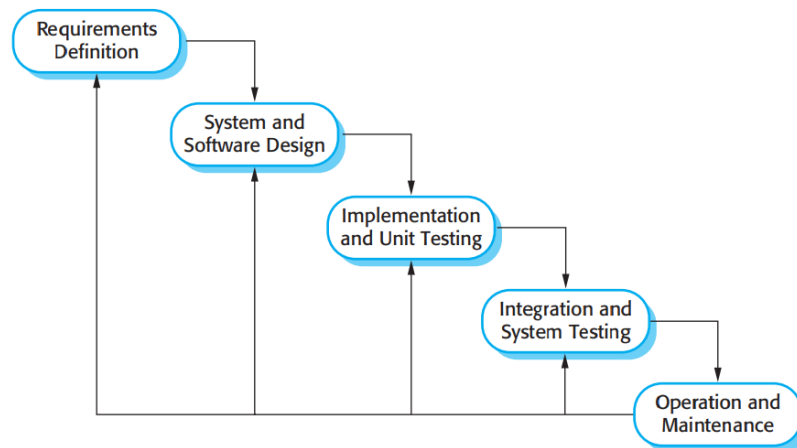


Figura 2.1: Il modello a cascata

Le fasi del modello a cascata riflettono direttamente le attività fondamentali dello sviluppo software:

1. Analisi e definizione dei requisiti.
2. Progettazione di sistema e software.
3. Implementazione e test unitari.
4. Integrazione e test di sistema.
5. Operazione e manutenzione.

In linea di principio, il risultato di ciascuna fase del modello a cascata è uno o più documenti approvati ("firmati"). La fase successiva non dovrebbe iniziare finché la fase precedente non è completata.

La suddivisione rigida del progetto in fasi distinte rende difficile rispondere ai cambiamenti nei requisiti del cliente. Pertanto, questo modello è appropriato solo quando i requisiti sono ben compresi e i cambiamenti saranno piuttosto limitati durante il processo di progettazione. Pochi sistemi aziendali hanno requisiti stabili.

Il modello a cascata è utilizzato principalmente per grandi progetti di ingegneria dei sistemi in cui un sistema viene sviluppato in più siti. In queste circostanze, la natura pianificata del modello a cascata aiuta a coordinare il lavoro.

2.1.2 Sviluppo incrementale

Lo sviluppo incrementale si basa sull'idea di sviluppare un'implementazione iniziale, ottenere feedback dagli utenti e da altri, e far evolvere il software attraverso diverse versioni fino a quando non è stato sviluppato il sistema richiesto (Figura 2.2).

Lo sviluppo incrementale presenta tre importanti vantaggi rispetto al modello a cascata:

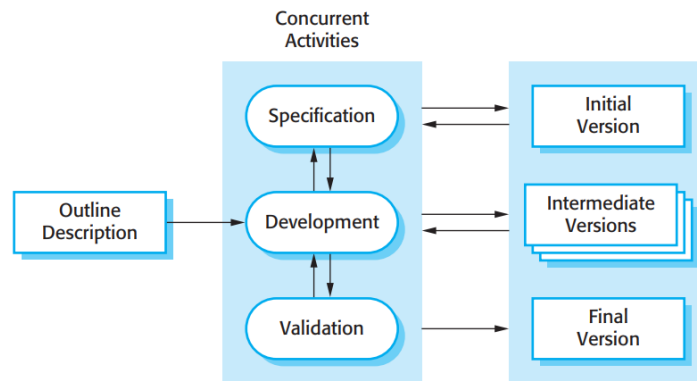


Figura 2.2: Sviluppo incrementale

1. *Riduzione dei costi per le modifiche ai requisiti:* L'ammontare di analisi e documentazione che deve essere rifatta è significativamente inferiore rispetto a quanto richiesto dal modello a cascata.
2. *Facilità di ottenere feedback dai clienti:* I clienti possono commentare le dimostrazioni del software e vedere quanto è stato implementato.
3. *Consegna e distribuzione precoce di software utile:* I clienti possono utilizzare e trarre valore dal software prima rispetto a quanto possibile con un processo a cascata.

Da una prospettiva gestionale, l'approccio incrementale presenta due problemi:

1. *Il processo non è visibile:* I manager hanno bisogno di risultati regolari per misurare i progressi. Se i sistemi vengono sviluppati rapidamente, non è economicamente efficace produrre documenti che riflettano ogni versione del sistema.
2. *La struttura del sistema tende a degradare:* I cambiamenti regolari portano a codice disordinato mentre vengono aggiunte nuove funzionalità in qualsiasi modo possibile. Diventa sempre più difficile e costoso aggiungere nuove caratteristiche a un sistema.

2.1.3 Integrazione e configurazione

Nella maggior parte dei progetti software, c'è un certo riutilizzo del software. Questo riutilizzo informale avviene indipendentemente dal processo di sviluppo utilizzato. Gli approcci orientati al riutilizzo si basano su un insieme di componenti software riutilizzabili e su un framework di integrazione per la composizione di questi componenti.

Tre tipi di componenti software sono frequentemente riutilizzati:

1. Sistemi applicativi stand-alone che sono configurati per l'uso in un ambiente particolare.
2. Collezioni di oggetti sviluppate come componente o come pacchetto da integrare con un framework di componenti, come il framework Java Spring (Wheeler e White 2013).
3. Web services che sono sviluppati secondo standard di servizio e che sono disponibili per l'invocazione remota su Internet.

La Figura 2.3 mostra un modello di processo generale per lo sviluppo basato sul riutilizzo, incentrato su integrazione e configurazione.

Le fasi di questo processo sono:

1. Specificazione dei requisiti.
2. Scoperta e valutazione del software.
3. Raffinamento dei requisiti.
4. Configurazione del sistema applicativo.
5. Adattamento e integrazione dei componenti.

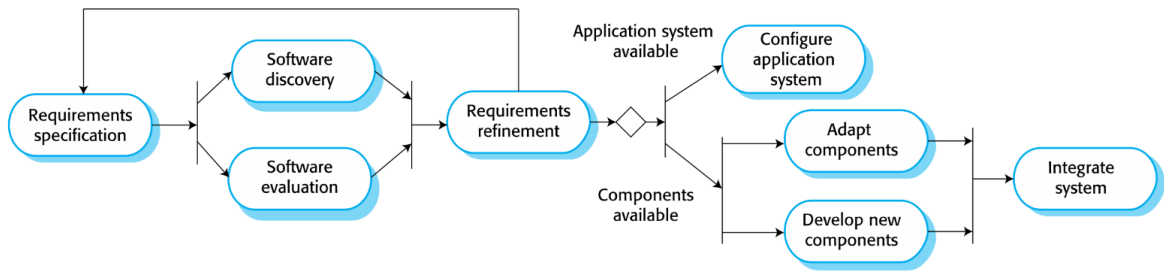


Figura 2.3: Ingegneria del software orientata al riutilizzo

L'ingegneria del software orientata al riutilizzo, basata su configurazione e integrazione, ha il vantaggio evidente di ridurre la quantità di software da sviluppare, riducendo così costi e rischi. Di solito porta anche a una consegna più rapida del software. Tuttavia, i compromessi sui requisiti sono inevitabili, e questo può portare a un sistema che non soddisfa le reali esigenze degli utenti. Inoltre, si perde parte del controllo sull'evoluzione del sistema poiché le nuove versioni dei componenti riutilizzabili non sono sotto il controllo dell'organizzazione che li utilizza.

2.2 Attività di processo

I veri processi software sono sequenze intrecciate di attività tecniche, collaborative e manageriali, con l'obiettivo complessivo di specificare, progettare, implementare e testare un sistema software.

Le quattro attività di processo di base di specifica, sviluppo, convalida ed evoluzione sono organizzate in modo diverso nei diversi processi di sviluppo. Nel modello waterfall, sono organizzate in sequenza, mentre nello sviluppo incrementale sono intrecciate.

2.2.1 Specificazione del software

La specificazione del software, o ingegneria dei requisiti, è il processo di comprensione e definizione dei servizi richiesti dal sistema e identificazione delle limitazioni sul funzionamento e sullo sviluppo del sistema.

Il processo di ingegneria dei requisiti (Figura 2.4) ha lo scopo di produrre un documento di requisiti concordato che specifichi un sistema in grado di soddisfare le esigenze degli stakeholder.

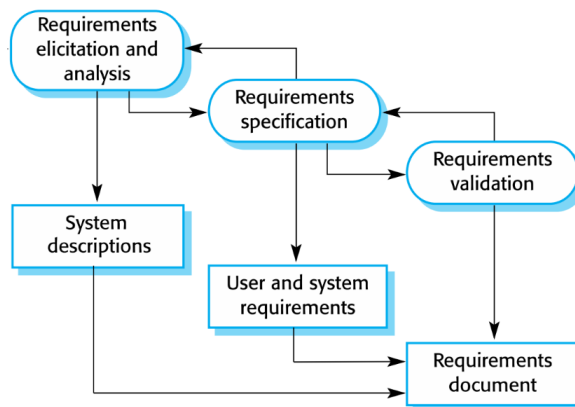


Figura 2.4: Il processo di ingegneria dei requisiti

Ci sono tre attività principali nel processo di ingegneria dei requisiti:

1. *Elicitazione e analisi dei requisiti*: Questo è il processo di derivazione dei requisiti del sistema attraverso l'osservazione dei sistemi esistenti, discussioni con potenziali utenti e acquirenti, analisi dei compiti, e così via.
2. *Specificazione dei requisiti*: La specificazione dei requisiti è l'attività di tradurre le informazioni raccolte durante l'analisi dei requisiti in un documento che definisce un insieme di requisiti.

3. *Validazione dei requisiti*: Questa attività verifica i requisiti per realismo, coerenza e completezza.

2.2.2 Progettazione e implementazione del software

La fase di implementazione dello sviluppo software è il processo di sviluppo di un sistema eseguibile da consegnare al cliente. A volte ciò comporta attività separate di progettazione software e programmazione. Tuttavia, se viene utilizzato un approccio agile allo sviluppo, progettazione e implementazione sono intrecciate.

Un design software è una descrizione della struttura del software da implementare, dei modelli e delle strutture dei dati utilizzati dal sistema, delle interfacce tra i componenti del sistema e, a volte, degli algoritmi utilizzati.

La Figura 2.5 è un modello astratto del processo di design che mostra gli input al processo di design, le attività del processo e le uscite del processo.

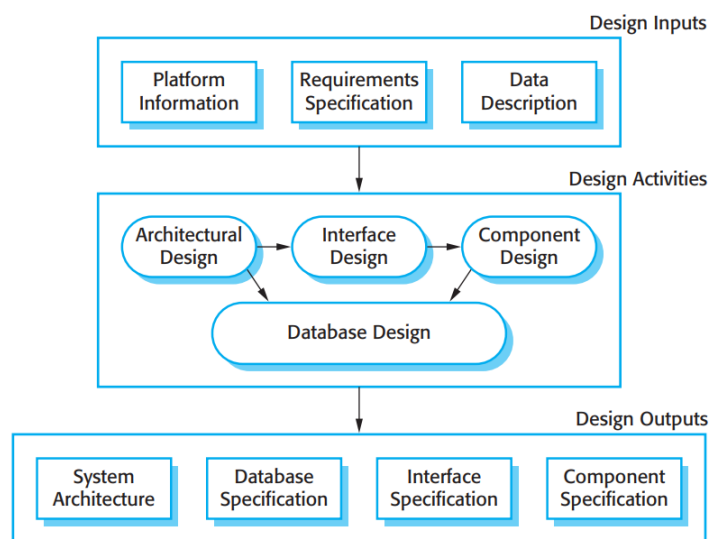


Figura 2.5: Un modello generale del processo di progettazione

La Figura 2.5 mostra quattro attività che possono far parte del processo di design per sistemi informativi:

1. *Design architetturale*: identificare la struttura complessiva del sistema, i componenti principali (a volte chiamati sottosistemi o moduli), le loro relazioni e come sono distribuiti.
2. *Design del database*: progettare le strutture dei dati del sistema e come queste devono essere rappresentate in un database.
3. *Design dell'interfaccia*: definire le interfacce tra i componenti del sistema.
4. *Selezione e design dei componenti*: cercare componenti riutilizzabili e, se non sono disponibili componenti adatti, progettare nuovi componenti software.

Lo sviluppo di un programma per implementare un sistema segue naturalmente dalla progettazione del sistema. Il software viene implementato sviluppando uno o più programmi o configurando un sistema applicativo. Progettazione e implementazione sono attività intervallate per la maggior parte dei tipi di sistema software. La programmazione è un'attività individuale, e non esiste un processo generale che viene solitamente seguito. Normalmente, i programmatori effettuano alcuni test del codice che hanno sviluppato. Questo spesso rivela difetti del programma (bug) che devono essere rimossi dal programma. Trovare e correggere i difetti del programma è chiamato debugging.

2.2.3 Validazione del software

La validazione del software o, più in generale, verifica e validazione (V&V), ha lo scopo di dimostrare che un sistema sia conforme alla sua specifica e soddisfi le aspettative del cliente del sistema. Il testing del programma, in cui il sistema viene eseguito utilizzando dati di test simulati, è la principale

tecnica di validazione. La validazione può anche comportare il controllo dei processi, come ispezioni e revisioni, in ogni fase del processo software, dalla definizione dei requisiti degli utenti allo sviluppo del programma. Tuttavia, la maggior parte del tempo e dell'impegno dedicati alla V&V è spesa nel testing del programma.

La Figura 2.6 mostra un processo di testing in tre fasi in cui i componenti del sistema vengono testati individualmente e poi il sistema integrato viene testato.

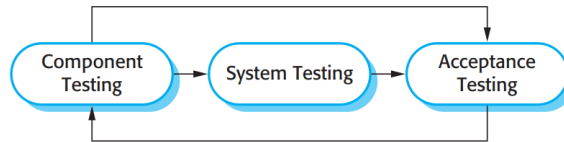


Figura 2.6: Fasi di test

Le fasi del processo di testing sono:

1. *Testing dei componenti*: ogni componente viene testato indipendentemente. I componenti possono essere entità semplici come funzioni o classi di oggetti, o possono essere raggruppamenti coerenti di queste entità.
2. *Testing del sistema*: questo processo si concentra sulla ricerca di errori che derivano da interazioni inaspettate tra i componenti e problemi con le interfacce dei componenti. Si occupa anche di dimostrare che il sistema soddisfa i requisiti funzionali e non funzionali, e di testare le proprietà emergenti del sistema.
3. *Testing da parte del cliente*: il sistema viene testato dal cliente del sistema (o da un cliente potenziale) piuttosto che con dati di test simulati. Il testing da parte del cliente può anche rivelare problemi relativi ai requisiti, dove le funzionalità del sistema non soddisfano realmente le esigenze degli utenti o le prestazioni del sistema sono inaccettabili.

Quando viene utilizzato un processo di plan-driven, il testing è guidato da un insieme di piani di test. Un team di tester indipendente lavora a partire da questi piani di test, che sono stati sviluppati dalla specifica e dal design del sistema. La Figura 2.7 illustra come i piani di test siano il collegamento tra le attività di testing e di sviluppo. Questo è talvolta chiamato il modello a V dello sviluppo (ruotolo di lato per vedere la V). Il modello a V mostra le attività di validazione del software che corrispondono a ciascuna fase del modello di processo a cascata.

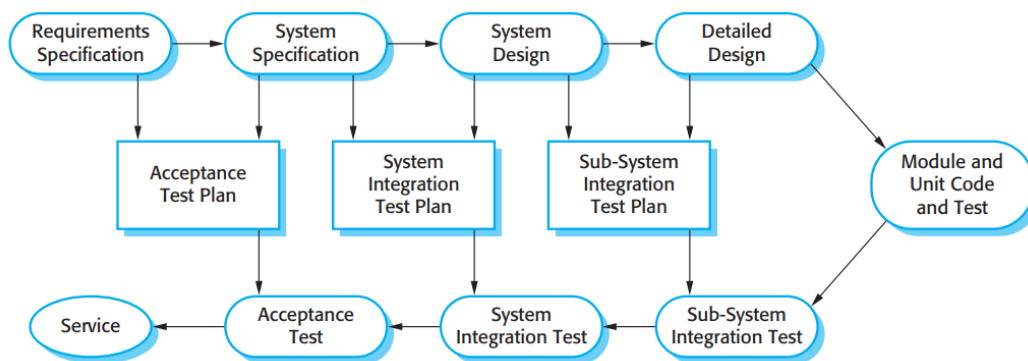


Figura 2.7: Fasi di test in un processo software plan-driven

2.2.4 Evoluzione del software

La flessibilità del software è una delle principali ragioni per cui sempre più software viene incorporato in sistemi complessi e di grandi dimensioni. Modifiche al software possono essere effettuate in qualsiasi momento durante o dopo lo sviluppo del sistema.

Pochissimi sistemi software sono completamente nuovi; è quindi molto più sensato vedere lo sviluppo e la manutenzione come un continuum. Piuttosto che considerare due processi separati, è

più realistico pensare all'ingegneria del software come a un processo evolutivo (Figura 2.8), in cui il software viene continuamente modificato nel corso della sua vita in risposta a requisiti e bisogni dei clienti in evoluzione.

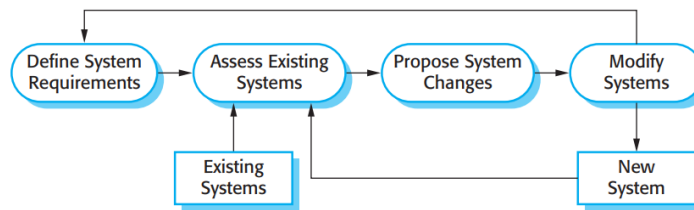


Figura 2.8: Evoluzione del sistema

2.3 Affrontare il cambiamento

Il cambiamento è inevitabile in tutti i grandi progetti software. I requisiti del sistema cambiano mentre le aziende rispondono a pressioni esterne, competizione e cambiamenti nelle priorità di gestione. Con l'emergere di nuove tecnologie, diventano possibili nuovi approcci al design e all'implementazione. Pertanto, qualunque modello di processo software venga utilizzato, è essenziale che possa accogliere i cambiamenti nel software in fase di sviluppo.

I cambiamenti aumentano i costi di sviluppo software poiché generalmente significa che il lavoro già completato deve essere rifatto. Questo è noto come rielaborazione.

Due approcci correlati possono essere utilizzati per ridurre i costi di rielaborazione:

1. *Anticipazione dei cambiamenti*: il processo software include attività in grado di anticipare o prevedere possibili cambiamenti prima che siano necessarie rielaborazioni significative. Ad esempio, potrebbe essere sviluppato un sistema prototipo per mostrare alcune funzionalità chiave del sistema ai clienti.
2. *Tolleranza ai cambiamenti*: il processo e il software sono progettati in modo che le modifiche possano essere apportate facilmente al sistema. Ciò comporta normalmente una forma di sviluppo incrementale. Le modifiche proposte possono essere implementate in incrementi che non sono stati ancora sviluppati. Se ciò non è possibile, solo un singolo incremento (una piccola parte del sistema) potrebbe dover essere modificato per incorporare il cambiamento.

Vediamo due modi per affrontare il cambiamento e i requisiti del sistema in evoluzione:

1. *Prototipazione del sistema*: una versione del sistema o parte di essa è sviluppata rapidamente per verificare i requisiti del cliente e la fattibilità delle decisioni progettuali. Questo è un metodo di anticipazione del cambiamento.
2. *Consegna incrementale*: gli incrementi del sistema vengono consegnati al cliente per commenti e sperimentazione. Questo supporta sia l'evitamento del cambiamento sia la tolleranza ai cambiamenti.

2.3.1 Prototipazione

Un prototipo è una versione preliminare di un sistema software utilizzata per dimostrare concetti, provare opzioni di design e approfondire la comprensione del problema e delle possibili soluzioni.

Un prototipo software può essere utilizzato in un processo di sviluppo software per aiutare a prevedere i cambiamenti che potrebbero essere necessari:

1. Nel processo di ingegneria dei requisiti, un prototipo può aiutare con l'estrazione e la validazione dei requisiti del sistema.
2. Nel processo di design del sistema, un prototipo può essere utilizzato per esplorare soluzioni software e nello sviluppo di un'interfaccia utente per il sistema.

I prototipi di sistema consentono agli utenti potenziali di vedere quanto bene il sistema supporti il loro lavoro. Possono generare nuove idee per requisiti e identificare aree di forza e debolezza nel software. Possono quindi proporre nuovi requisiti di sistema. Inoltre, man mano che il prototipo viene sviluppato, possono emergere errori e omissioni nei requisiti del sistema.

Un modello di processo per lo sviluppo di prototipi è mostrato nella Figura 2.9. Gli obiettivi della prototipazione dovrebbero essere esplicitati fin dall'inizio del processo.

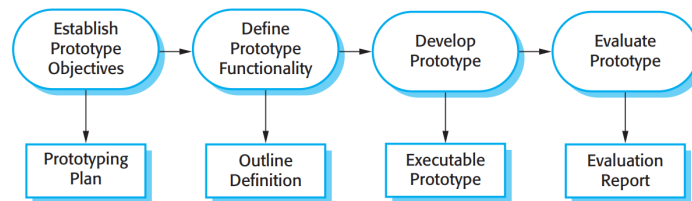


Figura 2.9: Il processo di sviluppo del prototipo

La fase successiva del processo è decidere cosa includere e, forse più importante, cosa escludere dal sistema prototipo. Per ridurre i costi di prototipazione e accelerare il programma di consegna, si può decidere di escludere alcune funzionalità dal prototipo. Il prototipo dovrebbe concentrarsi su aree del prodotto che non sono ben comprese e il controllo degli errori e il ripristino potrebbero non essere inclusi nel prototipo. L'ultima fase del processo è la valutazione del prototipo.

I prototipi dovrebbero essere scartati dopo lo sviluppo poiché non sono una buona base per un sistema di produzione. Infatti: potrebbe essere impossibile adattare il sistema per soddisfare i requisiti non funzionali; i prototipi sono normalmente privi di documentazione; la struttura del prototipo è solitamente degradabile a causa del cambiamento rapido; il prototipo probabilmente non soddisferà gli standard di qualità normali dell'organizzazione.

2.3.2 Consegna incrementale

La consegna incrementale (Figura 2.10) è un approccio allo sviluppo software in cui alcuni degli incrementi sviluppati vengono consegnati al cliente e implementati nel loro ambiente di lavoro. In un processo di consegna incrementale, i clienti definiscono quali servizi sono più importanti e quali sono meno importanti per loro. Vengono quindi definiti diversi incrementi di consegna, ciascuno dei quali fornisce un sottoinsieme delle funzionalità del sistema.

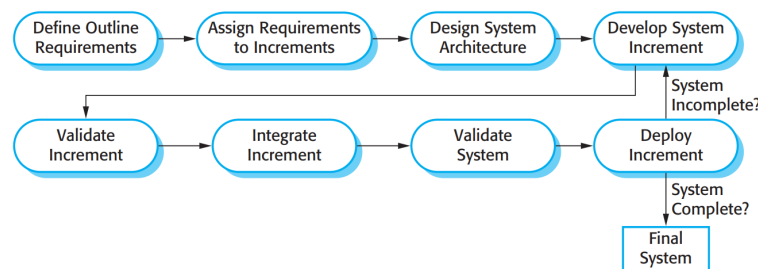


Figura 2.10: Consegna incrementale

Una volta identificati gli incrementi di sistema, i requisiti per i servizi da consegnare nel primo incremento vengono definiti in dettaglio e quell'incremento viene sviluppato. Man mano che nuovi incrementi vengono completati, vengono integrati con gli incrementi esistenti in modo che la funzionalità del sistema migliori con ogni incremento consegnato.

La consegna incrementale presenta diversi vantaggi:

1. I clienti possono utilizzare i primi incrementi come prototipi e acquisire esperienza che informi i loro requisiti per gli incrementi successivi del sistema.
2. I clienti non devono aspettare fino a quando l'intero sistema è consegnato prima di poter trarre valore da esso. Il primo incremento soddisfa i loro requisiti più critici, quindi possono utilizzare il software immediatamente.

3. Il processo mantiene i benefici dello sviluppo incrementale in quanto dovrebbe essere relativamente facile incorporare cambiamenti nel sistema.
4. Poiché i servizi di massima priorità vengono consegnati per primi e gli incrementi successivi vengono poi integrati, i servizi di sistema più importanti ricevono il maggior numero di test.

Tuttavia, ci sono problemi con la consegna incrementale. In pratica, funziona solo in situazioni in cui viene introdotto un sistema completamente nuovo e i valutatori del sistema hanno tempo per sperimentare con il nuovo sistema. I principali problemi con questo approccio sono:

1. La maggior parte dei sistemi richiede un insieme di funzionalità di base utilizzate da diverse parti del sistema. Poiché i requisiti non sono definiti in dettaglio fino a quando non deve essere implementato un incremento, può essere difficile identificare le funzionalità comuni necessarie per tutti gli incrementi.
2. L'essenza dei processi iterativi è che la specifica viene sviluppata in concomitanza con il software. Tuttavia, questo va in conflitto con il modello di approvvigionamento di molte organizzazioni, in cui la specifica completa del sistema fa parte del contratto di sviluppo del sistema.

2.4 Miglioramento dei processi

Molte aziende software si sono rivolte al miglioramento dei processi software come modo per aumentare la qualità del loro software, ridurre i costi o accelerare i loro processi di sviluppo. Il miglioramento del processo significa comprendere i processi esistenti e cambiarli per aumentare la qualità del prodotto e/o ridurre i costi e i tempi di sviluppo.

Vengono utilizzati due approcci abbastanza diversi per il miglioramento e il cambiamento dei processi:

1. *L'approccio della maturità del processo*, che si è concentrato sul miglioramento della gestione dei processi e dei progetti e sull'introduzione di buone pratiche di ingegneria del software in un'organizzazione. Il livello di maturità del processo riflette il grado in cui sono state adottate buone pratiche tecniche e di gestione nei processi di sviluppo software dell'organizzazione.
2. *L'approccio agile*, che si è concentrato sullo sviluppo iterativo e sulla riduzione dei costi generali nel processo software. Le caratteristiche principali dei metodi agili sono la rapida consegna delle funzionalità e la reattività ai cambiamenti nei requisiti del cliente.

Il processo generale di miglioramento del processo che sottende l'approccio della maturità del processo è un processo ciclico, come mostrato nella Figura 2.11.

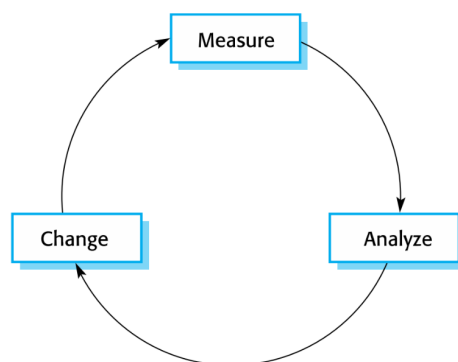


Figura 2.11: Il ciclo di miglioramento dei processi

Le fasi in questo processo sono:

1. *Misurazione del processo*: si misura uno o più attributi del processo software o del prodotto. Queste misurazioni formano una base di riferimento che aiuta a decidere se i miglioramenti del processo sono stati efficaci.

2. *Analisi del processo*: il processo attuale viene valutato e vengono identificati punti deboli e colli di bottiglia. Durante questa fase possono essere sviluppati modelli di processo (a volte chiamati mappe di processo) che descrivono il processo.
3. *Cambiamento del processo*: vengono proposti cambiamenti al processo per affrontare alcune delle debolezze identificate. Questi vengono introdotti e il ciclo riprende per raccogliere dati sull'efficacia delle modifiche.

Senza dati concreti su un processo o sul software sviluppato utilizzando quel processo, è impossibile valutare il valore del miglioramento del processo. Tuttavia, le aziende che avviano il processo di miglioramento potrebbero non avere dati sui processi disponibili come base di riferimento per il miglioramento. Pertanto, come parte del primo ciclo di cambiamenti, potrebbe essere necessario raccogliere dati sul processo software e misurare le caratteristiche del prodotto software.

L'idea di maturità del processo è stata introdotta alla fine degli anni '80 quando il Software Engineering Institute (SEI) propose il loro modello di maturità della capacità dei processi (Humphrey 1988). La maturità dei processi di un'azienda software riflette la gestione dei processi, la misurazione e l'uso di buone pratiche di ingegneria del software all'interno dell'azienda. Sono stati proposti cinque livelli di maturità del processo, come mostrato nella Figura 2.12.

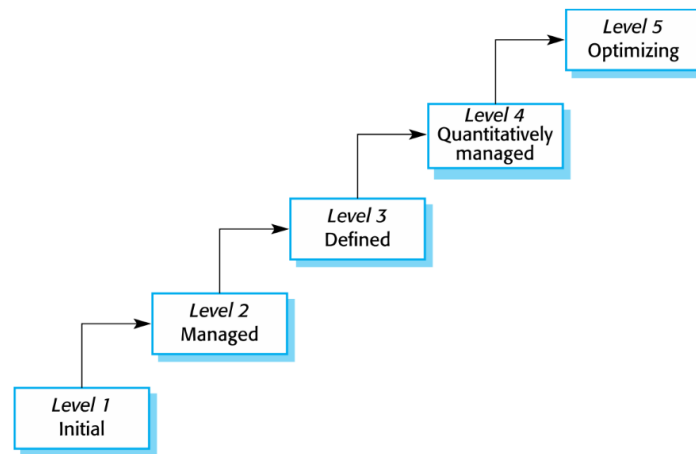


Figura 2.12: Livelli di maturità delle capacità

I livelli nel modello di maturità del processo sono:

1. *Iniziale*: gli obiettivi associati all'area del processo sono soddisfatti e, per tutti i processi, l'ambito del lavoro da svolgere è esplicitamente definito e comunicato ai membri del team.
2. *Gestito*: a questo livello, gli obiettivi associati all'area del processo sono raggiunti e le politiche organizzative sono in atto per definire quando ciascun processo dovrebbe essere utilizzato.
3. *Definito*: questo livello si concentra sulla standardizzazione e distribuzione organizzativa dei processi.
4. *Gestito quantitativamente*: a questo livello, c'è una responsabilità organizzativa per utilizzare metodi statistici e altri metodi quantitativi per controllare i sotto-processi.
5. *Ottimizzante*: a questo livello più alto, l'organizzazione deve utilizzare le misurazioni del processo e del prodotto per guidare il miglioramento del processo.

2.5 Punti chiave

- I processi software sono le attività coinvolte nella produzione di un sistema software. I modelli di processo software sono rappresentazioni astratte di questi processi.
- I modelli di processo generali descrivono l'organizzazione dei processi software. Esempi di questi modelli generali includono il modello a cascata, lo sviluppo incrementale e la configurazione e integrazione di componenti riutilizzabili.
- L'ingegneria dei requisiti è il processo di sviluppo di una specifica software.
- I processi di design e implementazione si occupano di trasformare una specifica dei requisiti in un sistema software eseguibile.
- La validazione del software è il processo di verifica che il sistema sia conforme alla sua specifica e che soddisfi le reali esigenze degli utenti del sistema.
- L'evoluzione del software avviene quando si modificano i sistemi software esistenti per soddisfare nuovi requisiti. I cambiamenti sono continui e il software deve evolvere per rimanere utile.
- I processi dovrebbero includere attività per affrontare i cambiamenti. Questo può comportare una fase di prototipazione che aiuta a evitare decisioni sbagliate sui requisiti e sul design. I processi possono essere strutturati per uno sviluppo e una consegna iterativa in modo che i cambiamenti possano essere apportati senza interrompere il sistema nel suo insieme.
- Le principali strategie per il miglioramento dei processi sono gli approcci agili, orientati a ridurre i costi dei processi, e gli approcci basati sulla maturità, che si fondano su una migliore gestione dei processi e sull'uso di buone pratiche di ingegneria del software.
- Il framework di maturità dei processi del SEI identifica livelli di maturità che corrispondono essenzialmente all'uso di buone pratiche di ingegneria del software.

Capitolo 3

Sviluppo software agile

Le aziende operano ora in un ambiente globale in rapido cambiamento. Il software è parte di quasi tutte le operazioni aziendali, quindi il nuovo software deve essere sviluppato rapidamente per cogliere le nuove opportunità e rispondere alla pressione competitiva. Lo sviluppo e la consegna rapida del software sono quindi i requisiti più critici per la maggior parte dei sistemi aziendali.

I processi di sviluppo software plan-driven, che specificano completamente i requisiti e poi progettano, costruiscono e testano un sistema, non sono adatti allo sviluppo rapido del software. Man mano che i requisiti cambiano o emergono problemi nei requisiti, il design o l'implementazione del sistema devono essere rielaborati e ritestati.

Lo sviluppo rapido del software ha veramente preso piede alla fine degli anni '90 con lo sviluppo dell'idea di "metodi agili" progettati per produrre software utile in tempi rapidi. Tutti i metodi agili proposti condividono una serie di caratteristiche comuni:

1. I processi di specifica, progettazione e implementazione sono intrecciati.
2. Il sistema viene sviluppato in una serie di incrementi. Gli utenti finali e altri stakeholder del sistema sono coinvolti nella specifica e valutazione di ciascun incremento.
3. Viene utilizzato un ampio supporto di strumenti per sostenere il processo di sviluppo.
4. Non esiste una specifica dettagliata del sistema, e la documentazione di progettazione è ridotta al minimo o generata automaticamente.

Gli approcci agili allo sviluppo software considerano la progettazione e l'implementazione come le attività centrali nel processo software. Includono altre attività, come l'elicitazione dei requisiti e il testing, nella progettazione e nell'implementazione. Al contrario, un approccio plan-driven per l'ingegneria del software identifica fasi separate nel processo software, con output associati a ciascuna fase. Gli output di una fase vengono utilizzati come base per pianificare l'attività di processo successiva.

La Figura 3.1 mostra le distinzioni essenziali tra approcci plan-driven e approcci agili alla specifica del sistema.

3.1 Metodi agili

Negli anni '80 e nei primi anni '90, era diffusa l'idea che il modo migliore per ottenere un software migliore fosse attraverso un'attenta pianificazione del progetto. L'insoddisfazione per questi approcci pesanti all'ingegneria del software ha portato allo sviluppo di metodi agili alla fine degli anni '90. Questi metodi hanno permesso ai team di sviluppo di concentrarsi sul software stesso piuttosto che sulla sua progettazione e documentazione. Sono più adatti allo sviluppo di applicazioni in cui i requisiti del sistema di solito cambiano rapidamente durante il processo di sviluppo. Sono progettati per consegnare software funzionante rapidamente ai clienti, che possono poi proporre nuovi requisiti e cambiamenti da includere nelle iterazioni successive del sistema. L'obiettivo è ridurre le spese generali nel processo software ed eliminando documentazione che probabilmente non verrà mai utilizzata.

La filosofia dietro i metodi agili si riflette nel manifesto agile pubblicato dai principali sviluppatori di questi metodi. Questo manifesto afferma:

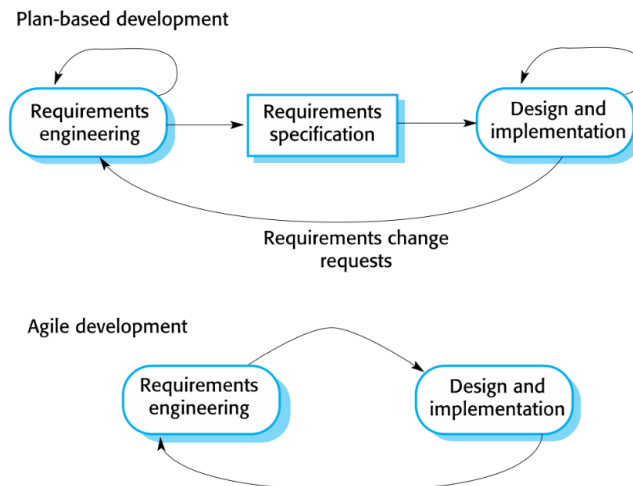


Figura 3.1: Sviluppo plan-driven e agile

Siamo alla scoperta di modi migliori per sviluppare software facendolo e aiutando altri a farlo. Attraverso questo lavoro, abbiamo imparato a dare valore a:

Le persone e le interazioni più che i processi e gli strumenti

Il software funzionante più che la documentazione esaustiva

La collaborazione con il cliente più che la negoziazione del contratto

Rispondere ai cambiamenti più che seguire un piano

Vale a dire, mentre c'è valore negli elementi a destra, diamo più valore agli elementi a sinistra.

Tutti i metodi agili suggeriscono che il software debba essere sviluppato e consegnato in modo incrementale. Questi metodi si basano su diversi processi agili, ma condividono un insieme di principi, basati sul manifesto agile, e quindi hanno molto in comune (vedi Tabella 3.1).

Principio	Descrizione
Coinvolgimento del cliente	I clienti dovrebbero essere coinvolti strettamente durante tutto il processo di sviluppo. Il loro ruolo è fornire e prioritizzare i nuovi requisiti del sistema e valutare le iterazioni del sistema.
Consegna incrementale	Il software viene sviluppato in incrementi, con il cliente che specifica i requisiti da includere in ogni incremento.
Persone, non processi	Le competenze del team di sviluppo dovrebbero essere riconosciute e sfruttate. I membri del team dovrebbero essere lasciati liberi di sviluppare i propri modi di lavorare senza processi prescrittivi.
Accogliere il cambiamento	È necessario aspettarsi che i requisiti del sistema cambino e progettare il sistema per adattarsi a questi cambiamenti.
Mantenere la semplicità	Concentrarsi sulla semplicità sia nel software che viene sviluppato, sia nel processo di sviluppo. Ovunque sia possibile, lavorare attivamente per eliminare la complessità dal sistema.

Tabella 3.1: I principi dei metodi agili

I metodi agili hanno avuto particolare successo in due tipi di sviluppo di sistemi:

1. Sviluppo di prodotti, in cui un'azienda di software sviluppa un prodotto di piccole o medie dimensioni per la vendita. Praticamente tutti i prodotti software e le app vengono ora sviluppati utilizzando un approccio agile.
2. Sviluppo di sistemi personalizzati all'interno di un'organizzazione, dove c'è un chiaro impegno da parte del cliente a essere coinvolto nel processo di sviluppo e dove ci sono pochi stakeholder esterni e regolamenti che influenzano il software.

3.2 Tecniche di sviluppo agili

Le idee alla base dei metodi agili sono state sviluppate all'incirca nello stesso periodo da diverse persone negli anni '90. Tuttavia, l'approccio che probabilmente ha avuto il maggiore impatto nel cambiare la cultura dello sviluppo software è stato lo sviluppo di Extreme Programming (XP). Il termine è stato coniato da Kent Beck (Beck 1998) poiché l'approccio è stato sviluppato portando a livelli "estremi" le pratiche riconosciute come buone, come lo sviluppo iterativo. Ad esempio, in XP, diverse nuove versioni di un sistema possono essere sviluppate da diversi programmatori, integrate e testate in un solo giorno. La Figura 3.2 illustra il processo XP per produrre un incremento del sistema in fase di sviluppo.

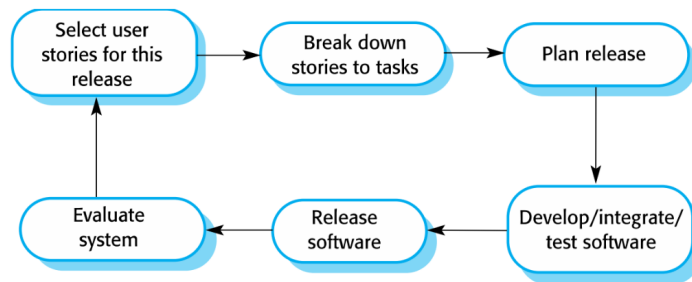


Figura 3.2: Il ciclo di rilascio dell'extreme programming

Extreme Programming è stato controverso poiché ha introdotto una serie di pratiche agili che erano piuttosto diverse dalle pratiche di sviluppo di quel tempo. Queste pratiche sono riassunte nella Tabella 3.2 e riflettono i principi del manifesto agile:

1. Sviluppo incrementale è supportato attraverso piccoli e frequenti rilasci del sistema.
2. Coinvolgimento del cliente è supportato attraverso il continuo coinvolgimento del cliente nel team di sviluppo.
3. Persone, non processi sono supportate attraverso la programmazione in coppia, la proprietà collettiva del codice del sistema e un processo di sviluppo sostenibile che non prevede lunghe ore di lavoro.
4. Il cambiamento viene accolto attraverso i rilasci regolari del sistema ai clienti.
5. Mantenere la semplicità è supportato da un refactoring costante che migliora la qualità del codice.

In pratica, l'applicazione di Extreme Programming come originariamente proposto si è rivelata più difficile del previsto. Non può essere facilmente integrato con le pratiche di gestione della maggior parte delle aziende. Pertanto, le aziende che adottano metodi agili scelgono le pratiche XP che sono più adatte al loro modo di lavorare. A volte queste sono incorporate nei loro processi di sviluppo, ma più comunemente vengono utilizzate in combinazione con un metodo agile più focalizzato sulla gestione.

3.2.1 Storie degli utenti

I requisiti software cambiano costantemente. Per gestire questi cambiamenti, i metodi agili prevedono l'idea delle "storie degli utenti," dove una storia utente è uno scenario d'uso che potrebbe essere vissuto da un utilizzatore del sistema.

Per quanto possibile, il cliente del sistema lavora a stretto contatto con il team di sviluppo e discute questi scenari con gli altri membri del team. Insieme, sviluppano una "carta della storia" che descrive brevemente una storia che incapsula le esigenze del cliente.

Le storie degli utenti possono essere utilizzate nella pianificazione delle iterazioni del sistema. Una volta sviluppate le carte delle storie, il team di sviluppo le suddivide in compiti e stima lo sforzo e le risorse necessari per implementare ciascun compito. Il cliente poi assegna la priorità alle storie da implementare, scegliendo quelle che possono essere utilizzate immediatamente per fornire supporto operativo.

Principio o pratica	Descrizione
Pianificazione incrementale	I requisiti sono registrati su "carte delle storie" e le storie da includere in un rilascio sono determinate in base al tempo disponibile e alla loro priorità relativa. Gli sviluppatori suddividono queste storie in "compiti" di sviluppo.
Piccoli rilasci	Il set minimo di funzionalità utile che fornisce valore aziendale viene sviluppato per primo. I rilasci del sistema sono frequenti e aggiungono funzionalità in modo incrementale al primo rilascio.
Progettazione semplice	Viene effettuata solo la progettazione necessaria per soddisfare i requisiti attuali e non di più.
Sviluppo test-first	Un framework di test unitario automatizzato viene utilizzato per scrivere test per una nuova funzionalità prima che la funzionalità stessa venga implementata.
Refactoring	Tutti gli sviluppatori sono tenuti a rifattorizzare continuamente il codice non appena vengono trovati potenziali miglioramenti. Questo mantiene il codice semplice e manutenibile.
Programmazione in coppia	Gli sviluppatori lavorano in coppia, controllando reciprocamente il lavoro e fornendo supporto per svolgere sempre un buon lavoro.
Proprietà collettiva	Le coppie di sviluppatori lavorano su tutte le aree del sistema, in modo che non si sviluppino isole di competenza e tutti gli sviluppatori si assumano la responsabilità per tutto il codice. Chiunque può modificare qualsiasi parte.
Integrazione continua	Non appena un compito è completato, viene integrato nel sistema completo. Dopo ogni integrazione, tutti i test unitari del sistema devono passare.
Ritmo sostenibile	Lavorare molte ore di straordinario non è accettabile, poiché spesso l'effetto netto è una riduzione della qualità del codice e della produttività a medio termine.
Cliente in loco	Un rappresentante dell'utente finale del sistema (il Cliente) deve essere disponibile a tempo pieno per il team XP. In un processo di extreme programming, il cliente è membro del team di sviluppo ed è responsabile di portare i requisiti del sistema al team per l'implementazione.

Tabella 3.2: Principi e pratiche di extreme programming

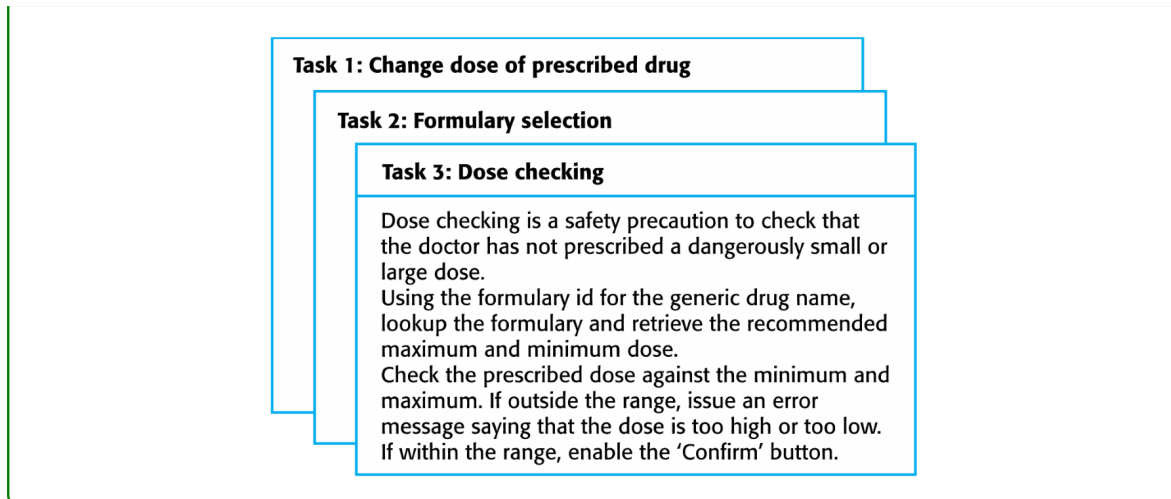
Esempio 3.2.1 – Una storia di "prescrizione di farmaci"

Il record del paziente deve essere aperto per l'inserimento. Fare clic sul campo relativo ai farmaci e selezionare 'medicazione corrente', 'nuova medicazione' o 'formulario'.

- Se si seleziona 'medicazione corrente', verrà richiesto di controllare il dosaggio. Se si desidera cambiarlo, inserire il nuovo dosaggio e confermare la prescrizione.
- Se si sceglie 'nuova medicazione', il sistema presuppone che si sappia quale farmaco prescrivere. Digitare le prime lettere del nome del farmaco e verrà visualizzato un elenco di farmaci possibili. Selezionare il farmaco richiesto, verificare che il farmaco scelto sia corretto, inserire il dosaggio e confermare la prescrizione.
- Se si sceglie 'formulario', verrà visualizzata una casella di ricerca per cercare nel formulario approvato. Selezionare il farmaco richiesto, verificare che sia corretto, inserire il dosaggio e confermare la prescrizione.

In tutti i casi, il sistema controllerà che il dosaggio sia all'interno del range approvato e chiederà di modificarlo se è fuori dal range raccomandato.

Dopo aver confermato la prescrizione, verrà visualizzata per un controllo finale. Fare clic su 'OK' per registrare la prescrizione nel database di audit oppure su 'Modifica' per rientrare nel processo di prescrizione.



3.2.2 Refactoring

Un concetto fondamentale nell'ingegneria del software tradizionale è progettare per il cambiamento, ossia prevedere le modifiche future al software e progettarlo in modo che possano essere facilmente implementate. Tuttavia, l'Extreme Programming (XP) ha abbandonato questo principio, sostenendo che spesso progettare per il cambiamento è uno spreco di tempo. Le modifiche anticipate spesso non si materializzano o si verificano richieste di cambiamento completamente diverse.

XP propone invece che il codice in sviluppo venga continuamente rifattorizzato. Il refactoring consiste nel cercare miglioramenti possibili al software e implementarli immediatamente, anche in assenza di una necessità immediata. Se i membri del team individuano codice migliorabile, apportano queste modifiche per evitare che, col tempo, la struttura del software si degradi.

Esempi di refactoring includono la riorganizzazione di gerarchie di classi per eliminare codice duplicato, la pulizia e rinominazione di attributi e metodi, e la sostituzione di sezioni di codice simili con chiamate a metodi definiti in una libreria.

3.2.3 Test-first development

L'Extreme Programming (XP) ha sviluppato un nuovo approccio al test del software per affrontare le difficoltà di test senza una specifica formale. I test sono automatizzati e centrali nel processo di sviluppo: lo sviluppo non può procedere finché tutti i test non sono stati eseguiti con successo. Le caratteristiche principali del testing in XP sono:

1. Sviluppo con test preliminari,
2. Sviluppo incrementale dei test a partire da scenari,
3. Coinvolgimento dell'utente nello sviluppo e nella validazione dei test,
4. Utilizzo di framework di testing automatizzato per eseguire tutti i test dei componenti ogni volta che viene creata una nuova versione.

Invece di scrivere il codice e successivamente i test, i test vengono scritti prima del codice. Questo consente di eseguire i test durante la scrittura del codice e scoprire eventuali problemi immediatamente.

Nel test-first development, gli sviluppatori devono comprendere a fondo la specifica per poter scrivere i test. Ciò significa che eventuali ambiguità e omissioni devono essere chiarite prima che inizi l'implementazione.

Esempio 3.2.2 – Verifica della dose

Input:

1. Un numero in mg che rappresenta una singola dose del farmaco.
2. Un numero che rappresenta il numero di dosi giornaliere.

Tests:

1. Test per singola dose corretta, ma frequenza troppo alta.
2. Test per dose singola troppo alta o troppo bassa.
3. Test per frequenza della dose troppo alta o troppo bassa.
4. Test per dose e frequenza all'interno del range consentito.

Output: Se i valori inseriti sono corretti, restituisce OK. In caso contrario, restituisce un messaggio di errore che specifica se la dose o la frequenza è fuori dal range sicuro.

L'automazione del testing è essenziale per lo sviluppo con test preliminari. I test sono scritti come componenti eseguibili prima dell'implementazione del task. Questi componenti dovrebbero essere autonomi, simulare l'invio di input da testare e verificare che il risultato soddisfi la specifica dell'output. Un esempio comune di framework di testing automatizzato è JUnit, utilizzato per testare programmi Java.

Nonostante i vantaggi di questo approccio, ci sono problemi nel garantire la completezza della copertura dei test:

1. Gli sviluppatori spesso preferiscono programmare piuttosto che scrivere test, e talvolta prendono scorciatoie scrivendo test incompleti.
2. Alcuni test sono difficili da scrivere in modo incrementale, come quelli per le interfacce utente complesse.

Pertanto, anche un ampio set di test eseguiti frequentemente potrebbe non garantire la copertura completa del sistema, e bug non rilevati potrebbero comunque essere presenti nelle versioni rilasciate.

3.2.4 Pair programming

Un'altra pratica innovativa introdotta in XP è che i programmatori lavorano in coppia per sviluppare il software. La coppia di programmatori si siede allo stesso computer per sviluppare il software. Tuttavia, la stessa coppia non programma sempre insieme. Piuttosto, le coppie vengono formate dinamicamente in modo che tutti i membri del team lavorino con ciascuno degli altri durante il processo di sviluppo.

La programmazione in coppia ha una serie di vantaggi:

1. Supporta l'idea di proprietà e responsabilità collettiva del sistema.
2. Funziona come un processo di revisione informale perché ogni riga di codice è esaminata da almeno due persone.
3. Incoraggia il refactoring per migliorare la struttura del software.

La condivisione delle conoscenze che avviene durante la programmazione in coppia è molto importante in quanto riduce i rischi complessivi per un progetto quando i membri del team se ne vanno. La programmazione in coppia non è necessariamente inefficiente e ci sono alcune prove che suggeriscono che una coppia che lavora insieme è più efficiente di due programmatori che lavorano separatamente.

3.3 Gestione agile dei progetti

In qualsiasi attività legata al software, i manager devono sapere cosa sta succedendo e se un progetto è in grado di raggiungere i suoi obiettivi e consegnare il software in tempo e con il budget previsto. Gli approcci allo sviluppo software plan driven si sono evoluti per soddisfare questa necessità: i manager redigono un piano per il progetto che mostra cosa dovrebbe essere consegnato, quando dovrebbe essere consegnato e chi lavorerà allo sviluppo dei deliverable del progetto. Un approccio basato su piani richiede che un manager abbia una visione stabile di tutto ciò che deve essere sviluppato e dei processi di sviluppo.

Scrum è un metodo agile che si concentra sulla gestione dello sviluppo iterativo piuttosto che su pratiche agili specifiche. Ci sono tre fasi in Scrum:

1. La fase iniziale è una fase di pianificazione generale in cui si stabiliscono gli obiettivi generali per il progetto e si progetta l'architettura software.
2. Questa è seguita da una serie di cicli di sprint, in cui ogni ciclo sviluppa un incremento del sistema.
3. La fase di chiusura del progetto conclude il progetto, completa la documentazione richiesta come frame di aiuto del sistema e manuali utente e valuta le lezioni apprese dal progetto.

Termine Scrum	Definizione
Team di sviluppo	Un gruppo auto-organizzato di sviluppatori software, composto da non più di sette persone. Sono responsabili dello sviluppo del software e dei documenti essenziali.
Incremento di prodotto potenzialmente rilasciabile	L'incremento software consegnato alla fine di uno sprint, che dovrebbe essere in uno stato "potenzialmente rilasciabile", senza bisogno di ulteriori lavori come test. In pratica, non sempre è raggiungibile.
Backlog del prodotto	Una lista di attività o elementi "da fare" che il team Scrum deve affrontare. Possono essere definizioni di funzionalità del software, requisiti, user stories o compiti supplementari come la definizione dell'architettura o la documentazione utente.
Proprietario del prodotto	Una persona (o un piccolo gruppo) incaricata di identificare le funzionalità o i requisiti del prodotto, priorizzarli per lo sviluppo e rivedere continuamente il backlog per garantire che il progetto continui a soddisfare i bisogni critici del business. Il Product Owner può essere un cliente o un product manager di un'azienda di software o un altro rappresentante degli stakeholder.
Scrum	Un incontro giornaliero del team Scrum per rivedere i progressi e dare priorità al lavoro da fare durante il giorno. Idealmente, dovrebbe essere un breve incontro faccia a faccia con l'intero team.
ScrumMaster	Lo ScrumMaster è responsabile di assicurare che il processo Scrum venga seguito e guida il team nell'uso efficace di Scrum. È responsabile dell'interfaccia con il resto dell'azienda e di garantire che il team Scrum non venga distratto da interferenze esterne. I creatori di Scrum sottolineano che lo ScrumMaster non dovrebbe essere considerato un project manager, anche se potrebbe non essere facile vedere la differenza.
Sprint	Una iterazione di sviluppo, generalmente di durata compresa tra 2 e 4 settimane.
Velocità	Una stima di quanto backlog il team può affrontare in un singolo sprint. Comprendere la velocità del team aiuta a stimare cosa si può coprire in uno sprint e a misurare le performance.

Tabella 3.3: Terminologia Scrum

Ogni ciclo sprint dura un periodo di tempo fisso, solitamente tra le 2 e le 4 settimane. Il punto di partenza per la pianificazione è il product backlog, ovvero l'elenco del lavoro da svolgere sul progetto. La fase di selezione coinvolge tutto il team di progetto che lavora con il cliente per selezionare le caratteristiche e le funzionalità dal product backlog da sviluppare durante lo sprint.

Una volta concordati, il team si organizza per sviluppare il software. Durante questa fase il team è isolato dal cliente e dall'organizzazione, con tutte le comunicazioni incanalate attraverso il cosiddetto "Scrum master". Il ruolo dello Scrum master è quello di proteggere il team di sviluppo da distrazioni esterne. Alla fine dello sprint, il lavoro svolto viene rivisto e presentato alle parti interessate. Quindi inizia il ciclo di sprint successivo. Lo "Scrum master" è un facilitatore che organizza riunioni giornaliere, tiene traccia dell'arretrato di lavoro da svolgere, registra le decisioni, misura i progressi rispetto all'arretrato e comunica con i clienti e la dirigenza esterna al team.

L'intero team partecipa a brevi riunioni giornaliere (Scrum) in cui tutti i membri del team condividono informazioni, descrivono i progressi dall'ultima riunione, i problemi emersi e cosa è previsto

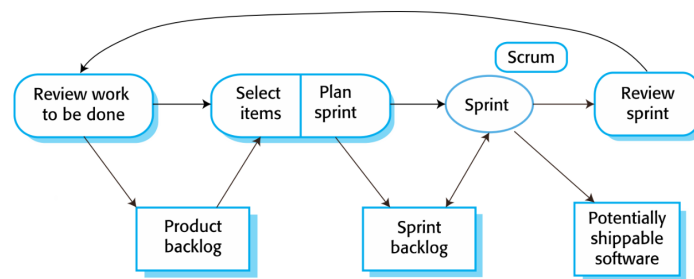


Figura 3.3: Ciclo sprint Scrum

per il giorno successivo. Ciò significa che tutti nel team sanno cosa sta succedendo e, se si presentano problemi, possono riprogrammare il lavoro a breve termine per affrontarli.

I benefici del metodo Scrum sono:

1. Il prodotto è suddiviso in una serie di componenti gestibili e comprensibili a cui gli stakeholder possono rapportarsi.
2. I requisiti instabili non bloccano i progressi.
3. L'intero team ha visibilità su tutto e, di conseguenza, la comunicazione del team e il morale migliorano.
4. I clienti vedono la consegna puntuale degli incrementi e ricevono feedback su come funziona il prodotto.
5. Si crea fiducia tra i clienti e gli sviluppatori, e si instaura una cultura positiva in cui tutti si aspettano che il progetto abbia successo.

Tuttavia, gran parte dello sviluppo software ora coinvolge team distribuiti, con membri del team situati in luoghi diversi in tutto il mondo. Di conseguenza, ci sono stati sviluppi di Scrum per ambienti di sviluppo distribuiti e per il lavoro multi-team (vedi Figura 3.4).

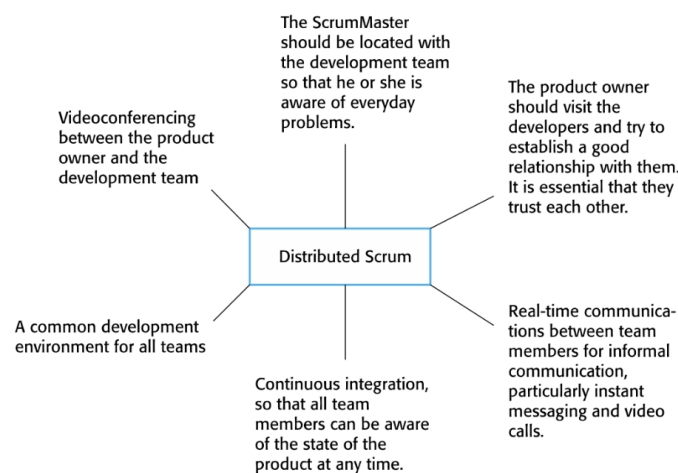


Figura 3.4: Scrum distribuito

3.4 Scalabilità dei metodi agili

I metodi agili hanno dimostrato di essere efficaci per progetti di piccole e medie dimensioni che possono essere sviluppati da un piccolo team co-localizzato. A volte si sostiene che il successo di questi metodi derivi dal miglioramento delle comunicazioni che è possibile quando tutti lavorano insieme. Naturalmente, la necessità di una consegna più rapida del software, che si adatta meglio alle esigenze dei clienti, si applica anche a sistemi di grandi dimensioni e a grandi aziende.

La scalabilità dei metodi agili ha aspetti strettamente correlati:

1. "Scaling up" riguarda l'uso di metodi agili per sviluppare grandi sistemi software che non possono essere sviluppati da un piccolo team.
2. "Scaling out" riguarda il modo in cui i metodi agili possono essere introdotti in una grande organizzazione con molti anni di esperienza nello sviluppo software.

3.4.1 Problemi pratici con i metodi agili

Per i sistemi grandi e di lunga durata sviluppati da un'azienda software per un cliente esterno, l'uso di un approccio agile presenta una serie di problemi.

1. L'informalità dello sviluppo agile è incompatibile con l'approccio legale alla definizione dei contratti comunemente utilizzato nelle grandi aziende.
2. I metodi agili sono più appropriati per lo sviluppo di nuovo software piuttosto che per la manutenzione del software. Tuttavia, la maggior parte dei costi software nelle grandi aziende proviene dalla manutenzione dei loro sistemi software esistenti.
3. I metodi agili sono progettati per piccoli team co-localizzati, ma gran parte dello sviluppo software ora coinvolge team distribuiti a livello mondiale.

Le questioni contrattuali possono essere un grosso problema quando si utilizzano metodi agili. Quando il cliente del sistema utilizza un'organizzazione esterna per lo sviluppo del sistema, viene redatto un contratto per lo sviluppo software tra loro. Poiché lo sviluppo intrecciato di requisiti e codice è fondamentale per i metodi agili, non esiste una dichiarazione definitiva dei requisiti che possa essere inclusa nel contratto. Di conseguenza, i metodi agili devono fare affidamento su contratti in cui il cliente paga per il tempo richiesto per lo sviluppo del sistema piuttosto che per lo sviluppo di un insieme specifico di requisiti.

Una grande quantità di sforzi di ingegneria del software viene dedicata alla manutenzione e all'evoluzione dei sistemi software esistenti. Quindi, se i metodi agili devono avere successo, devono supportare la manutenzione così come lo sviluppo originale.

Possono sorgere tre tipi di problemi:

- Mancanza di documentazione del prodotto.
- Mantenere i clienti coinvolti nel processo di sviluppo.
- Mantenere la continuità del team di sviluppo.

3.4.2 Metodi agili e metodi plan-driven

Un requisito fondamentale per scalare i metodi agili è integrarli con approcci basati sulla pianificazione. Per decidere il bilanciamento tra un approccio plan-driven e uno agile, è necessario rispondere a una serie di domande:

1. È importante avere una specifica e una progettazione molto dettagliate prima di passare all'implementazione, forse per motivi contrattuali? Se sì, probabilmente è necessario utilizzare un approccio plan-driven.
2. È realistica una strategia di consegna incrementale, in cui si consegna il software ai clienti o ad altri portatori di interesse del sistema e si ottiene un feedback rapido da loro? In tal caso, puoi considerare l'utilizzo di metodi agili.
3. Quanto è grande il sistema che si sta sviluppando? I metodi agili sono più efficaci quando il sistema può essere sviluppato con un piccolo team co-localizzato che può comunicare in modo informale. Questo potrebbe non essere possibile per sistemi di grandi dimensioni che richiedono team di sviluppo più grandi, quindi potrebbe essere necessario utilizzare un approccio plan-driven.

È necessario fare una certa pianificazione, progettazione e documentazione iniziali nel processo di ingegneria dei sistemi. Alcuni dei problemi chiave sono i seguenti:

Principio	Pratica
Coinvolgimento del cliente	Questo dipende dall'aver un cliente che sia disposto e in grado di trascorrere del tempo con il team di sviluppo e che possa rappresentare tutti gli stakeholder del sistema. Spesso, i rappresentanti dei clienti hanno altre richieste sul loro tempo e non possono partecipare pienamente allo sviluppo del software. Quando ci sono stakeholder esterni, come i regolatori, è difficile rappresentare le loro opinioni al team agile.
Accogliere il cambiamento	Dare priorità ai cambiamenti può essere estremamente difficile, specialmente in sistemi con molti stakeholder. Tipicamente, ogni stakeholder attribuisce priorità diverse a cambiamenti differenti.
Consegna incrementale	Iterazioni rapide e pianificazione a breve termine per lo sviluppo non si adattano sempre ai cicli di pianificazione a lungo termine della pianificazione aziendale e del marketing. I responsabili marketing potrebbero aver bisogno di conoscere le caratteristiche del prodotto diversi mesi in anticipo per preparare una campagna di marketing efficace.
Mantenere la semplicità	Sotto pressione dai programmi di consegna, i membri del team potrebbero non avere tempo per attuare semplificazioni desiderabili del sistema.
Persone, non processi	I singoli membri del team potrebbero non avere personalità adatte per il coinvolgimento intenso che è tipico dei metodi agili e, pertanto, potrebbero non interagire bene con gli altri membri del team.

Tabella 3.4: Principi agili e pratica organizzativa

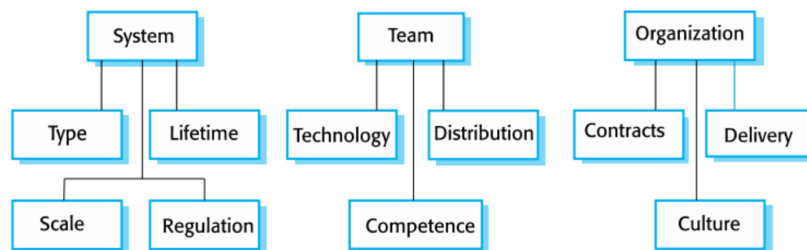


Figura 3.5: Fattori agili e plan-based

1. Quanto è grande il sistema che si sta sviluppando? I metodi agili sono più efficaci quando il sistema può essere sviluppato con un team relativamente piccolo e co-locato che può comunicare informalmente.
2. Che tipo di sistema si sta sviluppando? I sistemi che richiedono molte analisi prima dell'implementazione di solito necessitano di una progettazione abbastanza dettagliata per svolgere questa analisi.
3. Qual è la durata prevista del sistema? I sistemi di lunga durata potrebbero richiedere più documentazione di progettazione per comunicare le intenzioni originali degli sviluppatori del sistema al team di supporto.
4. Il sistema è soggetto a regolamentazione esterna? Se un sistema deve essere approvato da un regolatore esterno allora probabilmente sarà necessario produrre documentazione dettagliata come parte del caso di sicurezza del sistema.
5. Qual è il livello di competenza dei progettisti e dei programmatori nel team di sviluppo? A volte si sostiene che i metodi agili richiedano livelli di competenza più elevati rispetto agli approcci basati sulla pianificazione in cui i programmatori traducono semplicemente un design dettagliato in codice.
6. Come è organizzato il team di sviluppo? Se il team di sviluppo è distribuito potrebbe essere necessario sviluppare documenti di progettazione per comunicare tra i team di sviluppo.

7. Quali tecnologie sono disponibili per supportare lo sviluppo del sistema? Se stai sviluppando un sistema utilizzando un IDE che non ha buoni strumenti per la visualizzazione e l'analisi del codice, allora potrebbe essere necessaria più documentazione di progettazione.

3.4.3 Metodi agili per grandi sistemi

I metodi agili devono evolversi per essere utilizzati nello sviluppo software su larga scala. La ragione fondamentale di ciò è che i sistemi software di grande scala sono molto più complessi e difficili da comprendere e gestire rispetto ai sistemi o ai prodotti software di piccola scala. Sei fattori principali contribuiscono a questa complessità:

1. *Sistemi di sistemi*: I grandi sistemi sono solitamente sistemi di sistemi, collezioni di sistemi separati che comunicano tra loro, dove team separati sviluppano ciascun sistema. Spesso, questi team lavorano in luoghi diversi, a volte in fusi orari diversi.
2. *Sistemi brownfield*: I grandi sistemi sono sistemi brownfield; cioè, includono e interagiscono con diversi sistemi esistenti. Molti dei requisiti del sistema riguardano questa interazione e quindi non si prestano realmente alla flessibilità e allo sviluppo incrementale.
3. *Configurazione del sistema*: Dove diversi sistemi sono integrati per creare un sistema, una frazione significativa dello sviluppo è dedicata alla configurazione del sistema piuttosto che allo sviluppo di codice originale.
4. *Regole e normative esterne*: I grandi sistemi e i loro processi di sviluppo sono spesso vincolati da regole e normative esterne che limitano il modo in cui possono essere sviluppati.
5. *Lungo tempo di approvvigionamento e sviluppo*: I grandi sistemi hanno un lungo tempo di approvvigionamento e sviluppo. È difficile mantenere team coerenti che conoscano il sistema durante quel periodo poiché, inevitabilmente, le persone passano ad altri lavori e progetti.
6. *Stakeholder diversi*: I grandi sistemi di solito hanno un insieme diversificato di stakeholder. È praticamente impossibile coinvolgere tutti questi diversi stakeholder nel processo di sviluppo.

La Figura 3.6 mostra una panoramica del modello utilizzato dalla IBM per l'uso su larga scala dei metodi agili.

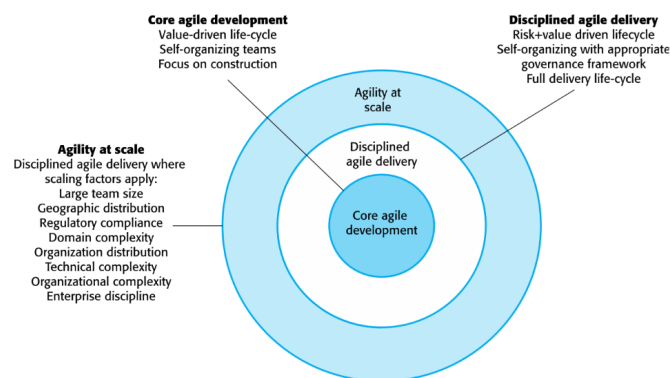


Figura 3.6: Modello agile di IBM

Non esiste un modello unico adatto a tutti i prodotti agili su larga scala, poiché il tipo di prodotto, i requisiti del cliente e le persone disponibili sono tutti diversi. Tuttavia, gli approcci per scalare i metodi agili hanno una serie di punti in comune:

1. Impossibilità di un approccio completamente incrementale all'ingegneria dei requisiti.
2. Non può esserci un unico proprietario del prodotto o rappresentante del cliente.
3. Non è possibile concentrarsi solo sul codice del sistema.
4. Devono essere progettati e utilizzati meccanismi di comunicazione tra i team.

5. Mantenere un'integrazione continua. Tuttavia, è essenziale mantenere costruzioni frequenti del sistema e rilasci regolari del sistema.

Scrum è stato adattato per lo sviluppo su larga scala. Le caratteristiche chiave dello Scrum multi-team sono:

1. *Replica dei ruoli*: Ogni team ha un Product Owner per la propria componente di lavoro e un ScrumMaster.
2. *Architetti del prodotto*: Ogni team sceglie un architetto del prodotto, e questi architetti collaborano per progettare e far evolvere l'architettura complessiva del sistema.
3. *Allineamento delle versioni*: Le date delle versioni del prodotto di ciascun team sono allineate in modo da produrre un sistema dimostrabile e completo.
4. *Scrum of Scrums*: C'è uno Scrum of Scrums quotidiano in cui i rappresentanti di ciascun team si incontrano per discutere i progressi, identificare problemi e pianificare il lavoro da svolgere quel giorno.

3.4.4 Metodi agili nelle organizzazioni

Può essere difficile introdurre metodi agili in grandi aziende per diversi motivi:

1. I project manager che non hanno esperienza con i metodi agili possono essere riluttanti ad accettare il rischio di un nuovo approccio.
2. Le grandi organizzazioni spesso hanno procedure e standard di qualità che tutti i progetti sono tenuti a seguire e, a causa della loro natura burocratica, è probabile che questi siano incompatibili con i metodi agili.
3. I metodi agili sembrano funzionare meglio quando i membri del team hanno un livello di abilità relativamente alto. Tuttavia, all'interno delle grandi organizzazioni, è probabile che ci sia una vasta gamma di competenze e abilità.
4. Potrebbe esserci una resistenza culturale ai metodi agili, specialmente in quelle organizzazioni che hanno una lunga storia nell'uso di processi ingegneristici convenzionali.

3.5 Punti chiave

- I metodi agili sono metodi di sviluppo incrementali che si concentrano sullo sviluppo rapido del software, sulle frequenti release del software, sulla riduzione delle spese generali dei processi riducendo al minimo la documentazione e sulla produzione di codice di alta qualità.
- Le pratiche di sviluppo agile includono requisiti espressi come Storie utente per le specifiche di sistema, rilasci frequenti del software, miglioramento continuo del software, sviluppo test-first e partecipazione del cliente al team di sviluppo.
- Scrum è un metodo agile che fornisce un framework per organizzare progetti agili. È incentrato su un insieme di sprint, che sono periodi di tempo fissi in cui viene sviluppato un incremento del sistema.
- Molti metodi di sviluppo pratici sono un mix di sviluppo agile e plan-driven.
- È difficile adattare i metodi agili ai sistemi di grandi dimensioni. I sistemi di grandi dimensioni necessitano di una progettazione iniziale e di una certa documentazione e le pratiche organizzative possono entrare in conflitto con l'informalità degli approcci agili.

Capitolo 4

Ingegneria dei requisiti

I requisiti per un sistema sono le descrizioni dei servizi che un sistema dovrebbe fornire e i vincoli relativi al suo funzionamento. Il processo di scoperta, analisi, documentazione e verifica di questi servizi e vincoli è chiamato **ingegneria dei requisiti (RE)**.

Il termine "requisito" non è utilizzato in modo uniforme nell'industria del software. In alcuni casi, un requisito è semplicemente una dichiarazione astratta di alto livello di un servizio che un sistema dovrebbe fornire o di un vincolo sul sistema. All'estremo opposto, è una definizione formale e dettagliata di una funzione del sistema.

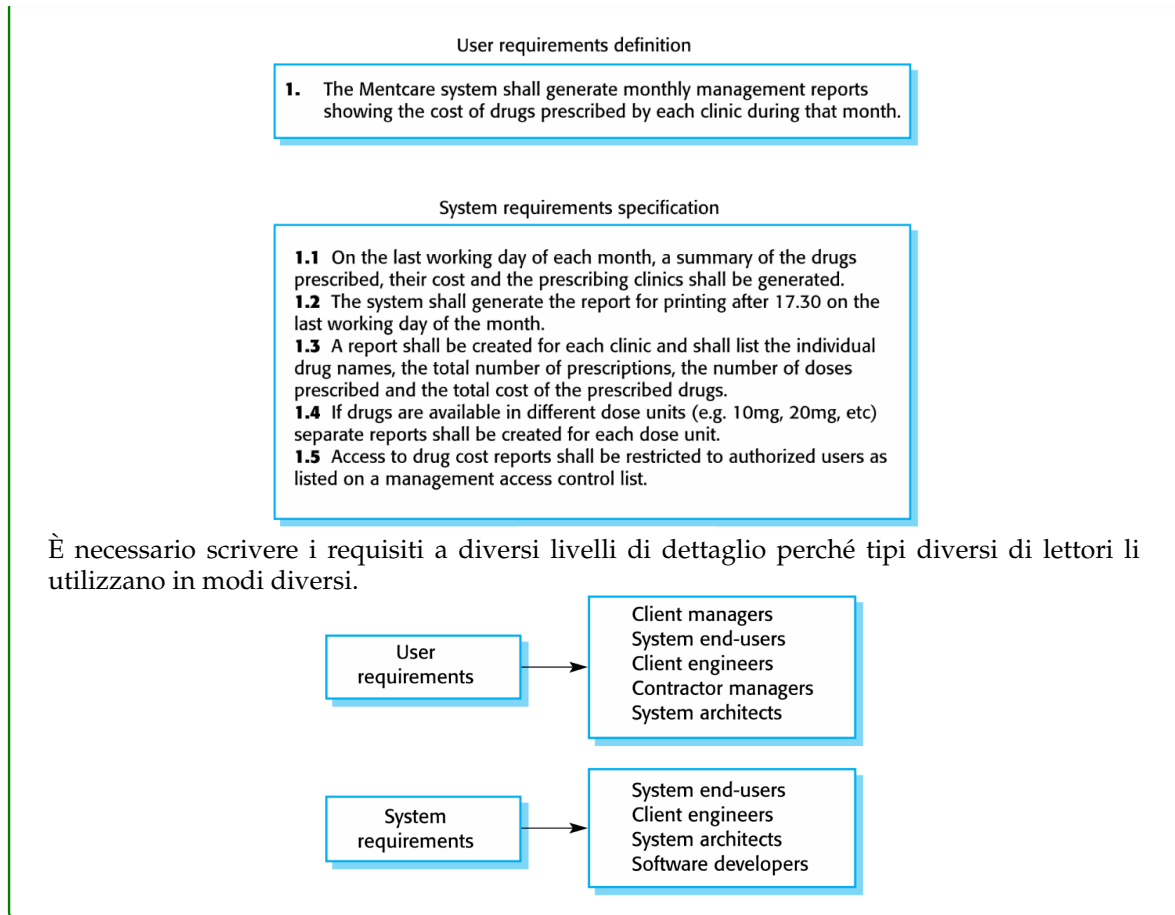
Se un'azienda desidera stipulare un contratto per un grande progetto di sviluppo software, deve definire le proprie esigenze in modo sufficientemente astratto affinché la soluzione non sia predefinita. I requisiti devono essere scritti in modo che più fornitori possano concorrere per il contratto, offrendo, forse, diversi modi di soddisfare le esigenze dell'organizzazione cliente. Una volta aggiudicato il contratto, il fornitore deve redigere una definizione del sistema per il cliente in modo più dettagliato, affinché il cliente comprenda e possa validare ciò che farà il software. Entrambi questi documenti possono essere chiamati documento dei requisiti per il sistema.

Si può distinguere tra loro utilizzando il termine "requisiti utente" per indicare i requisiti astratti di alto livello e "requisiti di sistema" per indicare la descrizione dettagliata di ciò che il sistema dovrebbe fare. I requisiti utente e i requisiti di sistema possono essere definiti come segue:

1. *Requisiti utente:* sono dichiarazioni, in linguaggio naturale e diagrammi, di quali servizi il sistema dovrebbe fornire agli utenti e dei vincoli entro i quali deve operare.
2. *Requisiti di sistema:* sono descrizioni più dettagliate delle funzioni, dei servizi e dei vincoli operativi del sistema software. Il documento dei requisiti di sistema dovrebbe definire esattamente ciò che deve essere implementato. Può far parte del contratto tra il cliente e gli sviluppatori software.

Esempio 4.0.1 – Requisiti Mentcare

L'esempio dal sistema informativo per la salute mentale dei pazienti (Mentcare) mostra come un requisito utente possa essere ampliato in diversi requisiti di sistema.



I diversi tipi di lettori del documento sono esempi di stakeholder del sistema. Gli stakeholder del sistema includono chiunque sia influenzato dal sistema in qualche modo, e quindi chiunque abbia un legittimo interesse per esso. Gli stakeholder spaziano dagli utenti finali di un sistema ai manager, fino agli stakeholder esterni come i regolatori, che certificano l'accettabilità del sistema. Ad esempio, gli stakeholder del sistema Mentcare includono:

1. I pazienti le cui informazioni sono registrate nel sistema e i loro parenti.
2. I medici responsabili della valutazione e del trattamento dei pazienti.
3. Gli infermieri che coordinano le consultazioni con i medici e somministrano alcuni trattamenti.
4. I receptionist medici che gestiscono gli appuntamenti dei pazienti.
5. Il personale IT che si occupa dell'installazione e della manutenzione del sistema.
6. Un manager dell'etica medica che deve garantire che il sistema rispetti le linee guida etiche attuali per la cura dei pazienti.
7. I manager della sanità che ottengono informazioni gestionali dal sistema.
8. Il personale addetto ai registri medici, responsabile di garantire che le informazioni del sistema siano mantenute e conservate correttamente e che le procedure di registrazione siano state implementate.

Molti metodi agili sostengono che produrre requisiti di sistema dettagliati sia una perdita di tempo poiché i requisiti cambiano rapidamente. I metodi agili solitamente utilizzano l'ingegneria dei requisiti incrementale e possono esprimere i requisiti come "user story". Ciò è pratico per i sistemi aziendali ma problematico per i sistemi che richiedono un'analisi pre-consegna (ad esempio, sistemi critici) o sistemi sviluppati da più team.

4.1 Requisiti funzionali e non funzionali

I requisiti di un sistema software sono spesso classificati come requisiti funzionali o non funzionali:

1. *Requisiti funzionali*: Questi sono dichiarazioni sui servizi che il sistema dovrebbe fornire, su come il sistema dovrebbe reagire a determinati input e su come dovrebbe comportarsi in situazioni particolari. In alcuni casi, i requisiti funzionali possono anche indicare esplicitamente cosa il sistema non dovrebbe fare.
2. *Requisiti non funzionali*: Questi sono vincoli sui servizi o sulle funzioni offerte dal sistema. Inclondono vincoli temporali, vincoli sul processo di sviluppo e vincoli imposti da standard. I requisiti non funzionali si applicano spesso all'intero sistema piuttosto che a singole funzionalità o servizi del sistema.

4.1.1 Requisiti funzionali

I requisiti funzionali per un sistema descrivono cosa il sistema dovrebbe fare. Questi requisiti dipendono dal tipo di software sviluppato, dagli utenti previsti e dall'approccio generale adottato dall'organizzazione nella scrittura dei requisiti. Quando espressi come requisiti utente, i requisiti funzionali possono essere dichiarazioni di alto livello su cosa dovrebbe fare il sistema. I requisiti funzionali del sistema espandono i requisiti utente e sono scritti per gli sviluppatori del sistema. Essi devono descrivere in dettaglio le funzioni del sistema, i loro input e output, e le eccezioni.

Esempio 4.1.1 – Requisiti funzionali Mentcare

Ecco alcuni esempi di requisiti funzionali per il sistema Mentcare, utilizzato per mantenere le informazioni sui pazienti che ricevono trattamenti per problemi di salute mentale:

1. Un utente deve poter cercare le liste degli appuntamenti per tutte le cliniche.
2. Il sistema deve generare ogni giorno, per ogni clinica, una lista dei pazienti che si prevede frequentino gli appuntamenti in quel giorno.
3. Ogni membro del personale che utilizza il sistema deve essere identificato univocamente dal suo numero dipendente a otto cifre.

L'imprecisione nella specifica dei requisiti può portare a dispute tra clienti e sviluppatori software. È naturale che uno sviluppatore interpreti un requisito ambiguo in modo tale da semplificare la sua implementazione.

Ad esempio, il primo requisito del sistema Mentcare nella lista sopra afferma che un utente deve poter cercare le liste degli appuntamenti per tutte le cliniche. La ragione di questo requisito è che i pazienti con problemi di salute mentale a volte si confondono. Potrebbero avere un appuntamento in una clinica ma in realtà recarsi in un'altra. Un membro del personale medico che specifica un requisito di ricerca potrebbe aspettarsi che "ricerca" significhi che, dato un nome di paziente, il sistema cerchi quel nome in tutti gli appuntamenti di tutte le cliniche. Tuttavia, questo non è esplicitato nel requisito. Gli sviluppatori di sistema potrebbero interpretare il requisito in modo che sia più facile da implementare. La loro funzione di ricerca potrebbe richiedere all'utente di scegliere una clinica e poi eseguire la ricerca dei pazienti che hanno frequentato quella clinica.

Idealmente, la specifica dei requisiti funzionali di un sistema dovrebbe essere completa e coerente. Completezza significa che tutti i servizi e le informazioni richiesti dall'utente dovrebbero essere definiti. Coerenza significa che i requisiti non dovrebbero essere contraddittori.

In pratica, è possibile ottenere la coerenza e la completezza dei requisiti solo per sistemi software molto piccoli.

4.1.2 Requisiti non funzionali

I requisiti non funzionali, come suggerisce il nome, sono requisiti che non riguardano direttamente i servizi specifici forniti dal sistema ai suoi utenti. Di solito, questi requisiti non funzionali specificano o vincolano le caratteristiche del sistema nel suo complesso. Possono riguardare proprietà emergenti del sistema, come l'affidabilità, il tempo di risposta e l'uso della memoria. In alternativa, possono

definire vincoli sull'implementazione del sistema, come le capacità dei dispositivi di input/output o le rappresentazioni dei dati utilizzate nelle interfacce con altri sistemi.

I requisiti non funzionali sono spesso più critici dei singoli requisiti funzionali. Gli utenti del sistema possono solitamente trovare modi per aggirare una funzione del sistema che non soddisfa completamente le loro esigenze. Tuttavia, non soddisfare un requisito non funzionale può significare che l'intero sistema è inutilizzabile.

L'implementazione di questi requisiti può essere distribuita in tutto il sistema, per due motivi:

1. I requisiti non funzionali possono influenzare l'architettura complessiva di un sistema piuttosto che i singoli componenti. Ad esempio, per garantire che i requisiti di prestazione siano soddisfatti in un sistema embedded, potrebbe essere necessario organizzare il sistema per minimizzare le comunicazioni tra i componenti.
2. Un singolo requisito non funzionale, come un requisito di sicurezza, può generare diversi requisiti funzionali correlati che definiscono nuovi servizi di sistema necessari per implementare il requisito non funzionale. Inoltre, può generare requisiti che vincolano quelli esistenti; ad esempio, potrebbe limitare l'accesso alle informazioni nel sistema.

La figura 4.1 rappresenta una classificazione dei requisiti non funzionali. Da questo diagramma, si può vedere che i requisiti non funzionali possono derivare da:

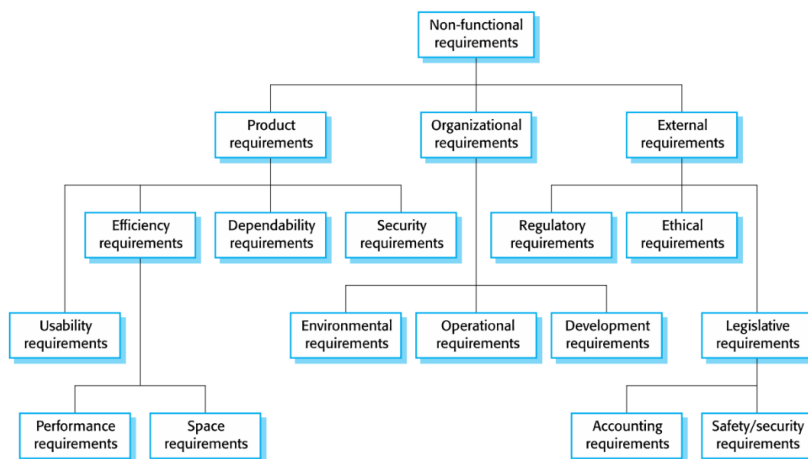


Figura 4.1: Tipi di requisiti non funzionali

1. *Requisiti di prodotto*: questi requisiti specificano che il prodotto consegnato deve comportarsi in un modo particolare. Esempi includono requisiti di prestazione per la velocità di esecuzione del sistema e la quantità di memoria richiesta.
2. *Requisiti organizzativi*: questi requisiti sono conseguenza di politiche e procedure organizzative. Esempi includono requisiti del processo operativo, requisiti del processo di sviluppo o gli standard di processo da utilizzare.
3. *Requisiti esterni*: requisiti derivati da fattori esterni al sistema e al suo processo di sviluppo. Questi possono includere requisiti normativi, requisiti legislativi e requisiti etici.

Esempio 4.1.2 – Requisiti di prodotto, organizzativi ed esterni Mentcare

Vengono mostrati esempi di requisiti di prodotto, organizzativi ed esterni che potrebbero essere inclusi nella specifica del sistema Mentcare.

1. Requisito di prodotto.

Il sistema Mentcare deve essere disponibile per tutte le cliniche durante l'orario di lavoro normale (lun-ven, 08:30–17:30). Il tempo di inattività durante l'orario di lavoro normale non deve superare i cinque secondi in un giorno.

2. Requisito organizzativo.

Gli utenti del sistema Mentcare devono autenticarsi utilizzando la loro carta d'identità dell'autorità sanitaria.

3. Requisito esterno.

Il sistema deve implementare le disposizioni sulla privacy dei pazienti come stabilito in HStan-03-2006-priv.

Il requisito di prodotto è un requisito di disponibilità che definisce quando il sistema deve essere disponibile e il tempo di inattività consentito ogni giorno. Il requisito organizzativo specifica come gli utenti si autenticano al sistema. Il requisito esterno deriva dalla necessità che il sistema sia conforme alla legislazione sulla privacy.

Un problema comune con i requisiti non funzionali è che gli stakeholder propongono requisiti come obiettivi generali, come la facilità d'uso, la capacità del sistema di recuperare da un guasto o una rapida risposta dell'utente. Gli obiettivi esprimono buone intenzioni ma causano problemi agli sviluppatori di sistema poiché lasciano spazio a interpretazioni. Ad esempio, il seguente obiettivo di sistema è tipico di come un manager potrebbe esprimere i requisiti di usabilità:

"Il sistema dovrebbe essere facile da usare per il personale medico e dovrebbe essere organizzato in modo tale da ridurre al minimo gli errori dell'utente."

È impossibile verificare oggettivamente l'obiettivo del sistema, ma nella seguente descrizione si può almeno includere la strumentazione software per contare gli errori commessi dagli utenti durante i test del sistema:

"Il personale medico deve essere in grado di utilizzare tutte le funzioni del sistema dopo due ore di formazione. Dopo questa formazione, il numero medio di errori commessi dagli utenti esperti non deve superare i due all'ora di utilizzo del sistema."

Ogni volta che possibile, i requisiti non funzionali dovrebbero essere scritti in modo quantitativo in modo che possano essere oggettivamente testati.

Proprietà	Misura
Velocità	Transazioni elaborate al secondo Tempo di risposta utente/evento Tempo di aggiornamento dello schermo
Dimensione	Megabyte Numero di chip ROM
Facilità d'uso	Tempo di formazione Numero di schermate di aiuto
Affidabilità	Tempo medio tra i guasti Probabilità di indisponibilità Tasso di occorrenza dei guasti Disponibilità
Robustezza	Tempo di riavvio dopo un guasto Percentuale di eventi che causano guasti Probabilità di corruzione dei dati in caso di guasto
Portabilità	Percentuale di istruzioni dipendenti dal sistema target Numero di sistemi target

Tabella 4.1: Metriche per specificare requisiti non funzionali

4.2 Processi di ingegneria dei requisiti

Come già discusso, l'ingegneria dei requisiti comporta tre attività chiave. Queste sono: scoprire i requisiti interagendo con gli stakeholders (elicitazione e analisi); convertire questi requisiti in una

forma standard (specifica); e verificare che i requisiti definiscano effettivamente il sistema desiderato dal cliente (validazione). Nella pratica, l'ingegneria dei requisiti è un processo iterativo in cui le attività sono interconnesse. La Figura 4.2 mostra questa interconnessione.

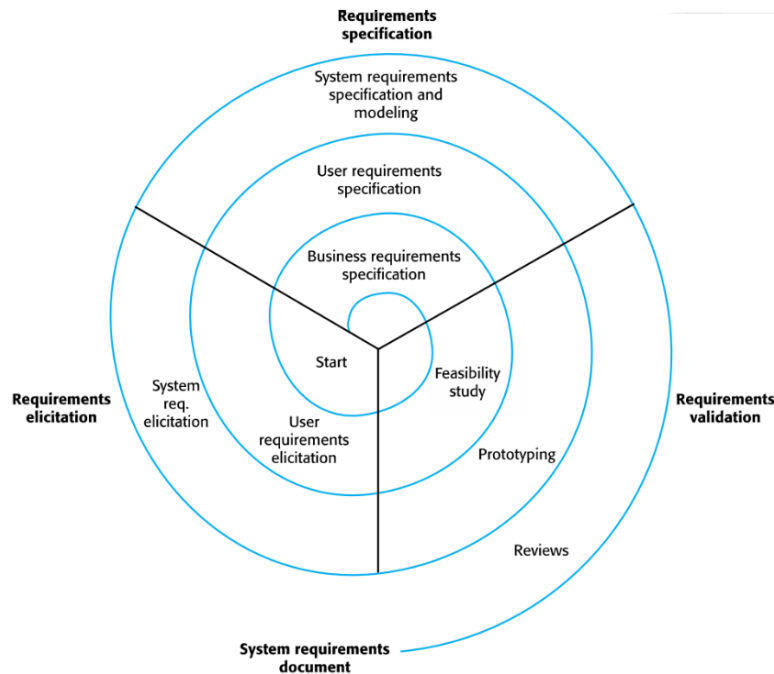


Figura 4.2: Una visione a spirale del processo di ingegneria dei requisiti

4.3 Elicitazione dei requisiti

Durante l'elicitazione dei requisiti, gli ingegneri del software collaborano con gli stakeholders per ottenere informazioni sul dominio dell'applicazione, le attività lavorative, i servizi e le funzionalità del sistema richieste, le prestazioni attese dal sistema, i vincoli hardware e così via.

Elicitare e comprendere i requisiti degli stakeholders del sistema è un processo difficile per diverse ragioni:

1. *Difficoltà nell'esprimere i requisiti:* Gli stakeholders spesso non sanno esattamente cosa vogliono da un sistema informatico, se non in termini generali.
2. *Comprensione dei requisiti:* Gli ingegneri dei requisiti potrebbero non comprendere pienamente i requisiti del cliente, il che può portare a incomprensioni.
3. *Diversità dei requisiti:* Diversi stakeholder, con esigenze diverse, potrebbero esprimere i loro requisiti in modi differenti.
4. *Fattori politici:* I fattori politici possono influenzare i requisiti del sistema.
5. *Ambiente dinamico:* L'ambiente economico e aziendale in cui si svolge l'analisi è dinamico e cambia inevitabilmente durante il processo. Questo può portare a modifiche nei requisiti prioritari e all'emergere di nuovi requisiti da stakeholder.

Un modello di processo dell'elicitazione e dell'analisi dei requisiti è mostrato nella Figura 4.3.

Le attività principali del processo sono:

1. *Scoperta e comprensione dei requisiti:* Questo è il processo di interazione con gli stakeholders del sistema per scoprire i loro requisiti. Durante questa attività vengono scoperti anche i requisiti del dominio provenienti dagli stakeholders e dalla documentazione.

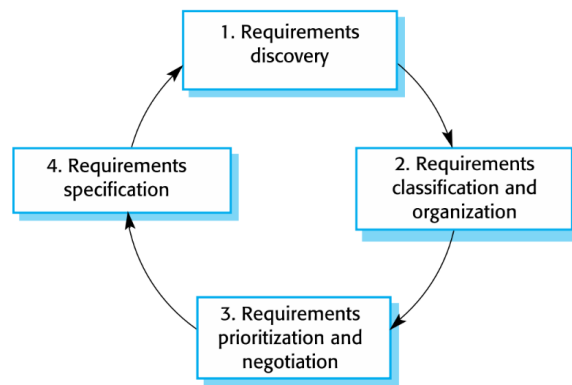


Figura 4.3: Il processo di individuazione e analisi dei requisiti

2. *Classificazione e organizzazione dei requisiti*: Questa attività prende la raccolta non strutturata dei requisiti, raggruppa i requisiti correlati e li organizza in cluster coerenti.
3. *Prioritizzazione e negoziazione dei requisiti*: Questa attività si occupa di dare priorità ai requisiti e risolvere i conflitti attraverso la negoziazione.
4. *Documentazione dei requisiti*: I requisiti vengono documentati e inseriti nel ciclo successivo della spirale.

4.3.1 Tecniche di elicitazione dei requisiti

Esistono due approcci fondamentali per l'elicitazione dei requisiti:

1. Interviste, in cui si parla con le persone di ciò che fanno.
2. Osservazione o etnografia, in cui si osservano le persone mentre svolgono il loro lavoro per vedere quali strumenti utilizzano, come li utilizzano e così via.

È consigliabile utilizzare una combinazione di interviste e osservazione per raccogliere informazioni da cui derivare i requisiti.

Interviste

Interviste formali o informali con gli stakeholder del sistema fanno parte della maggior parte dei processi di ingegneria dei requisiti. Le interviste possono essere di due tipi:

1. Interviste chiuse, in cui lo stakeholder risponde a una serie di domande predefinite.
2. Interviste aperte, in cui vengono esplorate varie questioni con gli stakeholder.

In pratica, le interviste con gli stakeholder sono normalmente una combinazione di entrambe queste tipologie. Potrebbe essere necessario ottenere risposte a determinate domande, ma queste di solito portano a ulteriori questioni che vengono discusse in modo meno strutturato.

Le interviste sono utili per ottenere una comprensione generale di ciò che fanno gli stakeholder, di come potrebbero interagire con il nuovo sistema e delle difficoltà che incontrano con i sistemi attuali. Tuttavia, a meno che non si abbia un prototipo di sistema da mostrare, non bisogna aspettarsi che gli stakeholder suggeriscano requisiti specifici e dettagliati. È difficile per tutti visualizzare come potrebbe essere un sistema. È necessario analizzare le informazioni raccolte e generare i requisiti da queste.

Per essere un intervistatore efficace, è necessario tenere a mente due cose:

1. Dovresti essere mentalmente aperto, evitare idee preconcepite sui requisiti ed essere disposto ad ascoltare gli stakeholder.
2. Dovresti stimolare l'intervistato per avviare le discussioni utilizzando una domanda di avvio o una proposta di requisiti, o lavorando insieme su un prototipo di sistema.

L'elicitazione delle conoscenze del dominio attraverso le interviste può essere difficile per due motivi:

1. Tutti gli specialisti di applicazioni usano un gergo specifico per il loro ambito lavorativo che gli ingegneri dei requisiti potrebbero fraintendere.
2. Alcune conoscenze del dominio sono così familiari agli stakeholder che trovano difficile spiegarle o pensano che siano così fondamentali da non valere la pena menzionarle.

Etnografia

L'etnografia è una tecnica di osservazione che può essere utilizzata per comprendere i processi operativi e aiutare a derivare i requisiti per il software che dovrà supportare questi processi. Un analista si immerge nell'ambiente di lavoro in cui il sistema verrà utilizzato. Si osserva il lavoro quotidiano e si prendono appunti sui compiti effettivi in cui sono coinvolti i partecipanti. Il valore dell'etnografia risiede nel fatto che aiuta a scoprire requisiti impliciti del sistema che riflettono i modi effettivi in cui le persone lavorano, piuttosto che i processi formali definiti dall'organizzazione.

L'etnografia è particolarmente efficace per scoprire due tipi di requisiti:

1. Requisiti derivati dal modo in cui le persone lavorano realmente, piuttosto che dal modo in cui le definizioni dei processi aziendali dicono che dovrebbero lavorare.
2. Requisiti derivati dalla cooperazione e dalla consapevolezza delle attività altrui. La consapevolezza di ciò che fanno le altre persone porta a cambiamenti nei modi in cui facciamo le cose.

L'etnografia è efficace per comprendere i processi esistenti ma non può identificare nuove caratteristiche che dovrebbero essere aggiunte a un sistema.

L'etnografia può essere combinata con lo sviluppo di un prototipo di sistema (Figura 4.4).

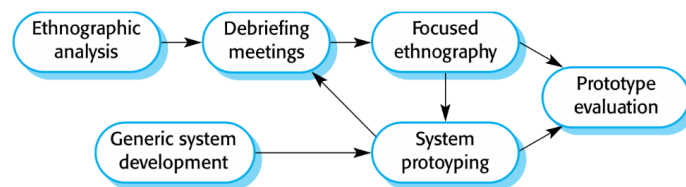


Figura 4.4: Etnografia e prototipazione per l'analisi dei requisiti

4.3.2 Storie e scenari

Gli scenari e le storie degli utenti sono esempi concreti di come un sistema può essere utilizzato. Le persone trovano più facile relazionarsi a esempi di vita reale rispetto a descrizioni astratte. Storie e scenari sono modi per catturare questo tipo di informazioni. Storie e scenari sono essenzialmente la stessa cosa: una descrizione di come il sistema possa essere utilizzato per svolgere un determinato compito.

Esempio 4.3.1 – Storie e scenari iLearn

Questa storia descrive una situazione in una scuola primaria (elementare) in cui l'insegnante utilizza l'ambiente per supportare progetti degli studenti sull'industria della pesca.

Jack è un insegnante di scuola primaria a Ullapool (un villaggio nel nord della Scozia). Ha deciso che un progetto di classe sarà incentrato sull'industria della pesca nella zona, analizzando la storia, lo sviluppo e l'impatto economico della pesca. Come parte di questo, agli alunni viene chiesto di raccogliere e condividere ricordi di parenti, usare archivi di giornali e collezionare vecchie fotografie legate alla pesca e alle comunità di pescatori della zona. Gli alunni usano un wiki di iLearn per raccogliere storie sulla pesca e SCRAN (un sito di risorse storiche) per accedere agli archivi di giornali e fotografie. Tuttavia, Jack ha anche bisogno di un sito per condividere foto, poiché vuole che gli alunni possano scattare foto, commentarle reciprocamente e caricare scansioni di vecchie fotografie che potrebbero avere in famiglia.

Jack invia un'email a un gruppo di insegnanti di scuola primaria, di cui è membro, per vedere se qualcuno può consigliargli un sistema adatto. Due insegnanti rispondono e entrambi suggeriscono di utilizzare KidsTakePics, un sito di condivisione di foto che consente agli insegnanti di controllare e moderare i contenuti. Poiché KidsTakePics non è integrato con il servizio di autenticazione di iLearn, Jack crea un account per sé e uno per la classe. Usa il servizio di configurazione di iLearn per aggiungere KidsTakePics ai servizi visualizzati dagli alunni della sua classe, in modo che, quando si collegano, possano utilizzare immediatamente il sistema per caricare foto dai loro dispositivi mobili e dai computer della classe.

- *Ipotesi iniziale:* Un utente o un gruppo di utenti ha una o più fotografie digitali da caricare sul sito di condivisione foto. Queste sono salvate su un tablet o su un computer portatile. Gli utenti si sono collegati con successo a KidsTakePics.
- *Normale:* L'utente sceglie l'opzione per caricare le foto e viene invitato a selezionare le foto da caricare sul proprio computer e a scegliere il nome del progetto sotto il quale verranno archiviate le foto. Gli dovrebbe anche essere data l'opzione di inserire parole chiave associate a ciascuna foto caricata. Le foto caricate vengono nominate creando una combinazione del nome utente e del nome del file della foto sul computer locale.

Al termine del caricamento, il sistema invia automaticamente un'email al moderatore del progetto per chiedergli di controllare il nuovo contenuto e genera un messaggio sullo schermo per l'utente, informandolo che il caricamento è stato completato.

- *Cosa può andare storto:* Nessun moderatore è associato al progetto selezionato. Viene automaticamente generata un'email per l'amministratore della scuola chiedendogli di nominare un moderatore di progetto. Agli utenti dovrebbe essere comunicato che potrebbe esserci un ritardo nel rendere visibili le loro foto.

Foto con lo stesso nome sono già state caricate dallo stesso utente. All'utente dovrebbe essere chiesto se desidera ricaricare le foto con lo stesso nome, rinominare le foto o annullare il caricamento. Se sceglie di ricaricare le foto, le originali vengono sovrascritte. Se sceglie di rinominare le foto, viene generato automaticamente un nuovo nome aggiungendo un numero al nome del file esistente.

- *Altre attività:* Il moderatore potrebbe essere collegato al sistema e approvare le foto man mano che vengono caricate.
- *Stato del sistema al termine:* L'utente è collegato. Le foto selezionate sono state caricate e assegnate con lo stato "in attesa di moderazione". Le foto sono visibili al moderatore e all'utente che le ha caricate.

A livello generale, uno scenario può includere:

1. Una descrizione di cosa si aspetta il sistema e gli utenti quando inizia lo scenario.
2. Una descrizione del flusso normale degli eventi nello scenario.
3. Una descrizione di ciò che può andare storto e di come gestire i problemi che ne derivano.
4. Informazioni su altre attività che potrebbero essere in corso contemporaneamente.
5. Una descrizione dello stato del sistema al termine dello scenario.

4.4 Specifica dei requisiti

La specifica dei requisiti è il processo di scrittura dei requisiti utente e dei requisiti di sistema in un documento di requisiti. Idealmente, i requisiti utente e di sistema dovrebbero essere chiari, univoci, facili da comprendere, completi e coerenti. I requisiti utente devono essere comprensibili per gli utenti finali e i clienti che non hanno una formazione tecnica. I requisiti di sistema sono requisiti più dettagliati e possono includere più informazioni tecniche. I requisiti possono essere parte di un contratto per lo sviluppo del sistema, è quindi importante che siano il più completi possibile.

Notazione	Descrizione
Linguaggio naturale	I requisiti sono scritti usando frasi numerate in linguaggio naturale. Ogni frase dovrebbe esprimere un requisito.
Linguaggio naturale strutturato	I requisiti sono scritti in linguaggio naturale su un modulo o modello standard. Ogni campo fornisce informazioni su un aspetto del requisito.
Linguaggi di descrizione del design	Questo approccio utilizza un linguaggio simile a un linguaggio di programmazione, ma con caratteristiche più astratte per specificare i requisiti definendo un modello operativo del sistema. Questo metodo è ora raramente usato, ma può essere utile per le specifiche delle interfacce.
Notazioni grafiche	Modelli grafici, integrati da annotazioni testuali, vengono utilizzati per definire i requisiti funzionali del sistema; i diagrammi dei casi d'uso e di sequenza UML sono comunemente utilizzati.
Specifiche matematiche	Queste notazioni si basano su concetti matematici come macchine a stati finiti o insiemi. Sebbene queste specifiche prive di ambiguità possano ridurre l'ambiguità in un documento di requisiti, la maggior parte dei clienti non comprende una specifica formale. Non possono verificare che essa rappresenti ciò che desiderano e sono riluttanti ad accettarla come contratto per il sistema.

Tabella 4.2: Modi di scrivere una specifica dei requisiti di sistema

Idealmente, i requisiti di sistema dovrebbero descrivere solo il comportamento esterno del sistema e i suoi vincoli operativi, senza riguardare il modo in cui il sistema dovrebbe essere progettato o implementato. Tuttavia, nella pratica, i requisiti e la progettazione sono inseparabili:

1. Potrebbe essere necessario progettare un'architettura iniziale del sistema per aiutare a strutturare la specifica dei requisiti.
2. Nella maggior parte dei casi, i sistemi devono interoperare con sistemi esistenti, il che vincola la progettazione e impone requisiti sul nuovo sistema.
3. L'uso di una specifica architettura per soddisfare requisiti non funzionali potrebbe essere necessario. Un regolatore esterno che deve certificare la sicurezza del sistema potrebbe specificare che venga utilizzato un design architettonico già certificato.

4.4.1 Specifica in linguaggio naturale

I requisiti sono scritti come frasi in linguaggio naturale integrate da diagrammi e tabelle. È espressivo, intuitivo e universale. Ciò significa che i requisiti possono essere compresi da utenti e clienti. Per ridurre i malintesi quando si scrivono requisiti in linguaggio naturale, raccomando di seguire queste semplici linee guida:

1. Creare un formato standard e assicurarsi che tutte le definizioni dei requisiti aderiscano a tale formato.
2. Utilizzare un linguaggio coerente per distinguere tra requisiti obbligatori e desiderabili.
3. Utilizzare l'evidenziazione del testo (grassetto, corsivo o colore) per mettere in risalto le parti chiave del requisito.
4. Non presumere che i lettori comprendano il linguaggio tecnico o di ingegneria del software. Evitare, dove possibile, l'uso di gergo, abbreviazioni e acronimi.
5. Includere una spiegazione (motivazione) del perché un requisito è necessario.

I problemi che possono sorgere utilizzando il linguaggio naturale per i requisiti sono:

1. *Mananza di chiarezza*: La precisione è difficile senza rendere il documento difficile da leggere.
2. *Confusione dei requisiti*: I requisiti funzionali e non funzionali tendono a essere confusi.
3. *Fusione dei requisiti*: Possono essere espressi insieme diversi requisiti.

Esempio 4.4.1 – Requisiti per la pompa per insulina

Vengono mostrate come applicare le linee guida, includendo due requisiti per il software incorporato nella pompa per insulina automatizzata.

1. Il sistema deve misurare la glicemia e somministrare insulina, se necessario, ogni 10 minuti. (Le variazioni della glicemia sono relativamente lente, quindi una misurazione più frequente non è necessaria; una misurazione meno frequente potrebbe portare a livelli di zucchero inutilmente alti.)
2. Il sistema deve eseguire una routine di autotest ogni minuto, con le condizioni da testare e le azioni associate definite. (Una routine di autotest può individuare problemi hardware e software e avvisare l'utente del fatto che il normale funzionamento potrebbe non essere possibile.)

4.4.2 Specifiche strutturate

Le specifiche strutturate in linguaggio naturale rappresentano un metodo per scrivere i requisiti di sistema utilizzando un formato standard, anziché un testo libero.

Quando si utilizza un formato standard per specificare i requisiti funzionali, dovrebbero essere incluse le seguenti informazioni:

1. Descrizione della funzione o entità specificata.
2. Descrizione degli input e della loro origine.
3. Descrizione degli output e della loro destinazione.
4. Informazioni sui dati necessari per il calcolo o altre entità richieste nel sistema.
5. Descrizione dell'azione da eseguire.
6. Se si utilizza un approccio funzionale, una preconditione che indica ciò che deve essere vero prima che la funzione sia chiamata, e una postcondizione che indica ciò che è vero dopo che la funzione è stata chiamata.
7. Descrizione degli effetti collaterali (se presenti) dell'operazione.

Tuttavia, può comunque risultare difficile scrivere requisiti in modo chiaro e non ambiguo, specialmente quando è necessario specificare calcoli complessi (ad esempio, come calcolare la dose di insulina). Per affrontare questo problema, è possibile aggiungere informazioni extra ai requisiti in linguaggio naturale, ad esempio tramite tabelle o modelli grafici del sistema. Le tabelle sono particolarmente utili quando ci sono diverse situazioni alternative e si devono descrivere le azioni da intraprendere per ciascuna di esse.

Esempio 4.4.2 – Specifica strutturata per la pompa per insulina

Insulin Pump/Control Software/SRS/3.3.2

- *Funzione:* Calcolo della dose di insulina: livello sicuro di zucchero.
- *Descrizione:* Calcola la dose di insulina da somministrare quando il livello di zucchero attuale misurato rientra nella zona sicura tra 3 e 7 unità.
- *Input:* Lettura attuale del livello di zucchero (r_2); le due letture precedenti (r_0 e r_1).
- *Fonte:* Lettura attuale del livello di zucchero dal sensore. Altre letture dalla memoria.
- *Output:* CompDose - la dose di insulina da somministrare.
- *Destinazione:* Ciclo di controllo principale.
- *Azione:* CompDose è zero se il livello di zucchero è stabile o in diminuzione, oppure se il livello è in aumento ma il tasso di aumento è in diminuzione. Se il livello è in aumento e

il tasso di aumento sta aumentando, allora *CompDose* è calcolato dividendo la differenza tra il livello di zucchero attuale e quello precedente per 4 e arrotondando il risultato. Se il risultato è arrotondato a zero, *CompDose* viene impostato alla dose minima che può essere somministrata.

- *Requisiti*: Due letture precedenti per calcolare il tasso di variazione del livello di zucchero.
- *Pre-condizione*: Il serbatoio di insulina contiene almeno la dose singola massima consentita di insulina.
- *Post-condizione*: r_0 viene sostituito da r_1 , quindi r_1 viene sostituito da r_2 .
- *Effetti collaterali*: Nessuno.

La pompa per insulina basa i suoi calcoli del fabbisogno di insulina sul tasso di variazione dei livelli di zucchero nel sangue. Si può usare una descrizione tabellare di come il tasso di variazione dello zucchero nel sangue viene utilizzato per calcolare la quantità di insulina da somministrare.

Condizione	Azione
Livello di zucchero in diminuzione ($r_2 < r_1$)	$CompDose = 0$
Livello di zucchero stabile ($r_2 = r_1$)	$CompDose = 0$
Livello di zucchero in aumento e tasso di aumento in diminuzione ($(r_2 - r_1) < (r_1 - r_0)$)	$CompDose = 0$
Livello di zucchero in aumento e tasso di aumento stabile o in aumento ($(r_2 - r_1) \geq (r_1 - r_0)$)	$CompDose = \text{round}((r_2 - r_1)/4)$ if rounded result=0 then $CompDose = MinimumDose$

4.4.3 Use cases

I casi d'uso sono un modo per descrivere le interazioni tra gli utenti e un sistema utilizzando un modello grafico e un testo strutturato. Sono diventati una caratteristica fondamentale del Linguaggio di Modellazione Unificato (UML). Nella loro forma più semplice, un caso d'uso identifica gli attori coinvolti in un'interazione e nomina il tipo di interazione. L'insieme dei casi d'uso rappresenta tutte le possibili interazioni che saranno descritte nei requisiti di sistema.

La Figura 4.5, mostra alcuni dei casi d'uso per il sistema Mentcare.

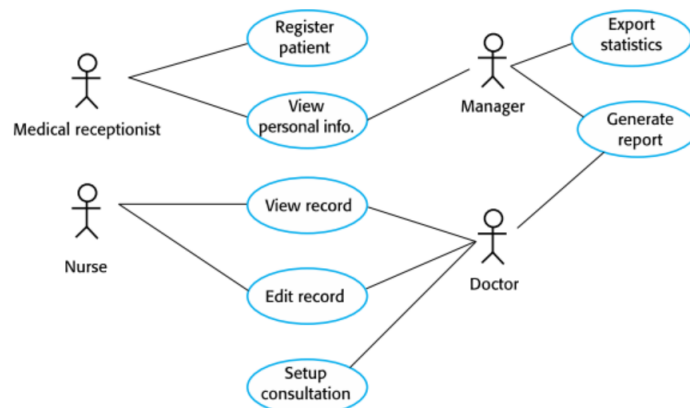


Figura 4.5: Casi d'uso per il sistema Mentcare

4.4.4 Documento sui requisiti software

Il documento dei requisiti software (chiamato talvolta specifica dei requisiti software o SRS) è una dichiarazione ufficiale di ciò che gli sviluppatori di sistemi dovrebbero implementare. Può includere sia i requisiti degli utenti per un sistema sia una specifica dettagliata dei requisiti di sistema. Non è un documento di progettazione e, per quanto possibile, dovrebbe impostare cosa dovrebbe fare il sistema piuttosto che come dovrebbe farlo.

Il documento dei requisiti ha un insieme di utenti diversificato. La Figura 4.6 mostra i possibili utenti del documento e come lo utilizzano.

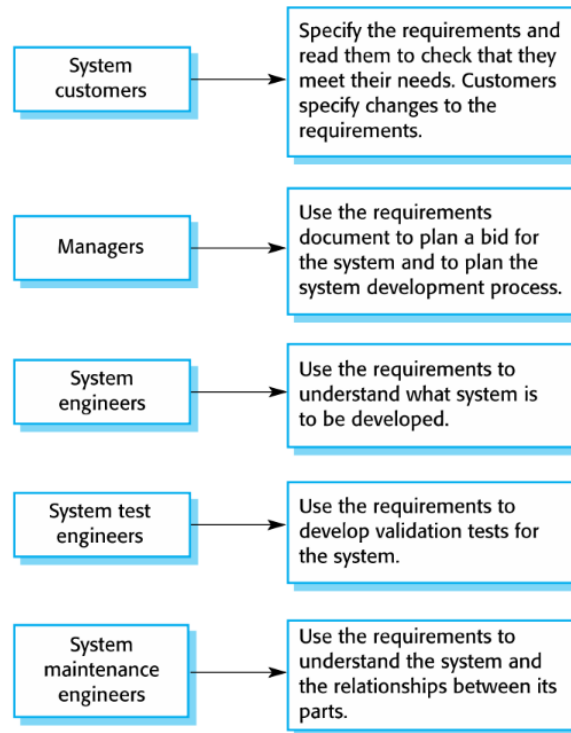


Figura 4.6: Utenti di un documento di requisiti

Il livello di dettaglio da includere in un documento di requisiti dipende dal tipo di sistema che viene sviluppato e dal processo di sviluppo utilizzato. I sistemi critici necessitano di requisiti dettagliati perché la sicurezza e l'affidabilità devono essere analizzate in dettaglio per trovare possibili errori nei requisiti. Se viene utilizzato un processo di sviluppo iterativo interno, il documento dei requisiti può essere meno dettagliato. I dettagli possono essere aggiunti ai requisiti e le ambiguità risolte durante lo sviluppo del sistema.

Sono stati progettati standard per i documenti dei requisiti, ad esempio lo standard IEEE (vedi Tabella 4.3). Questi sono per lo più applicabili ai requisiti per grandi progetti di ingegneria dei sistemi.

4.5 Validazione dei requisiti

La validazione dei requisiti è il processo di verifica che i requisiti definiscano il sistema che il cliente desidera realmente. La validazione dei requisiti è di importanza cruciale, poiché errori in un documento dei requisiti possono portare a costi di rifacimento significativi quando questi problemi vengono scoperti durante lo sviluppo o dopo che il sistema è in servizio.

Il costo per correggere un problema nei requisiti attraverso una modifica del sistema è solitamente molto maggiore rispetto alla correzione di errori di progettazione o di codifica. Una modifica dei requisiti di solito comporta anche cambiamenti nella progettazione e nell'implementazione del sistema, che devono poi essere nuovamente testati.

Durante il processo di validazione dei requisiti, devono essere eseguiti diversi tipi di controlli sui requisiti nel documento dei requisiti:

1. *Controlli di validità*: verificano che i requisiti riflettano le reali esigenze degli utenti del sistema.
2. *Controlli di coerenza*: i requisiti nel documento non devono essere in conflitto tra loro, ossia non devono esserci vincoli contraddittori o descrizioni differenti della stessa funzione di sistema.
3. *Controlli di completezza*: il documento dei requisiti dovrebbe includere i requisiti che definiscono tutte le funzioni e i vincoli previsti dall'utente del sistema.
4. *Controlli di realismo*: basandosi sulla conoscenza delle tecnologie esistenti, i requisiti devono essere verificati per assicurarsi che possano essere implementati entro il budget proposto per il sistema.
5. *Verificabilità*: i requisiti del sistema devono sempre essere scritti in modo che siano verificabili.

Diverse tecniche di validazione dei requisiti possono essere utilizzate individualmente o in combinazione tra loro:

1. *Revisione dei requisiti*: i requisiti sono analizzati sistematicamente da un team di revisori che verifica la presenza di errori e incoerenze.
2. *Prototipazione*: consiste nello sviluppo di un modello eseguibile di un sistema e nel suo utilizzo con utenti finali e clienti per verificare se soddisfa le loro esigenze e aspettative.
3. *Generazione di casi di test*: sviluppo di test per i requisiti per verificarne la testabilità.

Si dovrebbero tenere delle revisioni regolari mentre si formula la definizione dei requisiti. Sia il personale del cliente che quello dell'appaltatore dovrebbero essere coinvolti nelle revisioni. Le revisioni possono essere formali (con documenti completati) o informali. Una buona comunicazione tra sviluppatori, clienti e utenti può risolvere i problemi in una fase iniziale.

4.6 Requisiti in continua evoluzione

I requisiti per i grandi sistemi software cambiano sempre. La maggior parte delle modifiche ai requisiti del sistema si verifica a causa di cambiamenti nell'ambiente aziendale del sistema:

1. L'ambiente aziendale e tecnico del sistema cambia sempre dopo l'installazione. Potrebbe essere introdotto nuovo hardware e l'hardware esistente aggiornato. Potrebbe essere necessario interfacciare il sistema con altri sistemi. Le priorità aziendali possono cambiare e potrebbero essere introdotte nuove leggi e normative che richiedono la conformità del sistema.
2. Chi paga per un sistema e chi lo utilizza sono raramente le stesse persone. I clienti del sistema impongono requisiti a causa di vincoli organizzativi e di budget. Questi possono entrare in conflitto con i requisiti degli utenti finali, e, dopo la consegna, potrebbero dover essere aggiunte nuove funzionalità per supportare gli utenti affinché il sistema raggiunga i suoi obiettivi.
3. I grandi sistemi di solito hanno una comunità di stakeholder diversificata, con stakeholder che hanno requisiti differenti. Le loro priorità possono essere in conflitto o contraddittorie. I requisiti finali del sistema sono inevitabilmente un compromesso, e alcuni stakeholder devono essere prioritizzati. Con l'esperienza, spesso si scopre che è necessario modificare l'equilibrio del supporto fornito ai diversi stakeholder e riprioritizzare i requisiti.

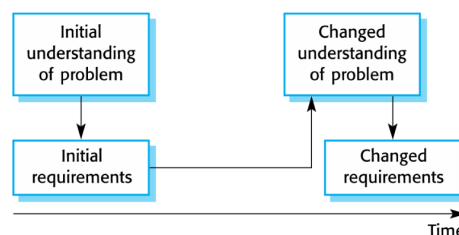


Figura 4.7: Evoluzione dei requisiti

La gestione dei requisiti è il processo di gestione dei requisiti in evoluzione durante il processo di ingegneria dei requisiti e lo sviluppo del sistema. Nuovi requisiti emergono man mano che un sistema viene sviluppato e dopo che è stato messo in uso. È necessario monitorare i requisiti individuali e mantenere i collegamenti tra i requisiti dipendenti in modo da poter valutare l'impatto delle modifiche ai requisiti. È importante stabilire un processo formale per presentare proposte di modifica e collegarle ai requisiti del sistema.

4.6.1 Pianificazione della gestione dei requisiti

La pianificazione della gestione dei requisiti riguarda la definizione di come gestire un insieme di requisiti in evoluzione. Durante questa fase, è necessario prendere decisioni su vari aspetti:

1. *Identificazione dei requisiti*: ogni requisito deve essere identificato in modo univoco, così da poter essere collegato ad altri requisiti e utilizzato nelle valutazioni di tracciabilità.
2. *Processo di gestione delle modifiche*: si tratta di una serie di attività per valutare l'impatto e il costo delle modifiche.
3. *Politiche di tracciabilità*: queste politiche definiscono le relazioni da registrare tra ciascun requisito e tra i requisiti e il design del sistema.
4. *Supporto tramite strumenti*: gli strumenti utilizzati possono variare da sistemi specializzati per la gestione dei requisiti a fogli di calcolo condivisi e semplici database.

4.6.2 Gestione delle modifiche ai requisiti

La gestione delle modifiche ai requisiti (Figura 4.8) deve essere applicata a tutte le modifiche proposte ai requisiti di un sistema dopo l'approvazione del documento dei requisiti.

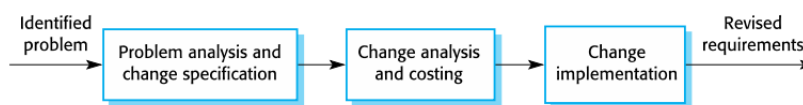


Figura 4.8: Gestione dei cambiamenti dei requisiti

Esistono tre fasi principali in un processo di gestione delle modifiche:

1. *Analisi del problema e specifica della modifica*: in questa fase, il problema o la proposta viene analizzata per verificarne la validità. Questa analisi viene comunicata al richiedente della modifica, che può rispondere con una proposta di modifica più dettagliata o decidere di ritirare la richiesta.
2. *Analisi e stima del costo della modifica*: l'effetto della modifica proposta viene valutato utilizzando le informazioni di tracciabilità e la conoscenza generale dei requisiti del sistema. Completata questa analisi, si decide se procedere o meno con la modifica dei requisiti.
3. *Implementazione della modifica*: il documento dei requisiti e, se necessario, la progettazione e l'implementazione del sistema, vengono modificati. È opportuno organizzare il documento dei requisiti in modo che sia possibile apportare modifiche senza riscrivere o riorganizzare ampiamente il documento.

4.7 Punti chiave

- I requisiti per un sistema software stabiliscono cosa il sistema dovrebbe fare e definiscono i vincoli sulla sua operatività e implementazione.
- I requisiti funzionali sono dichiarazioni dei servizi che il sistema deve fornire o descrizioni di come devono essere eseguiti alcuni calcoli.
- I requisiti non funzionali spesso vincolano il sistema in fase di sviluppo e il processo di sviluppo stesso.
Essi riguardano frequentemente le proprietà emergenti del sistema e quindi si applicano al sistema nel suo insieme.
- Il processo di ingegneria dei requisiti è un processo iterativo che include l'elicitazione, la specifica e la validazione dei requisiti.
- L'elicitazione dei requisiti è un processo iterativo che può essere rappresentato come una spirale di attività: scoperta dei requisiti, classificazione e organizzazione dei requisiti, negoziazione e documentazione dei requisiti.
- È possibile utilizzare una varietà di tecniche per l'elicitazione dei requisiti, tra cui interviste ed etnografia. Storie utente e scenari possono essere utilizzati per facilitare le discussioni.
- La specifica dei requisiti è il processo di documentazione formale dei requisiti utente e di sistema e di creazione di un documento dei requisiti software.
- Il documento dei requisiti software è una dichiarazione concordata dei requisiti del sistema. Dovrebbe essere organizzato in modo che sia i clienti del sistema che i sviluppatori software possano utilizzarlo.
- La validazione dei requisiti è il processo di verifica dei requisiti per validità, coerenza, completezza, realismo e verificabilità.
- I cambiamenti aziendali, organizzativi e tecnici portano inevitabilmente a modifiche nei requisiti per un sistema software. La gestione dei requisiti è il processo di gestione e controllo di tali cambiamenti.

Capitolo	Descrizione
Prefazione	Dovrebbe definire il pubblico previsto del documento e descriverne la storia delle versioni, inclusa una motivazione per la creazione di una nuova versione e un riassunto delle modifiche apportate in ciascuna versione.
Introduzione	Dovrebbe descrivere la necessità del sistema, riassumerne le funzioni e spiegare come funzionerà con altri sistemi. Dovrebbe inoltre descrivere come il sistema si inserisce negli obiettivi aziendali o strategici dell'organizzazione committente.
Glossario	Dovrebbe definire i termini tecnici usati nel documento, senza presupporre esperienze o competenze del lettore.
Definizione dei requisiti utente	Qui si descrivono i servizi forniti per l'utente. Anche i requisiti di sistema non funzionali devono essere descritti in questa sezione. La descrizione può utilizzare il linguaggio naturale, diagrammi o altre notazioni comprensibili ai clienti. Inoltre, devono essere specificati gli standard di prodotto e di processo da seguire.
Architettura di sistema	Questo capitolo dovrebbe presentare una panoramica di alto livello dell'architettura prevista del sistema, mostrando la distribuzione delle funzioni nei moduli del sistema. I componenti architetturali riutilizzati dovrebbero essere evidenziati.
Specifiche dei requisiti di sistema	Dovrebbe descrivere in dettaglio i requisiti funzionali e non funzionali. Se necessario, ulteriori dettagli possono essere aggiunti ai requisiti non funzionali. Possono essere definite le interfacce con altri sistemi.
Modelli di sistema	Potrebbe includere modelli grafici del sistema che mostrano le relazioni tra i componenti del sistema e tra il sistema e il suo ambiente. Esempi di modelli possibili sono modelli a oggetti, modelli di flusso di dati o modelli di dati semantici.
Evoluzione del sistema	Dovrebbe descrivere i presupposti fondamentali su cui si basa il sistema e i cambiamenti previsti dovuti all'evoluzione dell'hardware, ai cambiamenti nelle esigenze degli utenti, ecc. Questa sezione è utile ai progettisti del sistema, poiché può aiutarli a evitare decisioni di progettazione che limiterebbero futuri cambiamenti.
Appendici	Dovrebbero fornire informazioni dettagliate e specifiche relative all'applicazione in fase di sviluppo; ad esempio, descrizioni dell'hardware e del database. I requisiti hardware definiscono le configurazioni minime e ottimali per il sistema. I requisiti di database definiscono l'organizzazione logica dei dati utilizzati dal sistema e le relazioni tra i dati.
Indice	Possono essere inclusi vari indici del documento. Oltre a un indice alfabetico normale, possono esserci un indice dei diagrammi, un indice delle funzioni, ecc.

Tabella 4.3: La struttura di un documento di requisiti

Capitolo 5

Modellazione del sistema

La **modellazione del sistema** è il processo di sviluppo di modelli astratti di un sistema, con ciascun modello che presenta una diversa visione o prospettiva di quel sistema. Attualmente, la modellazione del sistema significa solitamente rappresentare un sistema usando una notazione grafica basata sui tipi di diagrammi nell'UML.

I modelli sono utilizzati durante il processo di ingegneria dei requisiti per aiutare a derivare i requisiti dettagliati di un sistema, durante il processo di progettazione per descrivere il sistema agli ingegneri che lo implementano e, dopo l'implementazione, per documentare la struttura e il funzionamento del sistema. È possibile sviluppare modelli sia del sistema esistente che di quello da sviluppare:

1. I modelli del sistema esistente sono utilizzati durante l'ingegneria dei requisiti per chiarire cosa fa il sistema attuale e per facilitare la discussione tra gli stakeholder sui suoi punti di forza e debolezza.
2. I modelli del nuovo sistema sono utilizzati durante l'ingegneria dei requisiti per spiegare i requisiti proposti ad altri stakeholder. Gli ingegneri utilizzano questi modelli per discutere le proposte di progettazione e per documentare il sistema per l'implementazione. Se si utilizza un processo di ingegneria guidata dai modelli, è possibile generare una implementazione completa o parziale del sistema dai modelli di sistema.

È possibile sviluppare modelli diversi per rappresentare il sistema da diverse prospettive, ad esempio:

1. Una prospettiva esterna, per modellare il contesto o l'ambiente del sistema.
2. Una prospettiva di interazione, per modellare le interazioni tra un sistema e il suo ambiente o tra i componenti di un sistema.
3. Una prospettiva strutturale, per modellare l'organizzazione di un sistema o la struttura dei dati elaborati dal sistema.
4. Una prospettiva comportamentale, per modellare il comportamento dinamico del sistema e come risponde agli eventi.

Il Linguaggio di Modellazione Unificato (UML) è un insieme di 13 tipi di diagrammi che possono essere utilizzati per modellare i sistemi software. Ci sono tre modi comuni in cui i modelli grafici sono usati:

1. Per stimolare e focalizzare la discussione su un sistema esistente o proposto. I modelli incompleti e non corretti vanno bene perché il loro ruolo è quello di supportare la discussione.
2. Per documentare un sistema esistente. I modelli dovrebbero essere una rappresentazione accurata del sistema, ma non devono essere completi.
3. Come una descrizione dettagliata del sistema da utilizzare per generare l'implementazione del sistema. I modelli devono essere sia corretti che completi.

La maggior parte degli utenti ritiene che cinque tipi di diagrammi rappresentino gli elementi essenziali di un sistema:

1. Diagrammi delle attività, che mostrano le attività coinvolte in un processo o nell'elaborazione dei dati.
2. Diagrammi dei casi d'uso, che mostrano le interazioni tra un sistema e il suo ambiente.
3. Diagrammi di sequenza, che mostrano le interazioni tra attori e il sistema, e tra i componenti del sistema.
4. Diagrammi delle classi, che mostrano le classi di oggetti nel sistema e le associazioni tra queste classi.
5. Diagrammi di stato, che mostrano come il sistema reagisce agli eventi interni ed esterni.

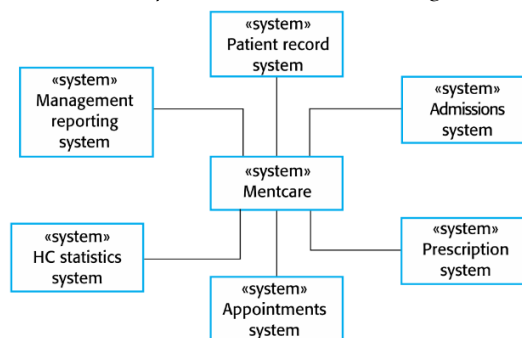
5.1 Modelli di contesto

Nelle prime fasi della specifica di un sistema, è fondamentale definire i confini del sistema, ossia determinare ciò che fa parte del sistema in fase di sviluppo e cosa no. Questo richiede di collaborare con gli stakeholder per decidere quale funzionalità includere nel sistema e quali operazioni devono essere gestite nell'ambiente operativo del sistema. Si potrebbe stabilire, ad esempio, che alcuni processi aziendali devono essere supportati automaticamente dal software in sviluppo, mentre altri possono essere gestiti manualmente o tramite altri sistemi.

La definizione dei confini di un sistema non è una decisione neutrale; infatti, possono influire fattori sociali e organizzativi.

Esempio 5.1.1 – Contesto del sistema Mentcare

La figura mostra un modello di contesto per il sistema Mentcare e gli altri sistemi del suo ambiente.



Il sistema Mentcare è collegato a un sistema di appuntamenti e a un sistema generale di archiviazione dei dati dei pazienti con cui condivide le informazioni. È inoltre connesso a sistemi per la reportistica gestionale, per le ammissioni ospedaliere e a un sistema statistico che raccoglie informazioni per la ricerca. Infine, utilizza un sistema di prescrizione per generare le ricette per i farmaci.

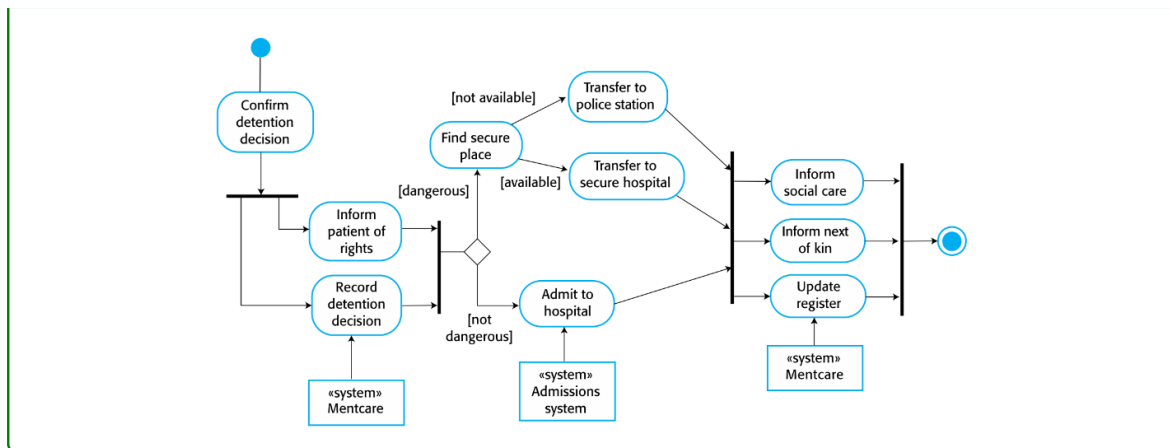
I modelli di contesto mostrano semplicemente gli altri sistemi nell'ambiente, non come il sistema in fase di sviluppo viene utilizzato in quell'ambiente.

I modelli di contesto normalmente indicano che l'ambiente include diversi altri sistemi automatizzati, ma non mostrano i tipi di relazioni tra il sistema e quelli esterni. Pertanto, i modelli di contesto semplici vengono utilizzati insieme ad altri modelli, come i modelli di processo aziendale, che descrivono i processi umani e automatizzati in cui vengono utilizzati particolari sistemi software.

I diagrammi delle attività UML possono essere utilizzati per mostrare i processi aziendali in cui vengono utilizzati i sistemi.

Esempio 5.1.2 – Modello di processo di detenzione involontaria

La figura è un diagramma delle attività UML che illustra il processo di detenzione involontaria nell'assistenza sanitaria mentale, in cui viene utilizzato il sistema Mentcare.



I diagrammi delle attività UML mostrano le attività di un processo e il flusso di controllo da un'attività all'altra. L'inizio del processo è indicato da un cerchio pieno, la fine da un cerchio pieno all'interno di un altro cerchio. I rettangoli con angoli arrotondati rappresentano le attività, ovvero i sottoprocessi specifici che devono essere eseguiti.

Le frecce rappresentano il flusso di lavoro tra le attività, e una barra piena indica il coordinamento delle attività. Se il flusso di più attività porta a una barra piena, tutte queste attività devono essere completate prima che si possa proseguire. Quando il flusso da una barra piena conduce a diverse attività, queste possono essere eseguite in parallelo.

Le frecce possono essere annotate con guardie (tra parentesi quadre) che specificano quando quel flusso deve essere seguito.

5.2 Modelli di interazione

La modellazione dell'interazione utente è importante per identificare i requisiti dell'utente. La modellazione dell'interazione tra sistemi evidenzia i problemi di comunicazione che potrebbero sorgere. La modellazione dell'interazione tra componenti ci aiuta a capire se una struttura proposta per il sistema è in grado di garantire le prestazioni e l'affidabilità richieste. I diagrammi dei casi d'uso e i diagrammi di sequenza possono essere utilizzati per la modellazione dell'interazione.

5.2.1 Modellazione degli use case

Ogni caso d'uso rappresenta un'attività specifica che comporta l'interazione con il sistema. Nella sua forma più semplice, un caso d'uso è rappresentato come un'ellisse, con gli attori coinvolti rappresentati come figure stilizzate.

Esempio 5.2.1 – Caso d'uso di trasferimento dati

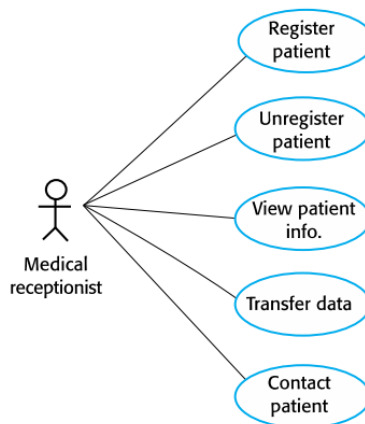
La figura mostra un caso d'uso per il sistema Mentcare, che rappresenta il compito di caricare i dati dal sistema Mentcare a un sistema generale di archiviazione dei dati dei pazienti, il quale mantiene i dati riassuntivi sui pazienti.



I diagrammi dei casi d'uso forniscono una panoramica semplice dell'interazione, ma è necessario aggiungere maggiori dettagli per una descrizione completa.

Caso d'Uso	MHC-PMS: Trasferimento dati
Attori	Receptionist medico, sistema di registrazione pazienti (PRS)
Descrizione	Un receptionist può trasferire dati dal sistema Mentcare a un database generale di registrazione pazienti gestito da un'autorità sanitaria. Le informazioni trasferite possono riguardare dati personali aggiornati (indirizzo, numero di telefono, ecc.) o un riepilogo della diagnosi e del trattamento del paziente.
Dati	Informazioni personali del paziente, riepilogo del trattamento
Stimolo	Comando dell'utente emesso dal receptionist medico
Risposta	Conferma che il PRS è stato aggiornato
Commenti	Il receptionist deve avere le autorizzazioni di sicurezza appropriate per accedere alle informazioni del paziente e al PRS.

I diagrammi di casi d'uso composti mostrano più casi d'uso. A volte è possibile includere tutte le interazioni in un unico diagramma composto, ma, se il numero di casi d'uso è elevato, si possono sviluppare più diagrammi con casi d'uso correlati.



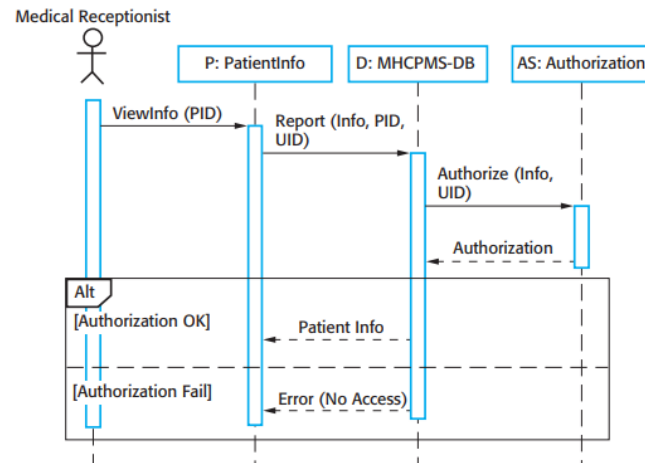
5.2.2 Diagrammi di sequenza

I diagrammi di sequenza in UML vengono utilizzati principalmente per modellare le interazioni tra attori e oggetti in un sistema e le interazioni tra gli oggetti stessi. Come suggerisce il nome, un diagramma di sequenza mostra la sequenza delle interazioni che avvengono durante un particolare caso d'uso o istanza di caso d'uso.

Gli oggetti e gli attori coinvolti sono elencati in alto, con una linea tratteggiata verticale che scende da ciascuno di essi. Le frecce annotate indicano le interazioni tra oggetti. Il rettangolo sulle linee tratteggiate rappresenta la "lifeline" dell'oggetto, ossia il periodo in cui l'oggetto è coinvolto nell'elaborazione. La sequenza delle interazioni si legge dall'alto verso il basso. Le annotazioni sulle frecce indicano le chiamate agli oggetti, i parametri e i valori di ritorno. Per denotare delle alternative, si può rappresentare da una casella denominata "alt" con le condizioni in parentesi quadre.

Esempio 5.2.2 – Diagramma di sequenza per visualizzare le informazioni del paziente

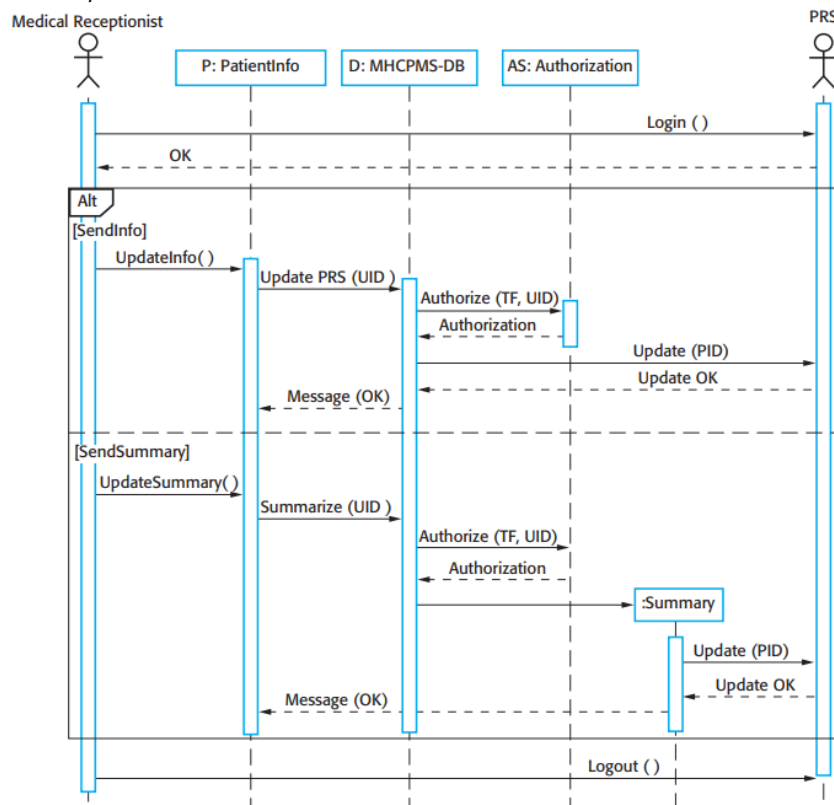
Questo diagramma modella le interazioni coinvolte nel caso d'uso "Visualizza informazioni sul paziente", in cui un receptionist medico può visualizzare alcune informazioni del paziente.



1. Il Medical Receptionist attiva il metodo `ViewInfo` in un'istanza `P` della classe di oggetti `PatientInfo`, fornendo l'identificatore `PID` del paziente.
2. L'istanza `P` interroga il database per ottenere le informazioni richieste.
3. Il database verifica l'autorizzazione del receptionist tramite un sistema di autorizzazione.
4. Se autorizzato, le informazioni del paziente vengono restituite e visualizzate sullo schermo; se l'autorizzazione fallisce, viene restituito un messaggio di errore.

Esempio 5.2.3 – Diagramma di sequenza per il trasferimento dei dati

Viene creato un oggetto di tipo *Summary* per contenere i dati di riepilogo da caricare su un PRS (sistema di registrazione dei pazienti) nazionale.



1. Il Receptionist accede al PRS.
2. Sono disponibili due opzioni (come mostrato nel riquadro "alt"): il trasferimento diretto delle informazioni aggiornate del paziente dal database Mentcare al PRS e il trasferimento dei dati riepilogativi della salute dal database Mentcare al PRS.
3. In ciascun caso, le autorizzazioni del receptionist vengono verificate utilizzando il sistema di autorizzazione.
4. Le informazioni personali possono essere trasferite direttamente dall'oggetto dell'interfaccia utente al PRS. In alternativa, può essere creato un record riepilogativo dal database e quindi trasferito.
5. Al termine del trasferimento, il PRS emette un messaggio di stato e l'utente effettua il logout.

5.3 Modelli strutturali

I modelli strutturali del software mostrano l'organizzazione di un sistema in termini di componenti che lo costituiscono e le loro relazioni. I modelli strutturali possono essere modelli statici, che mostrano l'organizzazione del progetto del sistema, o modelli dinamici, che mostrano l'organizzazione del sistema durante l'esecuzione.

Si creano modelli strutturali di un sistema durante la discussione e progettazione dell'architettura del sistema stesso.

5.3.1 Diagrammi delle classi

I diagrammi delle classi vengono utilizzati nello sviluppo di un modello orientato agli oggetti per mostrare le classi di un sistema e le associazioni tra queste classi. Una classe di oggetti può essere pensata come una definizione generale di un tipo di oggetto del sistema. Un'associazione è un collegamento tra classi che indica l'esistenza di una relazione tra di esse.

Durante la modellazione nelle prime fasi del processo di ingegneria del software, gli oggetti rappresentano qualcosa nel mondo reale, come un paziente, una prescrizione o un medico.

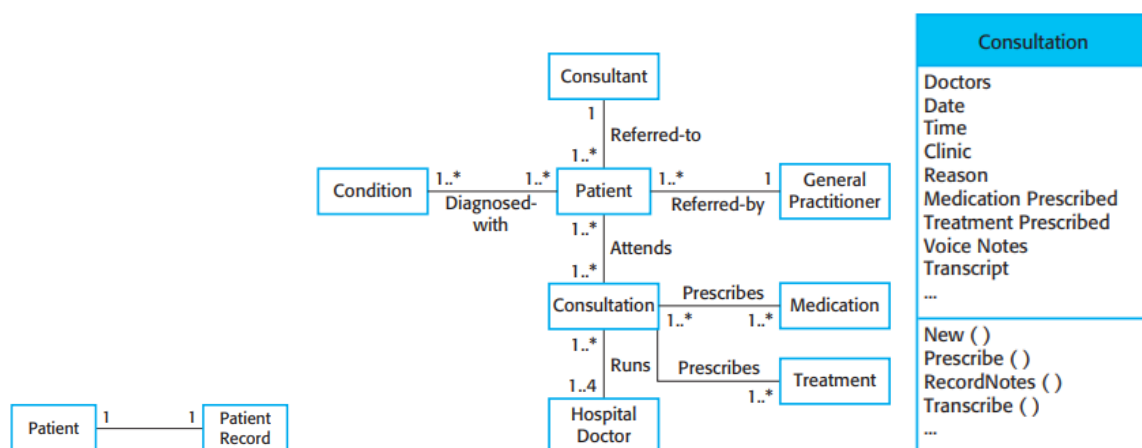


Figura 5.1: Classi e associazioni UML

Una caratteristica importante dei diagrammi delle classi è la capacità di mostrare quanti oggetti sono coinvolti in un'associazione. Ciascuna estremità dell'associazione è annotata con un "1", che significa che esiste una relazione 1:1 tra gli oggetti di queste classi. Cioè, ad esempio, ogni paziente ha esattamente un record e ogni record mantiene informazioni su un solo paziente. Sono possibili anche altre molteplicità, ad esempio "*", indica che è presente un numero indefinito di oggetti nell'associazione. Ad esempio, la molteplicità (1..) nella relazione tra Patient e Condition mostra

che un paziente può soffrire di più condizioni e che la stessa condizione può essere associata a più pazienti.

Per definire gli oggetti in modo più dettagliato, si aggiungono informazioni sui loro attributi (le caratteristiche dell'oggetto) e sulle operazioni (le funzioni dell'oggetto).

5.3.2 Generalizzazione

La generalizzazione è una tecnica quotidiana che utilizziamo per gestire la complessità. Anziché imparare le caratteristiche dettagliate di tutto ciò che incontriamo, apprendiamo classi generali (animali, automobili, case, ecc.) e le caratteristiche di queste classi. In seguito, riutilizziamo la conoscenza classificando gli oggetti e ci concentriamo sulle differenze tra essi e la loro classe. Ad esempio, gli scoiattoli e i ratti appartengono alla classe dei "roditori" e condividono quindi le caratteristiche dei roditori. Le affermazioni generali si applicano a tutti i membri della classe; ad esempio, tutti i roditori hanno denti per rosicchiare.

Quando si modellano sistemi, è spesso utile esaminare le classi in un sistema per verificare se esiste spazio per una generalizzazione e la creazione di classi. Questa è una buona pratica di progettazione, poiché significa che, se vengono proposte modifiche, non è necessario esaminare tutte le classi del sistema per vedere se sono influenzate dalla modifica. Nei linguaggi orientati agli oggetti, come Java, la generalizzazione viene implementata utilizzando i meccanismi di ereditarietà delle classi integrati nel linguaggio.

In una generalizzazione, gli attributi e le operazioni associati alle classi di livello superiore sono anche associati alle classi di livello inferiore. Le classi di livello inferiore sono sottoclassi che ereditano gli attributi e le operazioni dalle loro superclassi, alle quali aggiungono attributi e operazioni più specifici.

In UML la generalizzazione è mostrata con una freccia che punta verso l'alto alla classe più generale.

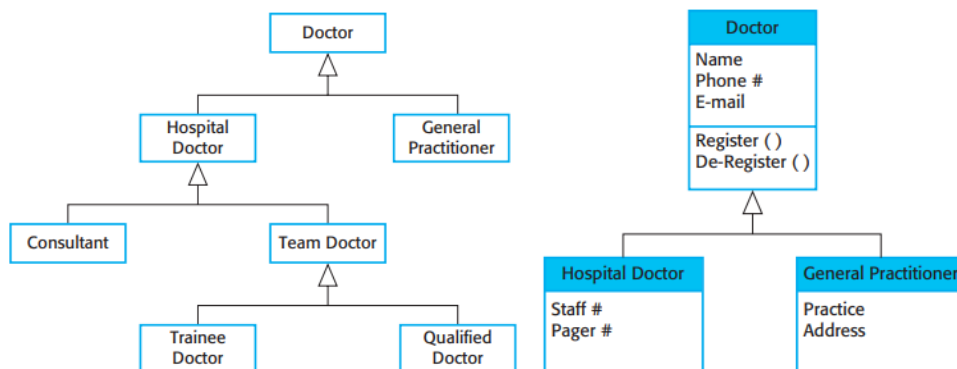


Figura 5.2: Gerarchia di generalizzazione

5.3.3 Aggregazione

L'UML fornisce un tipo speciale di associazione tra classi chiamato aggregazione, che indica che un oggetto (il tutto) è composto da altri oggetti (le parti). Per definire l'aggregazione, si aggiunge un rombo accanto al collegamento che rappresenta la classe che costituisce il tutto.

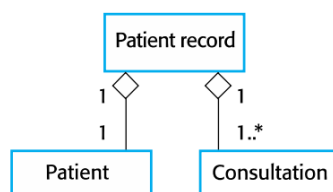


Figura 5.3: Associazione di aggregazione

5.4 Modelli comportamentali

I modelli comportamentali sono rappresentazioni del comportamento dinamico di un sistema durante la sua esecuzione. Mostrano cosa accade o dovrebbe accadere quando un sistema risponde a uno stimolo proveniente dall'ambiente esterno. Questi stimoli possono essere dati o eventi:

1. I dati diventano disponibili e devono essere elaborati dal sistema.
2. Si verifica un evento che attiva l'elaborazione del sistema. Gli eventi possono avere dati associati, anche se non sempre è così.

5.4.1 Modelli guidati dai dati

Molti sistemi aziendali sono sistemi di elaborazione dati, guidati principalmente dai dati stessi. Sono controllati dall'input di dati fornito al sistema e presentano relativamente pochi eventi esterni.

I modelli guidati dai dati mostrano la sequenza di azioni coinvolte nell'elaborazione dei dati di input e nella generazione di un output associato. Possono essere utilizzati durante l'analisi dei requisiti, poiché mostrano l'elaborazione end-to-end in un sistema. Ovvero, rappresentano l'intera sequenza di azioni che avviene dall'inizio dell'elaborazione di un input fino all'output corrispondente, che rappresenta la risposta del sistema.

I diagrammi di flusso dei dati possono essere rappresentati in UML utilizzando i diagrammi di attività. I passaggi di elaborazione, rappresentati come attività (rettangoli con angoli arrotondati), e i dati che fluiscono tra questi passaggi, rappresentati come oggetti (rettangoli).

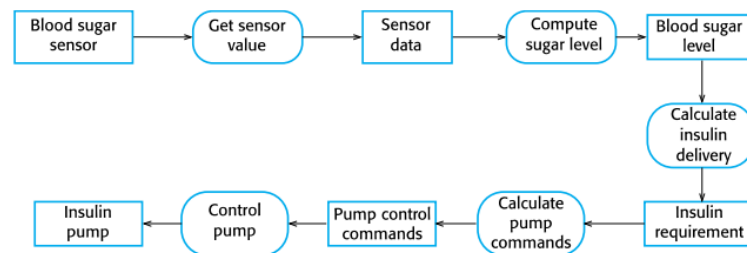


Figura 5.4: Modello di attività del funzionamento della pompa per insulina

Un'alternativa per mostrare la sequenza di elaborazione in un sistema è utilizzare i diagrammi di sequenza UML. Abbiamo già visto come questi diagrammi possano essere usati per modellare le interazioni, ma se li si disegna in modo che i messaggi siano inviati solo da sinistra a destra, mostrano la sequenza di elaborazione dei dati nel sistema.

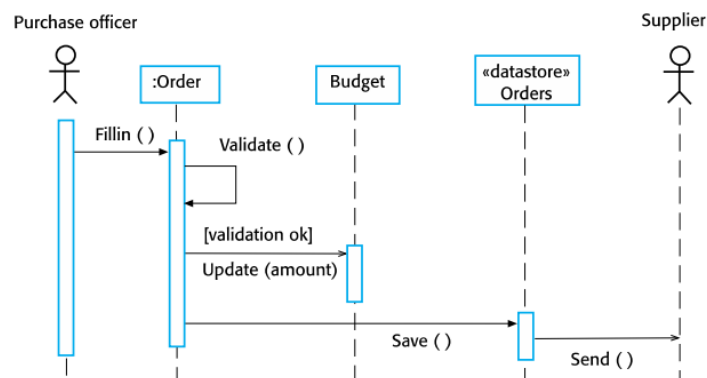


Figura 5.5: Elaborazione degli ordini

5.4.2 Modelli guidati dagli eventi

I sistemi in tempo reale sono solitamente guidati dagli eventi, con una limitata elaborazione dei dati. Ad esempio, un sistema di commutazione telefonica risponde ad eventi come "ricevitore sollevato" generando un segnale di linea libera.

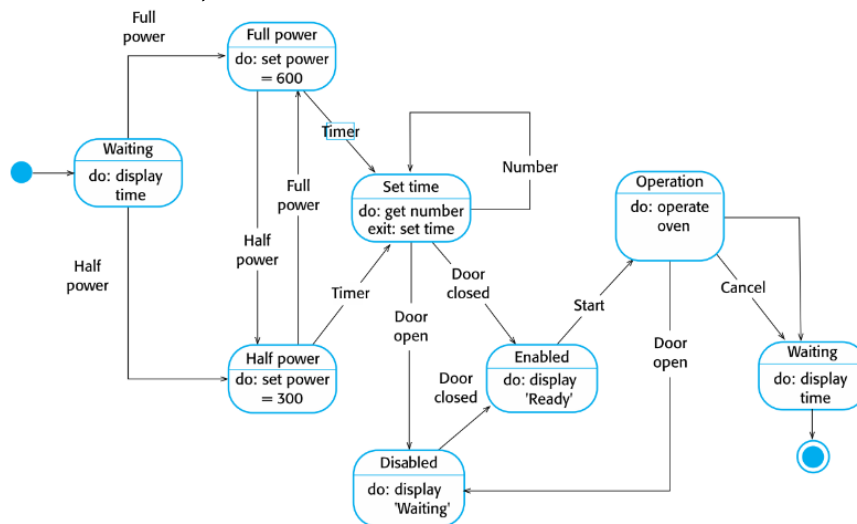
La modellazione guidata dagli eventi mostra come un sistema risponde a eventi esterni e interni. Si basa sull'assunzione che un sistema abbia un numero finito di stati e che gli eventi (stimoli) possano causare una transizione da uno stato all'altro.

L'UML supporta la modellazione basata sugli eventi mediante diagrammi di stato, basati sugli Statechart. I diagrammi di stato mostrano gli stati del sistema e gli eventi che causano le transizioni da uno stato all'altro.

Nei diagrammi di stato UML, i rettangoli con angoli arrotondati rappresentano gli stati del sistema. Possono includere una breve descrizione (dopo la parola "do") delle azioni eseguite in quello stato. Le frecce etichettate rappresentano gli stimoli che forzano una transizione da uno stato all'altro. È possibile indicare gli stati di inizio e fine con cerchi pieni, come nei diagrammi di attività.

Esempio 5.4.1 – Diagramma di stato di un forno a microonde

Per illustrare la modellazione guidata dagli eventi, utilizzo un esempio di software di controllo per un forno a microonde molto semplice.



Questo forno semplice ha un interruttore per selezionare potenza massima o ridotta, una tastiera numerica per impostare il tempo di cottura, un pulsante di avvio/arresto e un display alfanumerico.

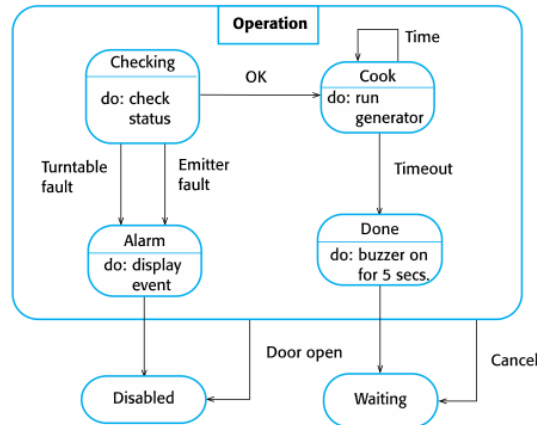
Si presume che la sequenza di azioni per l'uso del microonde sia la seguente:

1. Selezionare il livello di potenza (metà potenza o potenza massima).
2. Impostare il tempo di cottura utilizzando una tastiera numerica.
3. Premere Avvio e il cibo viene cotto per il tempo impostato.

Per motivi di sicurezza, il forno non dovrebbe funzionare quando la porta è aperta e, al termine della cottura, viene emesso un segnale acustico. Il forno ha un semplice display utilizzato per visualizzare vari avvisi e messaggi di allarme.

Si può vedere che il sistema parte in uno stato di attesa e risponde inizialmente a uno dei pulsanti di selezione della potenza. Gli utenti possono cambiare idea dopo aver selezionato uno di questi pulsanti e possono premere l'altro. Viene impostato il tempo e, se la porta è chiusa, il pulsante di avvio è abilitato. Premendo questo pulsante inizia l'operazione del forno, che cucina per il tempo specificato. Questo è il termine del ciclo di cottura e il sistema ritorna allo stato di attesa.

Il problema con la modellazione basata sugli stati è che il numero di stati possibili aumenta rapidamente. Per i modelli di grandi dimensioni, quindi, è necessario nascondere i dettagli nei modelli. Un modo per farlo è utilizzare il concetto di superstato che incapsula una serie di stati separati.



Lo stato di Operazione include una serie di sottostati. Mostra che l'operazione inizia con un controllo di stato e che, se vengono rilevati problemi, viene attivato un allarme e l'operazione viene disabilitata. La cottura implica l'attivazione del generatore a microonde per il tempo specificato; al termine, viene emesso un segnale acustico. Se la porta viene aperta durante l'operazione, il sistema passa allo stato di disabilitazione.

I modelli a stati di un sistema forniscono una panoramica dell'elaborazione degli eventi, ma normalmente è necessario estenderli con una descrizione più dettagliata degli stimoli e degli stati del sistema.

Stato	Descrizione
Attesa	Il forno è in attesa di input. Il display mostra l'ora corrente.
Mezza potenza	La potenza del forno è impostata a 300 watt. Il display mostra 'Mezza potenza'.
Potenza massima	La potenza del forno è impostata a 600 watt. Il display mostra 'Potenza massima'.
Imposta tempo	Il tempo di cottura è impostato al valore inserito dall'utente. Il display mostra il tempo di cottura selezionato e viene aggiornato mentre il tempo viene impostato.
Disabilitato	Il funzionamento del forno è disabilitato per sicurezza. La luce interna del forno è accesa. Il display mostra 'Non pronto'.
Abilitato	Il funzionamento del forno è abilitato. La luce interna del forno è spenta. Il display mostra 'Pronto per la cottura'.
Operazione	Il forno è in funzione. La luce interna del forno è accesa. Il display mostra il conto alla rovescia del timer. Al termine della cottura, il segnale acustico suona per cinque secondi. La luce del forno è accesa. Il display mostra 'Cottura completata' mentre il segnale acustico è attivo.

Stimolo	Descrizione
Mezza potenza	L'utente ha premuto il pulsante per la mezza potenza.
Potenza massima	L'utente ha premuto il pulsante per la potenza massima.
Timer	L'utente ha premuto uno dei pulsanti del timer.
Numero	L'utente ha premuto un tasto numerico.
Porta aperta	L'interruttore della porta del forno non è chiuso.
Porta chiusa	L'interruttore della porta del forno è chiuso.
Avvio	L'utente ha premuto il pulsante Avvio.
Annulla	L'utente ha premuto il pulsante Annulla.

5.5 Ingegneria guidata dai modelli

L'ingegneria guidata dai modelli (MDE, Model-Driven Engineering) è un approccio allo sviluppo software in cui i modelli, piuttosto che i programmi, sono i principali risultati del processo di sviluppo. I programmi che vengono eseguiti su una piattaforma hardware/software vengono generati

automaticamente a partire dai modelli. I sostenitori della MDE sostengono che questo innalza il livello di astrazione nell'ingegneria del software, così che gli ingegneri non devono più preoccuparsi dei dettagli del linguaggio di programmazione o delle specificità delle piattaforme di esecuzione.

L'ingegneria guidata da modelli è ancora in una fase iniziale di sviluppo, ed è incerto se avrà o meno un effetto significativo sulla pratica dell'ingegneria del software. Da un lato permette di considerare i sistemi a livelli di astrazione più alti e di generare codice automaticamente in modo da rendere meno costoso adattare i sistemi a nuove piattaforme; dall'altro i modelli per l'astrazione non sono necessariamente adatti all'implementazione e i risparmi derivanti dalla generazione automatica di codice possono essere superati dai costi di sviluppo dei traduttori per nuove piattaforme.

5.5.1 Architettura guidata da modelli

L'architettura guidata da modelli (Model-Driven Architecture o MDA) è un approccio alla progettazione e implementazione del software che si focalizza sui modelli e utilizza un sottoinsieme di modelli UML per descrivere un sistema. In questo approccio vengono creati modelli a diversi livelli di astrazione. Da un modello indipendente dalla piattaforma di alto livello è possibile, in teoria, generare un programma funzionante senza intervento manuale.

Il metodo MDA consiglia la produzione di tre tipi di modelli astratti del sistema:

1. *Modello indipendente dal calcolo (Computation Independent Model o CIM)*. I CIM rappresentano le astrazioni dominanti importanti utilizzate in un sistema e sono a volte chiamati modelli di dominio.
2. *Modello indipendente dalla piattaforma (Platform-Independent Model o PIM)*. I PIM rappresentano il funzionamento del sistema senza riferimenti alla sua implementazione. Di solito, un PIM è descritto usando modelli UML che mostrano la struttura statica del sistema e il modo in cui risponde a eventi esterni e interni.
3. *Modelli specifici della piattaforma (Platform-Specific Models o PSM)*. I PSM sono trasformazioni del modello indipendente dalla piattaforma con un PSM separato per ogni piattaforma applicativa. In teoria, potrebbero esserci livelli di PSM, con ogni livello che aggiunge qualche dettaglio specifico della piattaforma.

Il concetto fondamentale di MDA è che le trasformazioni tra modelli possono essere definite e applicate automaticamente tramite strumenti software, come illustrato nella Figura 5.6. Questo diagramma mostra anche un livello finale di trasformazione automatica in cui una trasformazione è applicata al PSM per generare il codice eseguibile che verrà eseguito sulla piattaforma software designata.

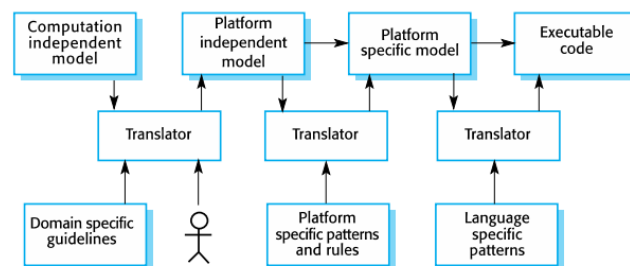


Figura 5.6: Trasformazioni MDA

La traduzione da modelli indipendenti dalla piattaforma a modelli specifici della piattaforma è un problema tecnico più semplice. Esistono strumenti commerciali e open-source che forniscono traduttori dai PIM a piattaforme comuni come Java e J2EE. Potrebbero esserci diversi PSM per ogni PIM nel sistema. Se un sistema software è destinato a funzionare su piattaforme diverse (ad es., J2EE e .NET), allora, in teoria, è sufficiente mantenere un unico PIM. I PSM per ciascuna piattaforma sono generati automaticamente (Figura 5.7).

Ritengo che ci siano diversi altri motivi per cui MDA non è diventato un approccio di riferimento nello sviluppo software:

1. I modelli sono un buon modo per facilitare le discussioni su una progettazione software. Tuttavia, ciò non implica sempre che le astrazioni utili per le discussioni siano le astrazioni giuste per l'implementazione.

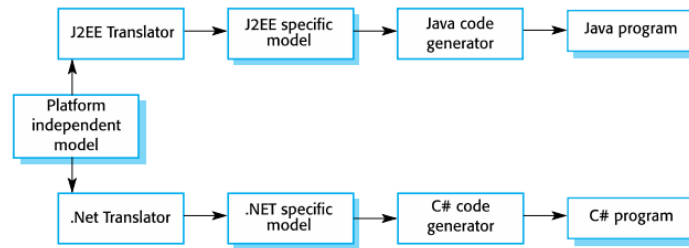


Figura 5.7: Modelli multipli specifici per piattaforma

2. Per la maggior parte dei sistemi complessi, l'implementazione non è il problema principale: l'ingegneria dei requisiti, la sicurezza e l'affidabilità, l'integrazione con i sistemi legacy e i test sono tutti aspetti più significativi.
3. Gli argomenti per l'indipendenza dalla piattaforma sono validi solo per sistemi di grandi dimensioni e di lunga durata. Per i prodotti software e i sistemi informativi sviluppati per piattaforme standard i risparmi derivanti dall'uso di MDA potrebbero essere superati dai costi di introduzione e di configurazione.
4. La diffusione dei metodi agili nello stesso periodo in cui MDA si stava sviluppando ha distolto l'attenzione dagli approcci guidati da modelli.

I casi di successo per MDA provengono principalmente da aziende che sviluppano prodotti di sistema, che includono sia hardware che software. Il software in questi prodotti ha una lunga durata e potrebbe dover essere modificato per riflettere i cambiamenti delle tecnologie hardware.

Esiste una relazione difficile tra metodi agili e architettura guidata da modelli. L'idea di una modellazione preliminare estesa contraddice i principi fondamentali del manifesto agile e sospetto che pochi sviluppatori agili si sentano a proprio agio con l'ingegneria guidata da modelli.

5.6 Punti chiave

- Un modello è una rappresentazione astratta di un sistema che ignora i dettagli specifici. È possibile sviluppare modelli complementari del sistema per mostrare il contesto, le interazioni, la struttura e il comportamento del sistema.
- I modelli di contesto mostrano come un sistema in fase di modellazione sia posizionato in un ambiente con altri sistemi e processi.
- I diagrammi dei casi d'uso e i diagrammi di sequenza vengono utilizzati per descrivere le interazioni tra utenti e sistemi in fase di progettazione. I casi d'uso descrivono le interazioni tra un sistema e gli attori esterni; i diagrammi di sequenza aggiungono ulteriori informazioni mostrando le interazioni tra gli oggetti del sistema.
- I modelli strutturali mostrano l'organizzazione e l'architettura di un sistema. I diagrammi delle classi sono utilizzati per definire la struttura statica delle classi in un sistema e le loro associazioni.
- I modelli comportamentali sono usati per descrivere il comportamento dinamico di un sistema in esecuzione. Questo comportamento può essere modellato dalla prospettiva dei dati elaborati dal sistema o dagli eventi che stimolano le risposte del sistema.
- I diagrammi di attività possono essere usati per modellare l'elaborazione dei dati, dove ogni attività rappresenta un passaggio del processo.
- I diagrammi di stato sono utilizzati per modellare il comportamento di un sistema in risposta a eventi interni o esterni.
- L'ingegneria guidata da modelli è un approccio allo sviluppo software in cui un sistema è rappresentato come un insieme di modelli che possono essere automaticamente trasformati in codice eseguibile.

Capitolo 6

Architectural design

La **progettazione architetturale** si occupa di comprendere come dovrebbe essere organizzato un sistema software e di progettare la struttura complessiva del sistema. Essa rappresenta il collegamento fondamentale tra progettazione e ingegneria dei requisiti, poiché identifica i principali componenti strutturali di un sistema e le relazioni tra loro. L'output del processo di progettazione architetturale è un modello architetturale che descrive come il sistema è organizzato come un insieme di componenti che comunicano tra loro.

Nei processi agili, è generalmente accettato che una fase iniziale del processo di sviluppo agile si concentri sulla progettazione dell'architettura del sistema nel suo insieme. Rifattorizzare i componenti in risposta ai cambiamenti è solitamente piuttosto semplice; tuttavia, rifattorizzare l'architettura del sistema è costoso, poiché potrebbe essere necessario modificare la maggior parte dei componenti del sistema per adattarli ai cambiamenti architetturali.

Per aiutarti a capire cosa si intende per architettura di sistema, osserva la Figura 6.1. Questo diagramma mostra un modello astratto dell'architettura di un sistema robotico di imballaggio. Questo sistema robotico è in grado di imballare diversi tipi di oggetti, usando un componente di visione per selezionare gli oggetti su un nastro trasportatore, identificarne il tipo e scegliere il tipo di imballaggio corretto. Il sistema poi trasferisce gli oggetti dal nastro trasportatore alla zona di imballaggio e deposita gli oggetti confezionati su un altro nastro trasportatore. Il modello architetturale mostra questi componenti e i collegamenti tra di loro.

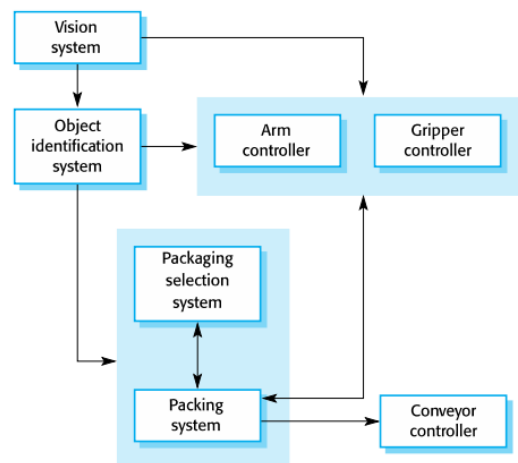


Figura 6.1: L'architettura di un sistema di controllo del robot di imballaggio

Si possono progettare le architetture software a due livelli di astrazione, denominati "architettura in piccolo" e "architettura in grande":

1. Architettura in piccolo riguarda l'architettura di singoli programmi, concentrandosi su come ciascun programma viene scomposto in componenti.

2. Architettura in grande si occupa dell'architettura di sistemi complessi, come i sistemi aziendali, che includono altri sistemi, programmi e componenti di programma. Questi sistemi possono essere distribuiti su diversi computer, gestiti da aziende differenti.

Progettare e documentare esplicitamente l'architettura software ha tre vantaggi principali:

1. *Comunicazione con gli stakeholder*: l'architettura è una rappresentazione ad alto livello del sistema che può essere usata come punto di discussione tra diversi stakeholder.
2. *Analisi del sistema*: le decisioni di progettazione architetturale hanno un profondo effetto sul fatto che il sistema possa soddisfare requisiti critici come prestazioni, affidabilità e manutenibilità.
3. *Riutilizzo su larga scala*: l'architettura del sistema è spesso la stessa per sistemi con requisiti simili e può quindi supportare il riutilizzo software su larga scala.

Le architetture di sistema sono spesso modellate informalmente utilizzando semplici diagrammi a blocchi, come nella Figura 6.1. Ogni riquadro rappresenta un componente; i riquadri interni indicano che il componente è stato scomposto in sottocomponenti. Le frecce indicano che dati o segnali di controllo vengono trasmessi da un componente all'altro nella direzione delle frecce. Tuttavia, non si considerano i diagrammi a blocchi come buone rappresentazioni architettureali poiché questi diagrammi non mostrano il tipo di relazioni tra i componenti del sistema né le proprietà visibili esternamente dei componenti.

I diagrammi a blocchi supportano efficacemente la comunicazione tra i partecipanti alla progettazione software, facilitando la comprensione da parte di esperti di dominio, ingegneri e manager. Tuttavia, per una documentazione dettagliata dell'architettura, è preferibile usare notazioni più rigorose, come i linguaggi di descrizione architetturale, che riducono la possibilità di incomprensioni.

1. Come mezzo per facilitare la discussione sulla progettazione del sistema: una visione architetturale ad alto livello è utile per comunicare con gli stakeholder e per la pianificazione del progetto, poiché non è sovraccarica di dettagli.
2. Come documentazione di un'architettura progettata: l'obiettivo qui è produrre un modello di sistema completo che mostri i diversi componenti del sistema, le loro interfacce e connessioni.

6.1 Decisioni di architectural design

La progettazione architetturale è un processo creativo in cui si organizza un sistema in modo che soddisfi i requisiti funzionali e non funzionali. Non esiste una formula predefinita per la progettazione architetturale: essa dipende dal tipo di sistema in fase di sviluppo, dal background e dall'esperienza dell'architetto e dai requisiti specifici del sistema. Pertanto, è più utile considerare la progettazione architetturale come una serie di decisioni piuttosto che come una sequenza di attività.

Durante il processo di progettazione architetturale, gli architetti devono prendere decisioni strutturali che influenzano in modo profondo sia il sistema che il suo sviluppo. Sulla base della loro esperienza e conoscenza, essi devono considerare le domande fondamentali riportate nella Figura 6.2.

Sebbene ogni sistema software sia unico, i sistemi appartenenti allo stesso dominio applicativo spesso condividono architetture simili, che riflettono i concetti fondamentali del dominio. Ad esempio, le linee di prodotto applicative sono costruite attorno a un'architettura centrale con varianti per soddisfare requisiti specifici dei clienti. Nella progettazione dell'architettura di un sistema, è quindi necessario determinare cosa il sistema e le sue categorie applicative più ampie abbiano in comune e quanto delle conoscenze architettureali disponibili sia riutilizzabile.

Poiché esiste una stretta relazione tra le caratteristiche non funzionali di un sistema e la sua architettura software, la scelta dello stile e della struttura architetturale dovrebbe dipendere dai requisiti non funzionali del sistema:

1. *Prestazioni*: l'architettura dovrebbe essere progettata per localizzare le operazioni critiche in un numero limitato di componenti. Questo potrebbe comportare l'uso di pochi componenti relativamente grandi anziché di componenti di dimensioni più ridotte.

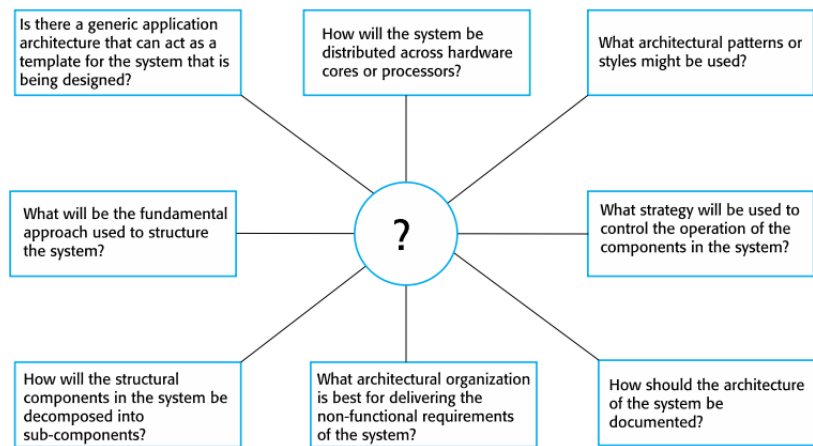


Figura 6.2: Decisioni di architectural design

2. *Sicurezza*: è preferibile utilizzare una struttura stratificata, con le risorse più critiche protette negli strati più interni e sottoposte a una rigorosa validazione di sicurezza.
3. *Sicurezza operativa*: l'architettura dovrebbe essere progettata in modo che le operazioni critiche siano concentrate in un solo componente o in pochi componenti.
4. *Disponibilità*: l'architettura dovrebbe includere componenti ridondanti per consentire la sostituzione e l'aggiornamento dei componenti senza interrompere il sistema.
5. *Manutenibilità*: l'architettura dovrebbe utilizzare componenti a grana fine e autonomi che possano essere modificati facilmente.

6.2 Architectural views

In questa sezione, affronterò due questioni rilevanti per entrambi questi scopi:

1. Quali viste o prospettive sono utili nella progettazione e nella documentazione dell'architettura di un sistema?
2. Quali notazioni dovrebbero essere utilizzate per descrivere i modelli architetturali?

È impossibile rappresentare tutte le informazioni rilevanti sull'architettura di un sistema in un unico diagramma, poiché un modello grafico può mostrare solo una vista o una prospettiva del sistema. Potrebbe, ad esempio, mostrare come il sistema è suddiviso in moduli, come i processi interagiscono a runtime, o come i componenti del sistema sono distribuiti in rete. Poiché tutte queste prospettive sono utili in diversi momenti sia per la progettazione che per la documentazione, di solito è necessario presentare più viste dell'architettura software.

Esistono opinioni diverse su quali viste siano necessarie. Krutchen, nel suo famoso modello di architettura software "4+1 view", suggerisce quattro viste architetturali fondamentali, che possono essere collegate attraverso casi d'uso o scenari comuni (Figura 6.3). Egli propone le seguenti viste:

1. Una vista logica, che mostra le astrazioni chiave del sistema come oggetti o classi di oggetti.
2. Una vista di processo, che mostra come, durante l'esecuzione, il sistema è composto da processi interagenti.
3. Una vista di sviluppo, che mostra come il software è scomposto per lo sviluppo; in altre parole, mostra la suddivisione del software in componenti implementati da singoli sviluppatori o team di sviluppo.
4. Una vista fisica, che mostra l'hardware del sistema e come i componenti software sono distribuiti sui processori del sistema.
5. Casi d'uso o scenari correlati (+1).

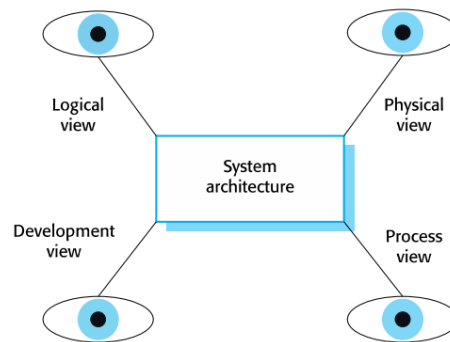


Figura 6.3: Architectural views

Ci sono opinioni divergenti riguardo l'uso dell'UML per descrivere e documentare le architetture software. L'UML è stato progettato per descrivere sistemi orientati agli oggetti e, nella fase di progettazione architeturale, spesso si desidera descrivere sistemi a un livello di astrazione superiore. Le classi di oggetti sono troppo vicine all'implementazione per essere utili nella descrizione architeturale. Non trovo l'UML particolarmente utile durante il processo di progettazione e preferisco notazioni informali, più veloci da scrivere e facilmente disegnabili su una lavagna. L'UML è più utile quando si documenta un'architettura in dettaglio o si utilizza lo sviluppo basato su modelli.

Diversi ricercatori hanno proposto l'uso di linguaggi descrittivi dell'architettura più specializzati (ADL) per descrivere le architetture dei sistemi. Tuttavia, essendo linguaggi specialistici, gli ADL sono difficili da comprendere e utilizzare per gli specialisti di dominio e applicazioni.

6.3 Architectural patterns

L'idea dei pattern come strumento per presentare, condividere e riutilizzare conoscenze sui sistemi software è stata adottata in vari ambiti dell'ingegneria del software. Si può pensare a un pattern architeturale come a una descrizione stilizzata e astratta delle buone pratiche, sperimentata e testata in diversi sistemi e ambienti. Un pattern architeturale dovrebbe quindi descrivere un'organizzazione del sistema che è risultata efficace in sistemi precedenti. Dovrebbe includere informazioni su quando è appropriato o meno utilizzare quel pattern, oltre a dettagli sui punti di forza e di debolezza del pattern.

Nome	MVC (Model-View-Controller)
Descrizione	Separa la presentazione e l'interazione dai dati del sistema. Il sistema è strutturato in tre componenti logiche che interagiscono tra loro. Il componente Modello gestisce i dati del sistema e le operazioni associate su tali dati. Il componente Vista definisce e gestisce come i dati vengono presentati all'utente. Il componente Controller gestisce l'interazione dell'utente (ad esempio, pressioni dei tasti, clic del mouse, ecc.) e passa queste interazioni alla Vista e al Modello. Vedi Figura 6.4.
Esempio	La Figura 6.4 mostra l'architettura di un sistema di applicazione web organizzato utilizzando il pattern MVC.
Quando utilizzato	Utilizzato quando ci sono modi diversi per visualizzare e interagire con i dati. Utilizzato anche quando i requisiti futuri per l'interazione e la presentazione dei dati sono sconosciuti.
Vantaggi	Permette ai dati di cambiare indipendentemente dalla loro rappresentazione e viceversa. Supporta la presentazione degli stessi dati in modi diversi, con le modifiche apportate in una rappresentazione visualizzate in tutte.
Svantaggi	Può comportare codice aggiuntivo e una complessità del codice maggiore quando il modello dei dati e le interazioni sono semplici.

Tabella 6.1: La struttura di un documento di requisiti

La Tabella 6.1 descrive il famoso pattern Model-View-Controller. Questo pattern è alla base della gestione dell'interazione in molti sistemi web-based ed è supportato da numerosi framework di linguaggi di programmazione.

I modelli grafici dell'architettura associata al pattern MVC sono mostrati nella Figura 6.4. Questi presentano l'architettura da diverse prospettive: una vista concettuale e un'architettura del sistema a runtime quando questo pattern viene utilizzato per la gestione dell'interazione in un sistema web-based.

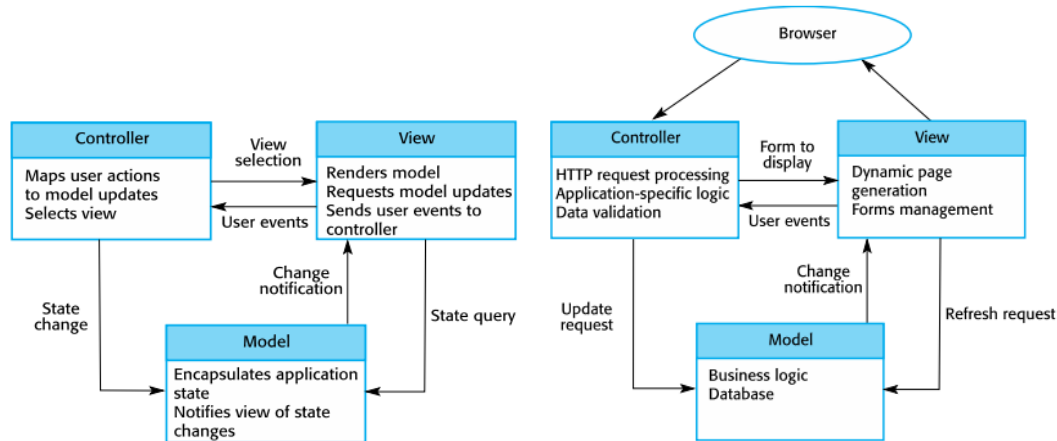


Figura 6.4: Modelli grafici del Model-View-Controller

Vengono presentati alcuni esempi selezionati di pattern ampiamente utilizzati e che riflettono buoni principi di progettazione architeturale.

6.3.1 Layered architecture

Il pattern dell'architettura a Livelli è un altro modo per ottenere separazione e indipendenza. Questo pattern è mostrato nella Tabella 6.2. Qui, la funzionalità del sistema è organizzata in livelli separati, e ogni livello si basa solo sui servizi e le funzionalità offerti dal livello immediatamente sottostante.

Nome	Architettura a livelli
Descrizione	Organizza il sistema in livelli con funzionalità correlate associate a ciascun livello. Un livello fornisce servizi al livello sopra di esso, quindi i livelli più bassi rappresentano servizi di base probabilmente usati in tutto il sistema. Vedi Figura 6.5.
Esempio	Un modello a livelli di un sistema per la condivisione di documenti con copyright conservati in biblioteche diverse, come mostrato in Figura 6.5.
Quando utilizzato	Utilizzato quando si costruiscono nuove funzionalità su sistemi esistenti; quando lo sviluppo è suddiviso tra più team, ciascuno responsabile di un livello di funzionalità; quando è richiesto un sistema con sicurezza multi-livello.
Vantaggi	Consente la sostituzione di interi livelli purché l'interfaccia sia mantenuta. Possono essere forniti servizi ridondanti (es. autenticazione) in ogni livello per aumentare l'affidabilità del sistema.
Svantaggi	In pratica, è spesso difficile fornire una separazione netta tra i livelli e un livello alto può dover interagire direttamente con i livelli inferiori anziché tramite il livello immediatamente sottostante. Le prestazioni possono essere un problema a causa dei multipli livelli di interpretazione di una richiesta di servizio, mentre essa viene elaborata in ciascun livello.

Tabella 6.2: Il modello di architettura a strati

Questo approccio a livelli supporta lo sviluppo incrementale dei sistemi. Durante lo sviluppo di un livello, alcuni dei servizi forniti possono essere resi disponibili agli utenti. Inoltre, quando le

interfacce dei livelli cambiano o si aggiungono nuove funzionalità a un livello, solo il livello adiacente ne risente. Poiché i sistemi a livelli localizzano le dipendenze dalla macchina, diventa più facile fornire implementazioni multi-piattaforma di un sistema applicativo.

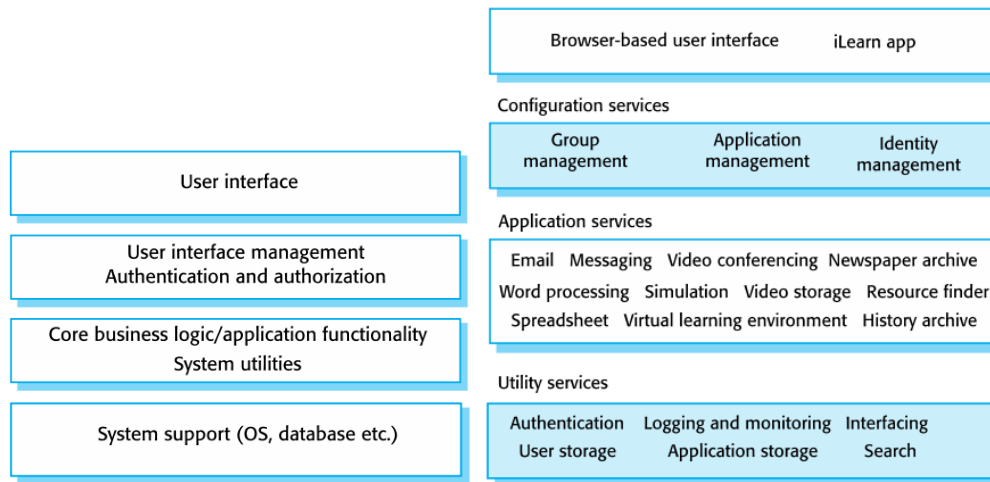


Figura 6.5: Modelli grafici del layered architecture

La Figura 6.5 è un esempio di un'architettura a livelli con quattro livelli. Il livello più basso include il software di supporto al sistema, tipicamente supporto per database e sistema operativo. Il livello successivo è quello applicativo, che comprende i componenti relativi alla funzionalità dell'applicazione e componenti di utilità utilizzati da altri componenti applicativi.

Il terzo livello riguarda la gestione dell'interfaccia utente e fornisce autenticazione e autorizzazione per l'utente, mentre il livello superiore offre le funzionalità dell'interfaccia utente. Ovviamente, il numero di livelli è arbitrario.

6.3.2 Repository architecture

Il pattern Repository (Tabella 6.3), descrive come un insieme di componenti interagenti possa condividere i dati. La maggior parte dei sistemi che gestiscono grandi quantità di dati è organizzata attorno a un database condiviso o a un repository. Questo modello è quindi adatto per applicazioni in cui i dati sono generati da un componente e utilizzati da un altro.

La Figura 6.6 illustra una situazione in cui potrebbe essere utilizzato un repository. Questo diagramma mostra un ambiente di sviluppo integrato (IDE) che include diversi strumenti per supportare lo sviluppo basato su modelli.

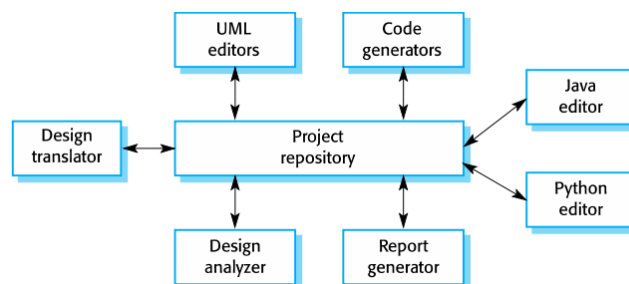


Figura 6.6: Un'architettura di repository per un IDE

Organizzare strumenti attorno a un repository è un modo efficiente per condividere grandi quantità di dati. Non è necessario trasmettere i dati esplicitamente da un componente all'altro. Tuttavia, i componenti devono operare su un modello di dati del repository concordato.

Nome	Repository
Descrizione	Tutti i dati in un sistema sono gestiti in un repository centrale accessibile a tutti i componenti del sistema. I componenti non interagiscono direttamente, ma solo attraverso il repository.
Esempio	La Figura 6.6 mostra un esempio di IDE in cui i componenti utilizzano un repository di informazioni di progettazione del sistema. Ogni strumento software genera informazioni che sono poi disponibili per altri strumenti.
Quando utilizzarlo	Dovresti usare questo pattern quando hai un sistema in cui vengono generati grandi volumi di informazioni che devono essere memorizzate a lungo termine. È utile anche in sistemi guidati dai dati, dove l'inserimento di dati nel repository attiva un'azione o uno strumento.
Vantaggi	I componenti possono essere indipendenti non hanno bisogno di conoscere l'esistenza di altri componenti. Le modifiche apportate da un componente possono essere propagate a tutti gli altri. Tutti i dati possono essere gestiti in modo coerente (ad esempio, i backup vengono eseguiti nello stesso momento), poiché sono tutti in un unico luogo.
Svantaggi	Il repository è un singolo punto di guasto, quindi eventuali problemi nel repository influenzano l'intero sistema. Possono esserci inefficienze nell'organizzare tutte le comunicazioni attraverso il repository. Distribuire il repository su più computer potrebbe risultare difficile.

Tabella 6.3: Il modello repository

6.3.3 Client-server architecture

Il pattern Client-Server (Tabella 6.4), illustra un'organizzazione comunemente utilizzata in fase di esecuzione per sistemi distribuiti. Un sistema che segue il pattern Client-Server è organizzato come un insieme di servizi e server associati, e di client che accedono e utilizzano i servizi. I principali componenti di questo modello sono:

1. Un insieme di server che offrono servizi ad altri componenti. Esempi di server includono server di stampa, server di file e un server di compilazione.
2. Un insieme di client che richiedono i servizi offerti dai server.
3. Una rete che permette ai client di accedere a questi servizi.

Nome	Client-server
Descrizione	In un'architettura client-server, la funzionalità del sistema è organizzata in servizi, con ciascun servizio fornito da un server separato. I client sono gli utenti di questi servizi e accedono ai server per utilizzarli.
Esempio	La Figura 6.7 mostra un esempio di una biblioteca di film e video/DVD organizzata come sistema client-server.
Quando utilizzarlo	Viene utilizzato quando i dati in un database condiviso devono essere accessibili da diverse località. Poiché i server possono essere replicati, può essere utilizzato anche quando il carico su un sistema è variabile.
Vantaggi	Il principale vantaggio di questo modello è che i server possono essere distribuiti su una rete. Funzionalità generali (ad esempio, un servizio di stampa) possono essere disponibili per tutti i client e non devono essere implementate da tutti i servizi.
Svantaggi	Ogni servizio rappresenta un singolo punto di guasto, quindi è suscettibile agli attacchi di negazione del servizio o al fallimento del server. Le prestazioni possono essere imprevedibili poiché dipendono sia dalla rete che dal sistema. Possono sorgere problemi di gestione se i server sono di proprietà di organizzazioni diverse.

Tabella 6.4: Il modello client-server

La Figura 6.7 è un esempio di sistema basato sul modello client-server. Questo è un sistema web multiutente per fornire una libreria di film e fotografie. In questo sistema, diversi server gestiscono

e visualizzano i vari tipi di media. I fotogrammi video devono essere trasmessi rapidamente e in sincronia ma a risoluzione relativamente bassa. Le immagini fisse, invece, devono essere mantenute ad alta risoluzione, quindi è appropriato mantenerle su un server separato.

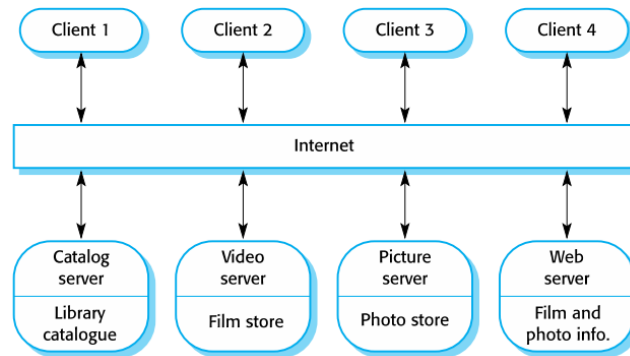


Figura 6.7: Un'architettura client-server per una cineteca

Il catalogo deve essere in grado di gestire una varietà di query e fornire collegamenti al sistema informativo web che include dati sui film e sui videoclip, e a un sistema di e-commerce che supporta la vendita di fotografie, film e videoclip. Il programma client è semplicemente un'interfaccia utente integrata, costruita utilizzando un browser web, per accedere a questi servizi.

6.3.4 Pipe and filter architecture

Il pattern Pipe e Filter (Tabella 6.5) è un modello dell'organizzazione in fase di esecuzione di un sistema in cui trasformazioni funzionali processano i loro input e producono output. I dati fluiscono da un processo all'altro e vengono trasformati man mano che attraversano la sequenza. Ogni fase di elaborazione è implementata come una trasformazione.

Il nome "pipe e filtro" proviene dal sistema Unix originale, in cui era possibile collegare processi usando i "pipe" (condotti). Questi trasferivano un flusso di testo da un processo all'altro.

Varianti di questo pattern sono quando le trasformazioni sono sequenziali e i dati vengono processati in batch, questo modello architetturale a pipe e filtri diventa un modello sequenziale a batch, una struttura comune per sistemi di elaborazione dei dati.

Nome	Pipe e Filter
Descrizione	L'elaborazione dei dati in un sistema è organizzata in modo che ogni componente di elaborazione (filtro) sia discreto e svolga un tipo di trasformazione dei dati. I dati fluiscono (come in un condotto) da un componente all'altro per l'elaborazione.
Esempio	La Figura 6.8 mostra un esempio di sistema pipe e filter utilizzato per l'elaborazione delle fatture.
Quando utilizzarlo	Comunemente utilizzato in applicazioni di elaborazione dati (sia in batch che basate su transazioni) dove gli input vengono processati in fasi separate per generare output correlati.
Vantaggi	Facile da comprendere e supporta il riutilizzo delle trasformazioni. Lo stile del flusso di lavoro si adatta alla struttura di molti processi aziendali. L'evoluzione attraverso l'aggiunta di trasformazioni è semplice. Può essere implementato sia come sistema sequenziale che concorrente.
Svantaggi	Il formato per il trasferimento dei dati deve essere concordato tra le trasformazioni che comunicano. Ogni trasformazione deve analizzare il proprio input e convertire l'output nel formato concordato. Ciò aumenta il sovraccarico del sistema e può rendere impossibile il riutilizzo delle trasformazioni funzionali che utilizzano strutture dati incompatibili.

Tabella 6.5: Il modello client-server

Un esempio di questo tipo di architettura di sistema, usato in un'applicazione di elaborazione batch, è mostrato in Figura 6.8. Un'organizzazione ha emesso fatture per i clienti. Una volta alla

settimana, i pagamenti effettuati vengono riconciliati con le fatture. Per le fatture che sono state pagate, viene emessa una ricevuta. Per quelle che non sono state pagate entro il termine di pagamento previsto, viene inviato un sollecito.

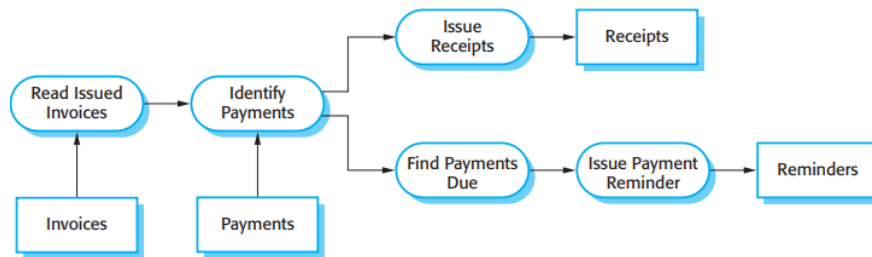


Figura 6.8: Un'architettura pipe and filter

6.4 Application architectures

I sistemi applicativi sono progettati per soddisfare le esigenze di un'azienda o di un'organizzazione. Tutte le aziende hanno molte esigenze in comune: devono assumere personale, emettere fatture, mantenere contabilità e così via. Le aziende che operano nello stesso settore utilizzano applicazioni specifiche per quel settore. Di conseguenza, i sistemi applicativi utilizzati da queste aziende hanno molto in comune.

Queste somiglianze hanno portato allo sviluppo di architetture software che descrivono la struttura e l'organizzazione di particolari tipi di sistemi software. Le architetture applicative racchiudono le caratteristiche principali di una classe di sistemi. Questi sistemi hanno un'architettura e componenti standardizzati, che vengono configurati e adattati per creare una specifica applicazione aziendale.

Come progettista software, è possibile utilizzare modelli di architetture applicative in vari modi:

1. Come punto di partenza per il processo di progettazione architeturale.
2. Come lista di controllo per la progettazione.
3. Come modo per organizzare il lavoro del team di sviluppo.
4. Come mezzo per valutare i componenti per il riuso.
5. Come vocabolario per parlare delle applicazioni.

Esistono molti tipi di sistemi applicativi e, in alcuni casi, possono sembrare molto diversi. Tuttavia, applicazioni superficialmente dissimili possono avere molto in comune e quindi condividere un'architettura applicativa astratta. Illustrerò questo descrivendo le architetture di due tipi di applicazioni:

1. *Applicazioni di elaborazione delle transazioni:* Queste sono applicazioni incentrate sui database che elaborano richieste degli utenti per informazioni e aggiornano le informazioni in un database. Sono i tipi più comuni di sistemi aziendali interattivi. Sono organizzati in modo tale che le azioni degli utenti non possano interferire tra loro e venga mantenuta l'integrità del database. Questa classe di sistemi include sistemi bancari interattivi, sistemi di e-commerce, sistemi informativi e sistemi di prenotazione.
2. *Sistemi di elaborazione del linguaggio:* Sono sistemi in cui le intenzioni dell'utente sono espresse in un linguaggio formale, come un linguaggio di programmazione. Il sistema di elaborazione del linguaggio elabora questo linguaggio in un formato interno e poi interpreta questa rappresentazione interna. I sistemi di elaborazione del linguaggio più noti sono i compilatori, che traducono programmi di alto livello in codice macchina.

6.4.1 Sistemi di elaborazione delle transazioni

I sistemi di elaborazione delle transazioni sono progettati per gestire le richieste degli utenti di informazioni da un database o le richieste per aggiornare un database.

Da una prospettiva dell'utente, una transazione è qualsiasi sequenza coerente di operazioni che soddisfa un obiettivo, come "trovare gli orari dei voli da Londra a Parigi".

I sistemi di elaborazione delle transazioni sono generalmente sistemi interattivi in cui gli utenti fanno richieste di servizio asincrone. La Figura 6.9 illustra la struttura architetturale concettuale delle applicazioni di elaborazione delle transazioni. In primo luogo, un utente effettua una richiesta al sistema tramite un componente di elaborazione I/O. La richiesta viene elaborata da una logica specifica dell'applicazione. Viene quindi creata una transazione e passata a un gestore delle transazioni, solitamente integrato nel sistema di gestione del database. Dopo che il gestore delle transazioni ha verificato che la transazione è stata completata correttamente, segnala all'applicazione che l'elaborazione è terminata.

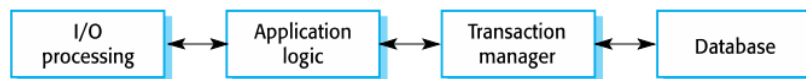


Figura 6.9: La struttura delle applicazioni di elaborazione delle transazioni

I sistemi di elaborazione delle transazioni possono essere organizzati come un'architettura "pipe and filter", con componenti di sistema responsabili per l'input, l'elaborazione e l'output. Ad esempio, consideriamo un sistema bancario che consente ai clienti di consultare i loro conti e prelevare denaro da un bancomat. Il sistema è composto da due componenti software cooperanti: il software del bancomat e il software di elaborazione dei conti nel server di database della banca. I componenti di input e output sono implementati come software nel bancomat, mentre il componente di elaborazione fa parte del server di database della banca. La Figura 6.10 mostra l'architettura di questo sistema, illustrando le funzioni dei componenti di input, processo e output.

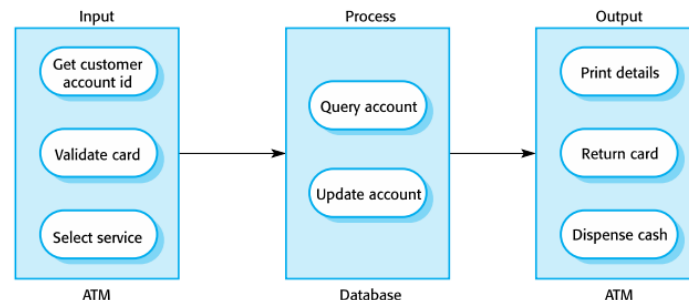


Figura 6.10: L'architettura software di un sistema ATM

6.4.2 Sistemi informativi

Un sistema informativo consente un accesso controllato a una grande base di informazioni. I sistemi informativi sono quasi sempre sistemi basati su web, dove l'interfaccia utente è implementata in un browser web.

La Figura 6.11 presenta un modello molto generale di un sistema informativo. Il sistema è modellato utilizzando un approccio a livelli in cui il livello superiore supporta l'interfaccia utente e il livello inferiore è il database del sistema. Il livello di comunicazione con l'utente gestisce tutti gli input e output dall'interfaccia utente, mentre il livello di recupero delle informazioni include la logica specifica dell'applicazione per accedere e aggiornare il database.

Come esempio di istanza di questo modello a livelli, la Figura 6.12 mostra l'architettura del sistema Mentcare:

1. Il livello superiore è un'interfaccia utente basata su browser.
2. Il secondo livello fornisce le funzionalità dell'interfaccia utente che sono erogate tramite il browser web.

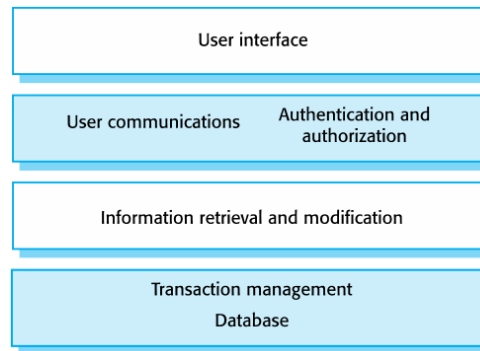


Figura 6.11: Architettura del sistema informativo stratificato

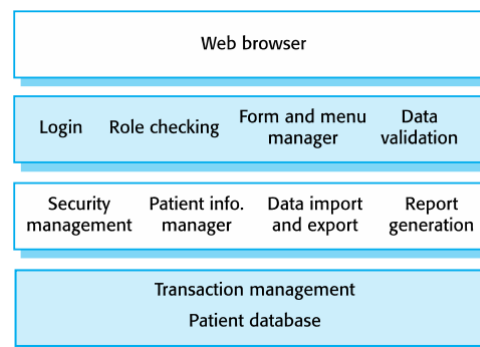


Figura 6.12: L'architettura del sistema Mentcare

3. Il terzo livello implementa le funzionalità del sistema e fornisce componenti che implementano la sicurezza del sistema, la creazione e l'aggiornamento delle informazioni sui pazienti, l'importazione e l'esportazione di dati dei pazienti da altri database e generatori di report che creano report di gestione.
4. Infine, il livello inferiore, costruito utilizzando un sistema di gestione di database commerciale, fornisce la gestione delle transazioni e la memorizzazione persistente dei dati.

I sistemi di gestione delle informazioni e delle risorse sono talvolta anche sistemi di elaborazione delle transazioni. Ad esempio, i sistemi di e-commerce sono sistemi di gestione delle risorse basati su Internet che accettano ordini elettronici per beni o servizi e quindi organizzano la consegna di questi beni o servizi al cliente. In un sistema di e-commerce, il livello specifico dell'applicazione include funzionalità aggiuntive che supportano un "carrello" in cui gli utenti possono inserire diversi articoli in transazioni separate e poi pagarli tutti insieme in un'unica transazione.

Questi sistemi sono spesso implementati come sistemi distribuiti con un'architettura client-server multi-tier:

1. Il server web è responsabile di tutte le comunicazioni con l'utente, con l'interfaccia utente implementata utilizzando un browser web;
2. Il server applicativo è responsabile dell'implementazione della logica specifica dell'applicazione e delle richieste di memorizzazione e recupero delle informazioni;
3. Il server di database sposta le informazioni da e verso il database e gestisce la gestione delle transazioni.

6.4.3 Sistemi di elaborazione del linguaggio

I sistemi di elaborazione del linguaggio traducono un linguaggio in una rappresentazione alternativa di quel linguaggio.

Una possibile architettura per un sistema di elaborazione del linguaggio per un linguaggio di programmazione è illustrata nella Figura 6.13. Le istruzioni del linguaggio sorgente definiscono il programma da eseguire, e un traduttore le converte in istruzioni per una macchina astratta. Queste istruzioni vengono quindi interpretate da un altro componente che le preleva per eseguirle e le esegue utilizzando (se necessario) i dati dall'ambiente. L'output del processo è il risultato dell'interpretazione delle istruzioni sui dati di input.

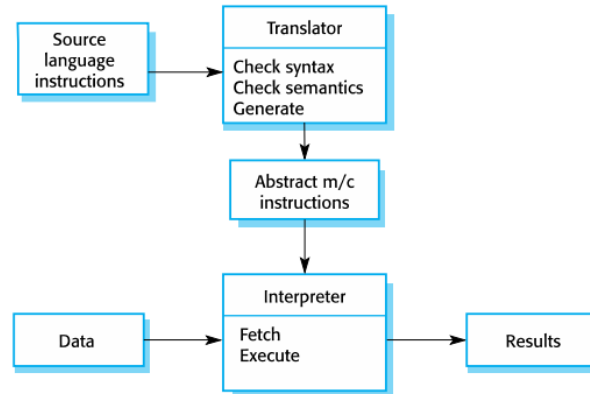


Figura 6.13: L'architettura di un sistema di elaborazione del linguaggio

I compilatori dei linguaggi di programmazione che fanno parte di un ambiente di programmazione più generale hanno un'architettura generica (Figura 6.14) che include i seguenti componenti:

1. Un analizzatore lessicale, che prende i token di input del linguaggio e li converte in una forma interna.
2. Una tabella dei simboli, che contiene informazioni sui nomi delle entità (variabili, nomi di classi, nomi di oggetti, ecc.) utilizzati nel testo che viene tradotto.
3. Un analizzatore sintattico, che controlla la sintassi del linguaggio in fase di traduzione.
4. Un albero sintattico, che è una struttura interna che rappresenta il programma in fase di compilazione.
5. Un analizzatore semantico, che utilizza le informazioni dall'albero sintattico e dalla tabella dei simboli per verificare la correttezza semantica del testo di input del linguaggio.
6. Un generatore di codice, che "percorre" l'albero sintattico e genera codice per una macchina astratta.

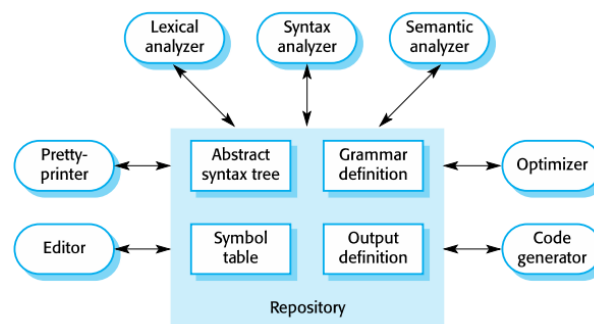


Figura 6.14: Un'architettura di repository per un sistema di elaborazione del linguaggio

La Figura 6.15 illustra come un sistema di elaborazione del linguaggio possa essere parte di un insieme integrato di strumenti di supporto alla programmazione. In questo esempio, la tabella dei simboli e l'albero sintattico agiscono come un repository centrale di informazioni.

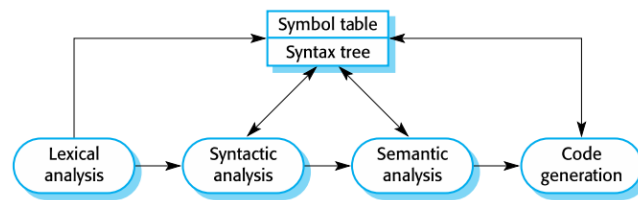


Figura 6.15: Un'architettura di repository per un sistema di elaborazione del linguaggio

I compilatori possono essere implementati utilizzando un modello composto di repository e pipe and filter. In un'architettura del compilatore, la tabella dei simboli è un repository per i dati condivisi. Le fasi di analisi lessicale, sintattica e semantica sono organizzate in sequenza, come mostrato nella Figura 6.15, e comunicano attraverso la tabella dei simboli condivisa.

6.5 Punti chiave

- Un'architettura software è una descrizione di come è organizzato un sistema software.
- Le decisioni di progettazione architetturale includono la scelta del tipo di applicazione, la distribuzione del sistema e gli stili architetturali da utilizzare.
- Le architetture possono essere documentate da diverse prospettive o viste, come una vista concettuale, una vista logica, una vista di processo e una vista di sviluppo.
- I pattern architetturali sono un mezzo per riutilizzare la conoscenza delle architetture di sistema generiche. Descrivono l'architettura, spiegano quando può essere utilizzata e ne descrivono vantaggi e svantaggi.
- I modelli delle architetture dei sistemi applicativi ci aiutano a comprendere e confrontare le applicazioni, a convalidare i progetti dei sistemi applicativi e a valutare i componenti su larga scala per il riutilizzo.
- I sistemi di elaborazione delle transazioni sono sistemi interattivi che consentono a più utenti di accedere e modificare in remoto le informazioni in un database.
- I sistemi di elaborazione del linguaggio vengono utilizzati per tradurre testi da un linguaggio a un altro e per eseguire le istruzioni specificate nel linguaggio di input. Includono un traduttore e una macchina astratta che esegue il linguaggio generato.

Capitolo 7

Test del software

Il **testing** ha l'obiettivo di mostrare che un programma svolge il compito per cui è stato progettato e di identificare eventuali difetti prima che venga messo in uso. Quando si testa il software, si esegue un programma utilizzando dati artificiali. Si verificano i risultati del test per individuare errori, anomalie o informazioni sulle caratteristiche non funzionali del programma.

Testare il software implica due obiettivi principali:

1. Dimostrare al team di sviluppo e al cliente che il software soddisfa i requisiti. Per il software personalizzato, ciò significa che dovrebbe esserci almeno un test per ogni requisito nel documento dei requisiti. Per i prodotti software generici, ci dovrebbero essere test per tutte le funzionalità del sistema che verranno incluse nella versione finale.
2. Trovare input o sequenze di input in cui il comportamento del software è scorretto, indesiderato o non conforme alle specifiche. Questi problemi sono causati da difetti (bug) nel software. Quando si testano i software per trovare difetti, si cerca di eliminare comportamenti indesiderati come crash di sistema, interazioni indesiderate con altri sistemi, calcoli errati e corruzione dei dati.

La prima attività è il testing di validazione, dove ci si aspetta che il sistema funzioni correttamente usando casi di test che riflettano l'uso previsto del sistema. La seconda è il testing per la rilevazione dei difetti, dove i casi di test sono progettati per esporre i difetti. Non c'è una separazione netta tra questi due approcci di testing: durante il testing di validazione, si possono trovare difetti e viceversa.

Il testing è parte di un processo più ampio di verifica e validazione (V&V). Verifica e validazione non sono la stessa cosa, anche se spesso vengono confuse.

- *Validazione*: Stiamo costruendo il prodotto giusto? Il software deve essere conforme alle specifiche. La verifica del software consiste nel controllare che il software soddisfi i requisiti funzionali e non funzionali stabiliti.
- *Verifica*: Stiamo costruendo il prodotto nel modo giusto? La validazione è un processo più generale, il cui scopo è assicurarsi che il software risponda alle aspettative del cliente, dimostrando che il software fa ciò che il cliente si aspetta.

Lo scopo dei processi di verifica e validazione è quello di stabilire la fiducia che il sistema software sia "adatto allo scopo" ("fit for purpose"). Il livello di fiducia richiesto dipende dallo scopo del sistema, dalle aspettative degli utenti e dall'ambiente di mercato attuale:

1. *Scopo del software*: Più il software è critico, maggiore è l'importanza che sia affidabile.
2. *Aspettative degli utenti*: A causa delle esperienze passate con software difettosi e inaffidabili, gli utenti a volte hanno basse aspettative sulla qualità del software.
3. *Ambiente di mercato*: In un ambiente competitivo, l'azienda può decidere di rilasciare un programma prima che sia stato completamente testato per arrivare prima sul mercato. Se il prodotto è molto economico, gli utenti possono tollerare un livello di affidabilità inferiore.

Oltre al testing del software, il processo di verifica e validazione può includere ispezioni e test. Queste analizzano e verificano i requisiti del sistema, i modelli di progettazione, il codice sorgente del programma e persino i test proposti. Queste tecniche di V&V “statiche” non richiedono l’esecuzione del software. La Figura 7.1 mostra che ispezioni e test supportano la V&V in diverse fasi del processo software.

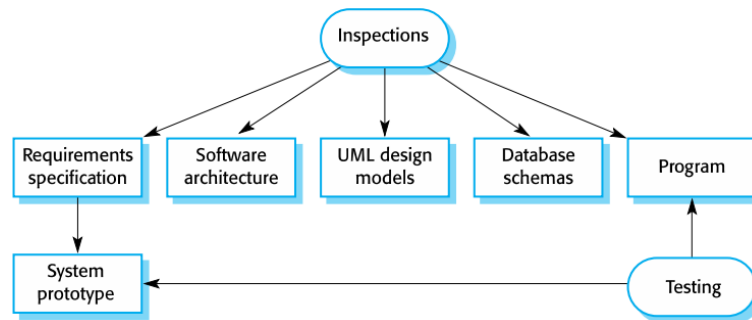


Figura 7.1: Ispezioni e test

Le ispezioni si concentrano principalmente sul codice sorgente di un sistema, ma qualsiasi rappresentazione leggibile del software, come i requisiti o un modello di progettazione, può essere ispezionata. Durante un’ispezione, si usa la conoscenza del sistema, del dominio di applicazione e del linguaggio di programmazione o modellazione per scoprire errori.

L’ispezione del software ha tre vantaggi rispetto al testing:

1. Durante il testing, un errore può mascherare altri errori. Poiché l’ispezione non implica l’esecuzione del sistema, non si devono affrontare le interazioni tra errori e una singola sessione di ispezione può scoprire molti errori.
2. Versioni incomplete di un sistema possono essere ispezionate senza costi aggiuntivi. Se un programma è incompleto, per testarlo servono “test harness” specializzati, aumentando i costi di sviluppo.
3. Oltre a cercare difetti, un’ispezione può considerare attributi qualitativi come la conformità agli standard, la portabilità e la manutenibilità.

Tuttavia, le ispezioni non possono sostituire il testing, poiché non sono efficaci nel rilevare difetti derivanti da interazioni impreviste tra parti diverse del programma, problemi di temporizzazione o problemi di prestazioni del sistema. Ispezioni e test sono tecniche di verifica complementari e non opposte. Entrambe dovrebbero essere utilizzate durante il processo V&V.

In questo capitolo ci concentriamo sul testing e sui processi di testing. La Figura 7.2 è un modello astratto del processo di testing tradizionale, utilizzato nello sviluppo guidato dalla pianificazione. I casi di test sono specifiche degli input e degli output attesi dal sistema (i risultati del test), insieme a una dichiarazione di ciò che viene testato. I dati di test sono gli input ideati per testare un sistema. L’esecuzione del test può essere automatizzata, e i risultati del test vengono confrontati automaticamente con quelli previsti.

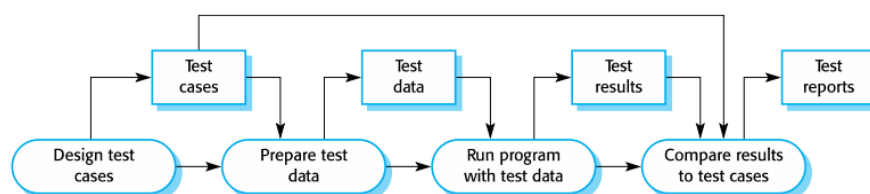


Figura 7.2: Un modello del processo di test del software

In genere, un sistema software commerciale passa attraverso tre fasi di testing:

1. Testing durante lo sviluppo, dove il sistema viene testato durante lo sviluppo per scoprire bug e difetti.

2. Testing della versione, dove un team di test separato testa una versione completa del sistema prima che venga rilasciata agli utenti.
3. Testing dell'utente, in cui gli utenti o potenziali utenti testano il sistema nel loro ambiente.

7.1 Test di sviluppo

Il testing durante lo sviluppo include tutte le attività di testing che vengono svolte dal team che sviluppa il sistema.

Ci sono tre fasi di testing durante lo sviluppo:

1. *Testing dell'unità*, in cui vengono testate singole unità del programma o classi di oggetti. Il testing dell'unità dovrebbe concentrarsi sul test delle funzionalità degli oggetti o dei metodi.
2. *Testing del componente*, in cui vengono integrate diverse unità per creare componenti compositi. Il testing del componente dovrebbe concentrarsi sul test delle interfacce del componente, che forniscono accesso alle funzioni del componente.
3. *Testing del sistema*, in cui alcuni o tutti i componenti di un sistema vengono integrati e il sistema viene testato nel suo complesso. Il testing del sistema dovrebbe concentrarsi sul test delle interazioni tra i componenti.

7.1.1 Unit testing

Il testing dell'unità è il processo di test dei componenti del programma, come metodi o classi di oggetti. Le singole funzioni o metodi sono il tipo più semplice di componente. I test dovrebbero consistere in chiamate a queste routine con diversi parametri di input.

Quando si testano classi di oggetti, è necessario progettare i test in modo da coprire tutte le funzionalità dell'oggetto. Ciò significa che bisogna testare tutte le operazioni associate all'oggetto, impostare e controllare il valore di tutti gli attributi associati e mettere l'oggetto in tutti gli stati possibili. In questo modo, si simuleranno tutti gli eventi che causano un cambiamento di stato.

Ad esempio, l'oggetto stazione meteorologica. Gli attributi e le operazioni di questo oggetto sono mostrati nella Figura 7.3. Ha un solo attributo, che è il suo identificatore. È quindi necessario solo un test per verificare che sia stata configurata correttamente. Bisogna definire casi di test per tutti i metodi associati all'oggetto, come `reportWeather` e `reportStatus`. Idealmente, i metodi dovrebbero essere testati isolatamente, ma in alcuni casi sono necessarie sequenze di test. Ad esempio, per testare il metodo che spegne gli strumenti della stazione meteorologica (`shutdown`), è necessario prima aver eseguito il metodo `restart`.

WeatherStation
identifier
<code>reportWeather ()</code> <code>reportStatus ()</code> <code>powerSave (instruments)</code> <code>remoteControl (commands)</code> <code>reconfigure (commands)</code> <code>restart (instruments)</code> <code>shutdown (instruments)</code>

Figura 7.3: Test della stazione meteorologica

Per testare gli stati della stazione meteorologica, si può utilizzare un modello di stato. Utilizzando questo modello, si identificano sequenze di transizioni di stato che devono essere testate e si definiscono sequenze di eventi per forzare queste transizioni. Esempi di sequenze di stato da testare nella stazione meteorologica includono:

- Shutdown → Running → Shutdown
- Configuring → Running → Testing → Transmitting → Running

- Running → Collecting → Running → Summarizing → Transmitting → Running

Quando possibile, il testing dell'unità dovrebbe essere automatizzato. Nel testing dell'unità automatizzato, si utilizza un framework di automazione del test, come JUnit per scrivere ed eseguire i test del programma. I framework di testing dell'unità forniscono classi di test generiche che possono essere estese per creare casi di test specifici. Possono quindi eseguire tutti i test implementati e riportare, spesso tramite un'interfaccia grafica (GUI), il successo o meno dei test.

Un test automatizzato ha tre parti:

1. Una *parte di configurazione*, dove si inizializza il sistema con il caso di test, ovvero gli input e gli output attesi.
2. Una *parte di chiamata*, dove si chiama l'oggetto o il metodo da testare.
3. Una *parte di asserzione*, dove si confronta il risultato della chiamata con il risultato atteso. Se l'asserzione è vera, il test ha avuto successo; se è falsa, il test è fallito.

7.1.2 Scelta dei casi di unit test

Il testing è costoso e richiede tempo, quindi è importante scegliere casi di test dell'unità efficaci. L'efficacia, in questo caso, implica due aspetti:

1. I casi di test dovrebbero dimostrare che, quando usato come previsto, il componente testato fa ciò che dovrebbe fare.
2. Se ci sono difetti nel componente, questi dovrebbero essere rivelati dai casi di test.

È quindi necessario progettare due tipi di casi di test. Il primo dovrebbe riflettere il funzionamento normale del programma e dimostrare che il componente funziona. L'altro tipo di caso di test dovrebbe basarsi sull'esperienza, prendendo in considerazione i problemi più comuni. Dovrebbe utilizzare input anomali per verificare che siano correttamente processati e non mandino in crash il componente.

Due strategie efficaci per scegliere i casi di test sono:

1. *Testing per partizione*, in cui si identificano gruppi di input con caratteristiche comuni e che devono essere processati allo stesso modo. I test vengono scelti all'interno di ciascun gruppo.
2. *Testing basato su linee guida*, in cui si utilizzano linee guida per scegliere i casi di test. Queste linee guida riflettono l'esperienza precedente riguardo ai tipi di errori che i programmatori spesso commettono nello sviluppo dei componenti.

Gli input e gli output di un programma possono essere considerati come membri di insiemi con caratteristiche comuni. I programmi di solito si comportano in modo simile per tutti i membri di un insieme.

A causa di questo comportamento equivalente, queste classi vengono talvolta chiamate partizioni di equivalenza o domini. Un approccio sistematico alla progettazione dei casi di test si basa sull'identificazione di tutte le partizioni di input e output per un sistema o componente. I casi di test sono progettati in modo che gli input o gli output rientrino in queste partizioni.

Nella Figura 7.4, l'ellisse grande a sinistra rappresenta l'insieme di tutti gli input possibili per il programma che viene testato. Le ellissi più piccole rappresentano le partizioni di equivalenza. Un programma in fase di test dovrebbe processare tutti i membri di una partizione di equivalenza di input allo stesso modo. Le partizioni di equivalenza di output sono partizioni in cui tutti gli output hanno qualcosa in comune.

Una volta identificato un insieme di partizioni, si scelgono casi di test da ciascuna di queste partizioni. Una buona regola per la selezione dei casi di test è scegliere casi ai confini delle partizioni, più alcuni vicini al punto medio della partizione.

Si identificano le partizioni utilizzando la specifica del programma o la documentazione utente e, in base all'esperienza, prevedendo le classi di valori di input che potrebbero rilevare errori. Ad esempio, si supponga che una specifica di programma stabilisca che il programma accetta da quattro a otto input costituiti da numeri interi di cinque cifre maggiori di 10.000. Si utilizza questa informazione per identificare le partizioni di input e i possibili valori di test. Questi sono mostrati nella Figura 7.5.

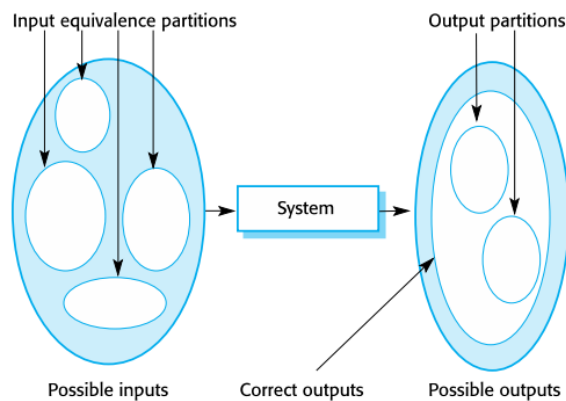


Figura 7.4: Partizionamento di equivalenza

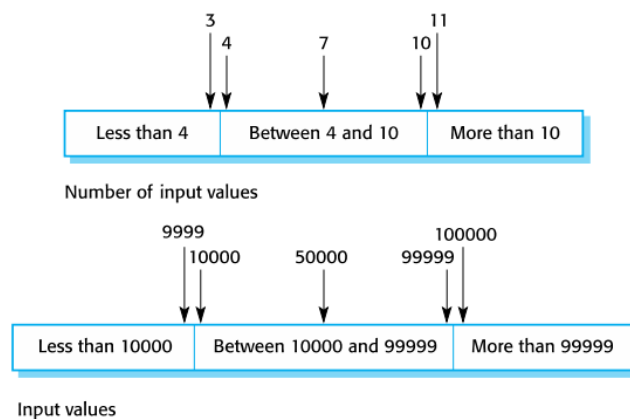


Figura 7.5: Esempio di partizionamento di equivalenza

È anche possibile utilizzare le linee guida di testing per scegliere i casi di test. Le linee guida raccolgono conoscenze sui tipi di casi di test efficaci per la rilevazione di errori. Ad esempio, quando si testano programmi con sequenze, array o liste, alcune linee guida che potrebbero rivelare difetti includono:

1. Testare il software con sequenze che contengono solo un singolo valore.
2. Utilizzare sequenze di dimensioni diverse in test diversi.
3. Derivare test in modo che il primo, il mezzo e l'ultimo elemento della sequenza siano accessibili.

Alcune delle linee guida più generali che suggerisce sono:

1. Scegliere input che costringano il sistema a generare tutti i messaggi di errore.
2. Progettare input che causino l'overflow dei buffer di input.
3. Ripetere più volte lo stesso input o una serie di input.
4. Forzare la generazione di output non validi.
5. Forzare i risultati del calcolo a essere troppo grandi o troppo piccoli.

7.1.3 Testing dei componenti

I componenti software sono spesso costituiti da diversi oggetti che interagiscono tra loro. Ad esempio, nel sistema della stazione meteorologica, il componente di riconfigurazione include oggetti che gestiscono ciascun aspetto della riconfigurazione. L'accesso alla funzionalità di questi oggetti avviene tramite le interfacce dei componenti. Il testing dei componenti composti dovrebbe quindi

concentrarsi sul dimostrare che l'interfaccia o le interfacce del componente si comportano secondo la specifica. Si può supporre che i test delle singole unità sugli oggetti all'interno del componente siano stati completati.

La Figura 7.6 illustra l'idea di test dell'interfaccia del componente. Supponiamo che i componenti A, B e C siano stati integrati per creare un componente o sottosistema più grande. I casi di test non vengono applicati ai singoli componenti, ma all'interfaccia del componente composito creato combinando questi componenti.

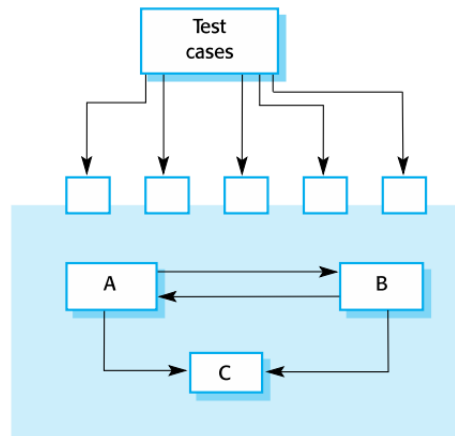


Figura 7.6: Test dell'interfaccia

Esistono diversi tipi di interfaccia tra i componenti del programma e, di conseguenza, diversi tipi di errore d'interfaccia che possono verificarsi:

1. *Interfacce a parametri*: sono interfacce in cui vengono passati dati o, talvolta, riferimenti a funzioni da un componente all'altro.
2. *Interfacce a memoria condivisa*: sono interfacce in cui un blocco di memoria è condiviso tra i componenti.
3. *Interfacce procedurali*: sono interfacce in cui un componente incapsula un insieme di procedure che possono essere chiamate da altri componenti.
4. *Interfacce di passaggio di messaggi*: sono interfacce in cui un componente richiede un servizio da un altro componente inviandogli un messaggio.

Gli errori d'interfaccia sono una delle forme più comuni di errore nei sistemi complessi. Questi errori si dividono in tre categorie:

1. *Uso improprio dell'interfaccia*: un componente chiamante richiama un altro componente e commette un errore nell'uso della sua interfaccia. Questo tipo di errore è comune nelle interfacce a parametri, dove i parametri possono essere del tipo sbagliato o passati nell'ordine sbagliato, oppure il numero di parametri passati può essere errato.
2. *Incomprensione dell'interfaccia*: un componente chiamante fraintende la specifica dell'interfaccia del componente chiamato e fa ipotesi sul suo comportamento.
3. *Errori di sincronizzazione*: il produttore di dati e il consumatore di dati possono operare a velocità diverse o può accedere a informazioni non aggiornate.

Alcune linee guida generali per il testing delle interfacce sono:

1. Progettare una serie di test in cui i valori dei parametri per i componenti esterni si trovino agli estremi dei loro intervalli.
2. Quando vengono passati puntatori tramite un'interfaccia, testare sempre l'interfaccia con parametri puntatori nulli.
3. Progettare test che causino deliberatamente il fallimento del componente.

4. Usare il testing di stress nei sistemi di passaggio di messaggi.
5. Quando più componenti interagiscono tramite memoria condivisa, progettare test che variano l'ordine in cui questi componenti vengono attivati.

7.1.4 Testing del sistema

Il testing del sistema durante lo sviluppo prevede l'integrazione dei componenti per creare una versione del sistema e poi testare il sistema integrato. Il testing del sistema verifica che i componenti siano compatibili, interagiscano correttamente e trasferiscano i dati giusti al momento giusto attraverso le loro interfacce. Ovviamente si sovrappone al testing dei componenti, ma ci sono due differenze importanti:

1. Durante il testing del sistema, i componenti riutilizzabili che sono stati sviluppati separatamente e i sistemi commerciali possono essere integrati con componenti di nuova progettazione. Successivamente viene testato il sistema completo.
2. In questa fase possono essere integrati componenti sviluppati da diversi membri del team o sottoteam. Il testing del sistema è un processo collettivo piuttosto che individuale. In alcune aziende, il testing del sistema può coinvolgere un team di testing separato senza il coinvolgimento di designer e programmatori.

Grazie alla sua attenzione alle interazioni, il testing basato sui casi d'uso è un approccio efficace per il testing del sistema. Di solito diversi componenti o oggetti implementano ciascun caso d'uso nel sistema. Il testing del caso d'uso forza queste interazioni. Se si è sviluppato un diagramma di sequenza per modellare l'implementazione del caso d'uso, si possono vedere gli oggetti o i componenti coinvolti nell'interazione.

Nel caso di esempio della stazione meteorologica, il software di sistema riporta dati meteorologici riassunti a un computer remoto. La Figura 7.7 mostra la sequenza di operazioni nella stazione meteorologica quando risponde a una richiesta di raccolta dati per il sistema di mappatura.

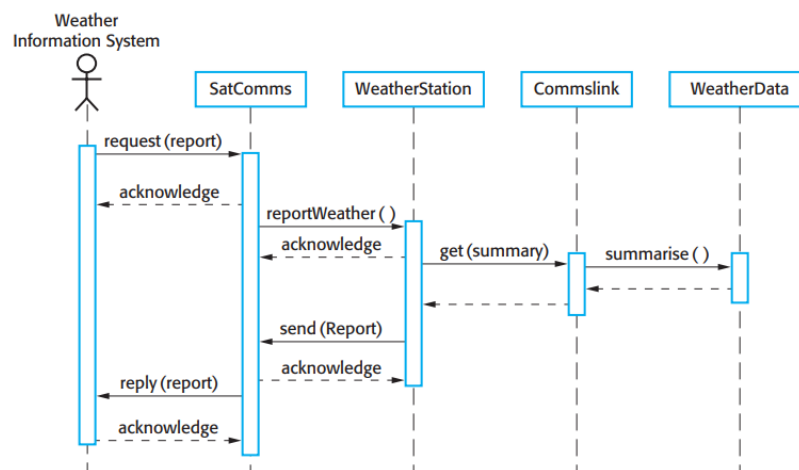


Figura 7.7: Raccogliere la sequenza dei dati meteorologici

Pertanto, emettere una richiesta per un report comporterà l'esecuzione della seguente sequenza di metodi:

```
SatComms:request → WeatherStation:reportWeather → Commslink:Get(summary) →
WeatherData:summarize
```

Il diagramma di sequenza aiuta a progettare i casi di test specifici di cui si ha bisogno, poiché mostra quali input sono richiesti e quali output vengono creati:

1. Un input di una richiesta per un report dovrebbe avere un riconoscimento associato. Da una richiesta dovrebbe essere infine restituito un report. Durante il testing, si dovrebbero creare dati riassunti che possono essere utilizzati per verificare che il report sia organizzato correttamente.

2. Una richiesta di input per un report a `WeatherStation` risulta nella generazione di un report riassunto. Si può testare questo in isolamento creando dati grezzi corrispondenti al riepilogo preparato per il test di `SatComms` e verificando che l'oggetto `WeatherStation` produca correttamente questo riepilogo. Questi dati grezzi vengono utilizzati anche per testare l'oggetto `WeatherData`.

Il testing esaustivo, in cui viene testata ogni possibile sequenza di esecuzione del programma, è impossibile. Il testing, quindi, deve basarsi su un sottoinsieme dei possibili casi di test. Ad esempio:

1. Tutte le funzioni del sistema accessibili tramite menu dovrebbero essere testate.
2. Le combinazioni di funzioni (ad esempio, la formattazione del testo) che sono accessibili dallo stesso menu devono essere testate.
3. Dove è previsto l'input dell'utente, tutte le funzioni devono essere testate sia con input corretto che con input errato.

7.2 Test-driven development

Lo sviluppo guidato dai test (Test-Driven Development - TDD) è un approccio allo sviluppo del software in cui si alternano la scrittura dei test e lo sviluppo del codice. Si sviluppa il codice in modo incrementale, insieme a un set di test per ogni incremento. Non si inizia a lavorare sul prossimo incremento finché il codice sviluppato non supera tutti i test. Lo sviluppo guidato dai test è stato introdotto come parte del metodo di sviluppo agile. Tuttavia, oggi è ampiamente accettato e può essere utilizzato sia in processi agili che pianificati.

Il processo fondamentale di TDD è mostrato nella Figura 7.8. I passaggi del processo sono i seguenti:

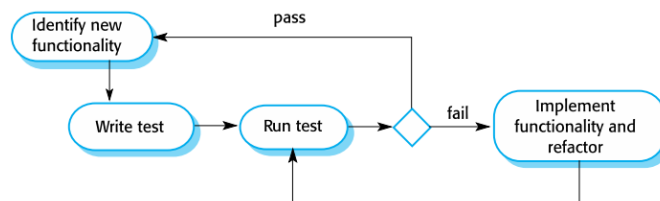


Figura 7.8: Sviluppo basato sui test

1. Si inizia identificando l'incremento di funzionalità richiesto. Questo dovrebbe essere normalmente piccolo e implementabile in poche righe di codice.
2. Si scrive un test per questa funzionalità e lo si implementa come un test automatizzato.
3. Successivamente, si esegue il test, insieme a tutti gli altri test che sono stati implementati. Inizialmente, non si è implementata la funzionalità, quindi il nuovo test fallirà.
4. Si implementa quindi la funzionalità e si riesegue il test.
5. Una volta che tutti i test passano con successo, si passa all'implementazione del prossimo blocco di funzionalità.

Oltre a una migliore comprensione del problema, altri vantaggi dello sviluppo guidato dai test sono:

1. *Copertura del codice.* In linea di principio, ogni segmento di codice che si scrive dovrebbe avere almeno un test associato.
2. *Testing di regressione.* Un set di test viene sviluppato in modo incrementale man mano che il programma viene sviluppato.
3. *Debugging semplificato.* Quando un test fallisce, dovrebbe essere ovvio dove si trova il problema. Il codice appena scritto deve essere controllato e modificato.

4. *Documentazione del sistema.* I test stessi fungono da una forma di documentazione che descrive cosa dovrebbe fare il codice.

Uno dei vantaggi più importanti del TDD è che riduce i costi del testing di regressione. Il testing di regressione comporta l'esecuzione di set di test che sono stati eseguiti con successo dopo che sono state apportate modifiche a un sistema. Il test di regressione verifica che queste modifiche non abbiano introdotto nuovi bug nel sistema e che il nuovo codice interagisca come previsto con il codice esistente. Il testing di regressione è costoso e a volte impraticabile quando un sistema viene testato manualmente, poiché i costi in termini di tempo e sforzo sono molto alti. Il testing automatizzato riduce drasticamente i costi del testing di regressione. Dopo aver apportato una modifica a un sistema nello sviluppo test-first, tutti i test esistenti devono passare con successo prima che venga aggiunta altra funzionalità.

7.3 Testing di rilascio

Il testing di rilascio è il processo di test di una particolare versione di un sistema destinata all'uso al di fuori del team di sviluppo. Normalmente, la release del sistema è destinata a clienti e utenti. Tuttavia, in un progetto complesso, la release potrebbe essere destinata ad altri team che stanno sviluppando sistemi correlati.

Ci sono due importanti differenze tra il testing di rilascio e il testing di sistema durante il processo di sviluppo:

1. Il team di sviluppo del sistema non dovrebbe essere responsabile del testing di rilascio.
2. Il testing di rilascio è un processo di verifica per assicurarsi che un sistema soddisfi i suoi requisiti e sia sufficientemente buono per l'uso da parte dei clienti del sistema.

Il testing di sistema eseguito dal team di sviluppo dovrebbe concentrarsi sul trovare bug nel sistema (defect testing).

L'obiettivo principale del processo di testing di rilascio è convincere il fornitore del sistema che esso sia sufficientemente buono per l'uso. Se è così, può essere rilasciato come prodotto o consegnato al cliente. Il testing di rilascio, quindi, deve dimostrare che il sistema fornisce la funzionalità, le prestazioni e l'affidabilità specificate e che non fallisce durante l'uso normale.

Il testing di rilascio è di solito un processo di testing black-box in cui i test vengono derivati dalle specifiche del sistema. Il sistema viene trattato come una "scatola nera" il cui comportamento può essere determinato solo studiando i suoi input e gli output correlati.

7.3.1 Testing basato sui requisiti

Un principio generale di buona pratica nell'ingegneria dei requisiti è che i requisiti dovrebbero essere testabili. Questo significa che il requisito dovrebbe essere scritto in modo tale che sia possibile progettare un test per quel requisito.

Ad esempio, consideriamo i seguenti requisiti del sistema Mentcare riguardanti il controllo delle allergie ai farmaci:

- Se un paziente è noto per essere allergico a un particolare farmaco, la prescrizione di quel farmaco deve generare un messaggio di avviso per l'utente del sistema.
- Se chi prescrive sceglie di ignorare un avviso di allergia, deve fornire una motivazione per questa scelta.

Per verificare se questi requisiti sono stati soddisfatti, potrebbero essere necessari diversi test correlati:

1. Creare un record del paziente senza allergie note. Prescrivere farmaci per allergie note e verificare che il sistema non generi un messaggio di avviso.
2. Creare un record del paziente con un'allergia nota. Prescrivere il farmaco a cui il paziente è allergico e verificare che il sistema generi un avviso.

3. Creare un record del paziente in cui siano registrate allergie a due o più farmaci. Prescrivere separatamente entrambi i farmaci e verificare che il sistema generi correttamente l'avviso per ciascun farmaco.
4. Prescrivere due farmaci a cui il paziente è allergico. Verificare che il sistema emetta correttamente due avvisi.
5. Prescrivere un farmaco che genera un avviso e ignorare quell'avviso. Verificare che il sistema richieda all'utente di fornire una spiegazione del motivo per cui l'avviso è stato ignorato.

7.3.2 Testing basato su scenari

Il testing basato su scenari è un approccio al testing di rilascio in cui si creano scenari tipici di utilizzo e si usano questi scenari per sviluppare casi di test per il sistema. Uno scenario è una storia realistica e facilmente comprensibile che descrive un modo in cui il sistema potrebbe essere utilizzato.

Esempio 7.3.1 – Scenario per il sistema Mentcare

George è un infermiere specializzato in assistenza sanitaria mentale. Una delle sue responsabilità è visitare i pazienti a domicilio per verificare che il loro trattamento sia efficace e che non soffrano di effetti collaterali dovuti ai farmaci.

In una giornata dedicata alle visite domiciliari, George accede al sistema Mentcare e lo utilizza per stampare il programma delle visite domiciliari previste per quella giornata, insieme a informazioni riepilogative sui pazienti da visitare. Richiede che i record di questi pazienti vengano scaricati sul suo laptop e gli viene richiesto di inserire la sua frase chiave per criptare i record sul laptop.

Uno dei pazienti che visita è Jim, che viene trattato con farmaci per la depressione. Jim ritiene che la terapia lo stia aiutando, ma crede che gli causi insonnia. George consulta il record di Jim e viene invitato a inserire la sua frase chiave per decrittare il record. Controlla il farmaco prescritto e verifica i suoi effetti collaterali. L'insonnia è un effetto collaterale noto, quindi annota il problema nel record di Jim e gli suggerisce di recarsi in clinica per cambiare farmaco. Jim è d'accordo, così George inserisce un promemoria per contattarlo al suo ritorno in clinica e fissare un appuntamento con un medico. George conclude la consultazione e il sistema recripta il record di Jim.

Dopo aver terminato le consultazioni, George torna in clinica e carica i record dei pazienti visitati nel database. Il sistema genera una lista di chiamate per George, che include i pazienti da contattare per raccogliere informazioni di follow-up e per fissare appuntamenti in clinica.

Questo scenario testa diverse funzionalità del sistema Mentcare:

1. Autenticazione tramite accesso al sistema.
2. Scaricamento e caricamento dei record dei pazienti su un laptop.
3. Pianificazione delle visite a domicilio.
4. Crittografia e decrittografia dei record dei pazienti su un dispositivo mobile.
5. Recupero e modifica dei record.
6. Collegamenti con il database dei farmaci che mantiene le informazioni sugli effetti collaterali.
7. Sistema di promemoria per le chiamate.

7.3.3 Testing delle prestazioni

Una volta che un sistema è stato completamente integrato, è possibile testare proprietà emergenti, come prestazioni e affidabilità. I test delle prestazioni devono essere progettati per garantire che il sistema possa gestire il carico previsto. Ciò di solito comporta una serie di test in cui si aumenta il carico fino a quando le prestazioni del sistema diventano inaccettabili.

Lo stress testing permette di testare il comportamento del sistema in caso di guasto e rivelare difetti che si manifestano solo quando il sistema è completamente carico.

7.4 Testing dell'utente

Il test dell'utente o del cliente è una fase del processo di testing in cui gli utenti o i clienti forniscono feedback e consigli sui test del sistema. Il test dell'utente è essenziale, anche quando sono stati effettuati test di sistema e di rilascio completi, poiché l'ambiente di lavoro dell'utente può influire notevolmente su affidabilità, prestazioni, usabilità e robustezza del sistema.

È praticamente impossibile per uno sviluppatore replicare l'ambiente di lavoro del sistema. Ad esempio, un sistema destinato all'uso in un ospedale opera in un ambiente clinico dove avvengono altre attività, come emergenze dei pazienti e conversazioni con i familiari. Questi fattori influenzano l'uso del sistema, ma gli sviluppatori non possono includerli nel loro ambiente di test.

Esistono tre diversi tipi di test dell'utente:

1. *Test alpha*, in cui un gruppo selezionato di utenti collabora strettamente con il team di sviluppo per testare le prime versioni del software.
2. *Test beta*, in cui una versione del software viene resa disponibile a un gruppo più ampio di utenti per permettere loro di sperimentare e segnalare eventuali problemi ai sviluppatori del sistema.
3. *Test di accettazione*, in cui i clienti testano un sistema per decidere se accettarlo o meno dai sviluppatori e implementarlo nel loro ambiente.

Le fasi nel processo di test di accettazione sono sei (Figura 7.9):

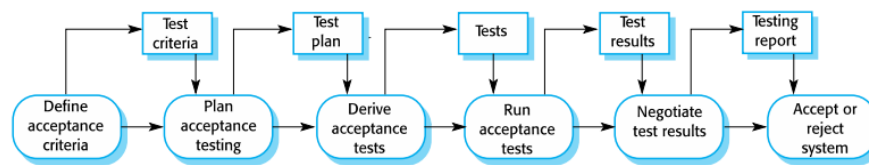


Figura 7.9: Il processo di test di accettazione

1. *Definizione dei criteri di accettazione*: questa fase dovrebbe idealmente svolgersi all'inizio, prima della firma del contratto per il sistema.
2. *Pianificazione del test di accettazione*: include risorse, tempo e budget, e stabilisce un programma di test.
3. *Progettazione dei test di accettazione*: i test sono progettati per verificare che il sistema sia accettabile, sia per le caratteristiche funzionali che non funzionali.
4. *Esecuzione dei test di accettazione*: i test vengono eseguiti sul sistema, idealmente nell'ambiente reale in cui verrà utilizzato.
5. *Negoziiazione dei risultati dei test*: poiché difficilmente tutti i test di accettazione saranno superati senza problemi, cliente e sviluppatore devono negoziare per decidere se il sistema è accettabile.
6. *Rifiuto/accettazione del sistema*: incontro tra sviluppatori e cliente per decidere se accettare il sistema o richiedere ulteriori sviluppi.

Nei metodi agili potrebbe non esserci una fase separata per il test di accettazione. L'utente finale è parte del team di sviluppo e fornisce i requisiti del sistema in termini di user stories, definendo anche i test per verificare se il software sviluppato le supporta. Questi test sono equivalenti ai test di accettazione e vengono automatizzati; lo sviluppo non procede fino a quando i test di accettazione delle user stories non sono stati superati.

Tuttavia, trovare utenti "tipici" può essere difficile, e i test di accettazione possono non riflettere appieno come un sistema venga effettivamente utilizzato.

7.5 Punti chiave

- Il testing può solo mostrare la presenza di errori in un programma. Non può dimostrare che non ci siano più difetti.
- Il testing durante lo sviluppo è responsabilità del team di sviluppo software. Un team separato dovrebbe essere incaricato di testare un sistema prima che venga rilasciato ai clienti.
- Il testing durante lo sviluppo include: il testing unitario, in cui vengono testati singoli oggetti e metodi; il testing dei componenti, in cui vengono testati gruppi correlati di oggetti; il testing del sistema, in cui vengono testati sistemi parziali o completi.
- Quando si testa il software, bisogna cercare di "romperlo" usando l'esperienza e le linee guida per scegliere tipi di casi di test che si sono dimostrati efficaci nello scoprire difetti in altri sistemi.
- Quando possibile, è preferibile scrivere test automatizzati. I test vengono integrati in un programma che può essere eseguito ogni volta che viene apportata una modifica a un sistema.
- Sviluppo orientato ai test è un approccio di sviluppo in cui i test vengono scritti prima del codice da testare.
- Il testing basato su scenari prevede l'invenzione di uno scenario di utilizzo tipico e l'uso di questo scenario per derivare i casi di test.
- Il testing di accettazione è un processo di testing condotto dall'utente, con l'obiettivo di stabilire se il software è sufficientemente buono per essere implementato e utilizzato nel suo ambiente operativo.

Capitolo 8

Pianificazione del progetto

La **pianificazione del progetto** è uno dei compiti più importanti di un project manager nel settore software. Come manager, devi suddividere il lavoro in parti, assegnarle ai membri del team, anticipare i problemi che potrebbero sorgere e preparare soluzioni provvisorie per affrontarli. Il piano di progetto, creato all'inizio del progetto e aggiornato man mano che il progetto progredisce, serve a mostrare come verrà svolto il lavoro e a valutare l'andamento del progetto.

La pianificazione avviene in tre fasi principali durante il ciclo di vita di un progetto:

1. *Fase di proposta.* Durante la gara d'appalto per un contratto, è necessario elaborare un piano per decidere se si hanno le risorse per completare il lavoro e calcolare il prezzo da proporre al cliente.
2. *Fase di avvio del progetto.* Una volta ottenuto il contratto, si pianifica chi lavorerà al progetto, come suddividere il lavoro in incrementi e come allocare le risorse.
3. *Aggiornamenti periodici.* Durante il progetto, il piano viene aggiornato per riflettere le nuove informazioni acquisite sul software, il processo di sviluppo e il team di sviluppo.

Nella fase di proposta, la pianificazione è inevitabilmente speculativa, poiché non si dispone di un set completo di requisiti per il software da sviluppare. Si risponde a una richiesta basata su una descrizione ad alto livello delle funzionalità richieste. Un piano credibile è spesso richiesto come parte della proposta. Se il contratto viene assegnato, si deve ripianificare il progetto considerando eventuali cambiamenti e nuove informazioni emerse.

Durante la preparazione della proposta, è necessario calcolare il prezzo da proporre al cliente. Questo si basa sulla stima dei costi per completare il lavoro, considerando i seguenti parametri principali: costi di sforzo, costi hardware e software, costi di viaggio e formazione, costi indiretti...

Dopo l'assegnazione del contratto, il piano preliminare deve essere affinato per creare un piano di avvio dettagliato. Questo aiuta a prendere decisioni su budget e personale, allocare risorse e monitorare l'avanzamento del progetto. Il piano di avvio dovrebbe anche definire i meccanismi di monitoraggio del progetto. Anche nei metodi agili è necessario un piano iniziale per allocare risorse e stabilire le basi del progetto.

Il piano di progetto dovrebbe essere regolarmente aggiornato man mano che il progetto avanza e si acquisiscono maggiori informazioni sul software e sul suo sviluppo. Il programma del progetto, la stima dei costi e i rischi devono essere revisionati regolarmente.

8.1 Prezzi del software

In linea di principio, il prezzo di un sistema software sviluppato per un cliente corrisponde semplicemente al costo dello sviluppo più il profitto per il fornitore. Tuttavia, in pratica, la relazione tra il costo del progetto e il prezzo proposto al cliente non è quasi mai così semplice. Nel calcolo di un prezzo, si devono considerare fattori organizzativi, economici, politici e commerciali più ampi (Tabella 8.1). Questi aspetti possono portare ad un adeguamento del prezzo verso l'alto o verso il basso.

Fattore	Descrizione
Termini contrattuali	Un cliente potrebbe consentire allo sviluppatore di mantenere la proprietà del codice sorgente e riutilizzarlo in altri progetti. In questo caso, il prezzo richiesto potrebbe essere inferiore rispetto alla consegna del codice sorgente al cliente.
Incertezza nella stima dei costi	Se un'organizzazione non è sicura della stima dei costi, potrebbe aumentare il prezzo aggiungendo una percentuale di contingenza oltre al normale margine di profitto.
Salute finanziaria	Gli sviluppatori in difficoltà finanziarie potrebbero abbassare il prezzo per ottenere un contratto. È preferibile ottenere un profitto inferiore al normale o andare in pareggio piuttosto che uscire dal mercato. In tempi difficili, il flusso di cassa è più importante del profitto.
Opportunità di mercato	Un'organizzazione di sviluppo potrebbe offrire un prezzo basso per entrare in un nuovo segmento di mercato software. Accettare un basso profitto su un progetto può dare l'opportunità di ottenere profitti maggiori in futuro. L'esperienza acquisita potrebbe anche aiutare a sviluppare nuovi prodotti.
Volatilità dei requisiti	Se i requisiti potrebbero cambiare, un'organizzazione può abbassare il prezzo per aggiudicarsi il contratto. Dopo l'assegnazione, possono essere applicati prezzi più alti per le modifiche ai requisiti.

Tabella 8.1: Fattori che influenzano il prezzo del software

Esistono due tipologie di strategie di prezzo:

1. *Prezzo al ribasso.* Un'azienda può offrire un prezzo inferiore per aggiudicarsi un contratto che consenta di mantenere il personale per opportunità future. Un'azienda può offrire un prezzo inferiore per accedere a un nuovo segmento di mercato.
2. *Prezzo aumentato.* Il prezzo può essere aumentato quando l'acquirente richiede un contratto a prezzo fisso, e il venditore incrementa il prezzo per coprire eventuali rischi imprevisti.

Il pricing to win implica che un'azienda abbia un'idea del prezzo che il cliente si aspetta di pagare e prepari un'offerta per il contratto basata su tale aspettativa. Un costo di progetto viene concordato sulla base di una proposta preliminare. Successivamente, si avviano negoziazioni tra cliente e fornitore per definire le specifiche dettagliate del progetto.

8.2 Sviluppo plan-driven

Lo sviluppo basato su piani (plan-driven) è un approccio all'ingegneria del software in cui il processo di sviluppo viene pianificato in dettaglio. Viene creato un piano di progetto che registra il lavoro da svolgere, chi lo eseguirà, il programma di sviluppo e i prodotti finali. I responsabili utilizzano il piano per supportare le decisioni relative al progetto e per misurare i progressi. Lo sviluppo plan-driven si basa su tecniche di gestione dei progetti ingegneristici ed è considerato il metodo "tradizionale" per gestire grandi progetti di sviluppo software. Lo sviluppo agile, al contrario, utilizza un processo di pianificazione diverso, in cui le decisioni vengono posticipate.

Il problema dello sviluppo plan-driven è che le decisioni prese in anticipo devono spesso essere riviste a causa di cambiamenti negli ambienti in cui il software viene sviluppato e utilizzato. Ritardare le decisioni di pianificazione può evitare inutili rifacimenti. Tuttavia, i vantaggi di un approccio plan-driven sono che una pianificazione anticipata consente di tenere conto di questioni organizzative (disponibilità del personale, altri progetti, ecc.). Problemi e dipendenze potenziali vengono scoperti prima che il progetto inizi, piuttosto che quando il progetto è già avviato.

8.2.1 Piani di progetto

In un progetto di sviluppo plan-driven, il piano del progetto stabilisce le risorse disponibili, la suddivisione del lavoro e un programma per l'esecuzione delle attività. I dettagli dei piani di progetto variano in base al tipo di progetto e organizzazione, ma in genere includono le seguenti sezioni:

1. *Introduzione.* Descrive brevemente gli obiettivi del progetto e stabilisce i vincoli (ad esempio, budget, tempi) che influenzano la gestione del progetto.
2. *Organizzazione del progetto.* Descrive il modo in cui il team di sviluppo è organizzato, le persone coinvolte e i loro ruoli all'interno del team.
3. *Analisi dei rischi.* Descrive i possibili rischi del progetto, la probabilità che si verifichino e le strategie di riduzione del rischio che vengono proposte.
4. *Requisiti di risorse hardware e software.* Specifica l'hardware e il software di supporto necessari per eseguire lo sviluppo. Se è necessario acquistare hardware, possono essere incluse stime sui prezzi e sul programma di consegna.
5. *Suddivisione del lavoro.* Definisce la suddivisione del progetto in attività e identifica gli input e gli output di ciascuna attività del progetto.
6. *Pianificazione del progetto.* Mostra le dipendenze tra le attività, il tempo stimato per raggiungere ogni traguardo e l'assegnazione del personale alle attività.
7. *Meccanismi di monitoraggio e reporting.* Definisce i report di gestione da produrre, quando devono essere prodotti e i meccanismi di monitoraggio del progetto da utilizzare.

Il piano principale del progetto dovrebbe sempre includere una valutazione dei rischi e un programma del progetto. Inoltre, potrebbero essere sviluppati piani supplementari per attività specifiche (Tabella 8.2).

Piano	Descrizione
Piano di gestione delle configurazioni	Descrive le procedure e le strutture di gestione delle configurazioni che saranno utilizzate.
Piano di distribuzione	Descrive come il software e l'hardware associato (se richiesto) saranno distribuiti nell'ambiente del cliente. Questo dovrebbe includere un piano per la migrazione dei dati dai sistemi esistenti.
Piano di manutenzione	Prevede i requisiti, i costi e l'impegno necessari per la manutenzione.
Piano di qualità	Descrive le procedure e gli standard di qualità che saranno utilizzati in un progetto.
Piano di validazione	Descrive l'approccio, le risorse e il programma utilizzati per la validazione del sistema.

Tabella 8.2: Supplementi al piano di progetto

8.2.2 Il processo di pianificazione

La pianificazione del progetto è un processo iterativo che inizia con la creazione di un piano iniziale durante la fase di avvio del progetto.

Le modifiche al piano sono inevitabili. Man mano che si ottengono maggiori informazioni sul sistema e sul team di progetto, è necessario rivedere regolarmente il piano per riflettere i cambiamenti nei requisiti, nella pianificazione e nei rischi. Anche la modifica degli obiettivi aziendali comporta modifiche nei piani di progetto. Man mano che gli obiettivi aziendali cambiano, ciò potrebbe influire su tutti i progetti, che potrebbero quindi dover essere ripianificati.

All'inizio del processo di pianificazione, si dovrebbero valutare i vincoli che influenzano il progetto, come la data di consegna richiesta, il personale disponibile, il budget complessivo e gli strumenti disponibili. Bisogna anche identificare i traguardi e i prodotti da consegnare.

Il processo entra quindi in un ciclo che termina solo quando il progetto è completato. Durante questo ciclo, si redige un programma stimato, si avviano le attività definite e si rivedono regolarmente i progressi.

Se emergono seri problemi che potrebbero causare ritardi significativi, è necessario adottare azioni di mitigazione del rischio per ridurre il rischio di fallimento del progetto. Se queste azioni non sono efficaci, è opportuno avviare una revisione tecnica formale del progetto, che può anche portare alla decisione di annullare il progetto se non è più allineato agli obiettivi aziendali.

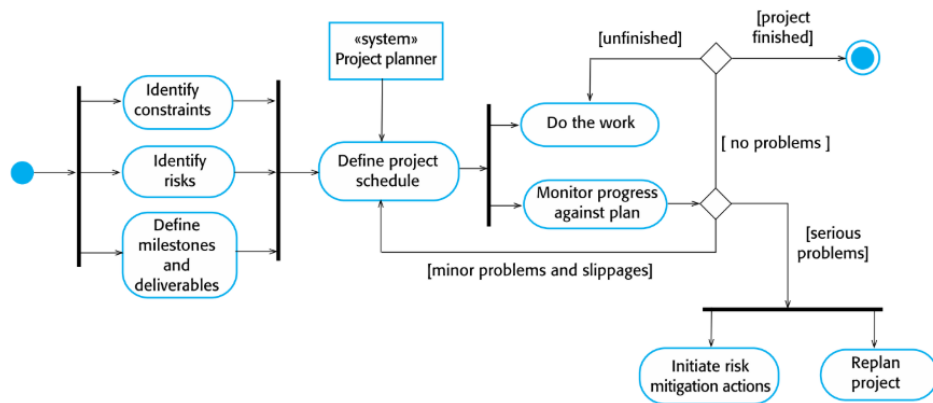


Figura 8.1: Il processo di pianificazione del progetto

È importante fare ipotesi realistiche piuttosto che ottimistiche durante la definizione di un piano di progetto. Durante un progetto sorgono sempre problemi di qualche tipo, che portano a ritardi nello sviluppo. Pertanto, le ipotesi iniziali e la pianificazione devono tenere conto di problemi imprevedibili. Bisogna includere una contingenza nel piano per evitare che eventuali problemi compromettano gravemente la scadenza di consegna.

Se si verificano problemi significativi nello sviluppo che potrebbero causare ritardi considerevoli, è necessario avviare azioni di mitigazione del rischio per ridurre i rischi di fallimento del progetto. In parallelo a queste azioni, bisogna anche ripianificare il progetto. Questo potrebbe comportare la rinegoziazione dei vincoli e delle consegne del progetto con il cliente. Bisogna stabilire e concordare con il cliente un nuovo programma per il completamento delle attività.

8.3 Pianificazione del progetto

La pianificazione del progetto è il processo di definizione di come il lavoro in un progetto sarà organizzato in compiti separati, e quando e come questi compiti saranno eseguiti. Devi stimare il tempo necessario per completare ogni compito e l'impegno richiesto, suggerendo chi lavorerà sui compiti identificati. Devi anche stimare le risorse hardware e software necessarie per completare ciascun compito. Ad esempio, se stai sviluppando un sistema embedded, devi stimare il tempo necessario per utilizzare hardware specializzato e i costi per eseguire un simulatore di sistema.

Sia i processi basati su piano che quelli agili richiedono una pianificazione iniziale del progetto. Questa pianificazione iniziale viene utilizzata per pianificare come le persone saranno allocate ai progetti e per verificare i progressi del progetto rispetto agli impegni contrattuali.

La pianificazione nei progetti basati su piano (Figura 8.2) implica suddividere il lavoro totale di un progetto in compiti separati e stimare il tempo necessario per completare ciascun compito. I compiti dovrebbero normalmente durare almeno una settimana e non più di 2 mesi. Una suddivisione più fine comporta un dispendio di tempo sproporzionato per la ripianificazione e l'aggiornamento del piano di progetto. Il tempo massimo per ciascun compito dovrebbe essere di 6-8 settimane. Se un compito richiede più tempo, dovrebbe essere suddiviso in sottocompiti per la pianificazione e la schedulazione del progetto.

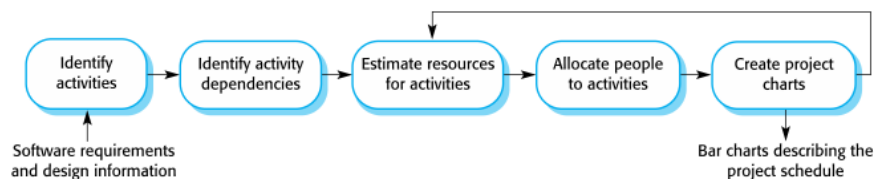


Figura 8.2: Il processo di pianificazione del progetto

Alcuni di questi compiti vengono eseguiti in parallelo, con persone diverse che lavorano su componenti differenti del sistema. Devi coordinare questi compiti paralleli e organizzare il lavoro in

modo che la forza lavoro venga utilizzata in modo ottimale e che non vengano introdotte dipendenze inutili tra i compiti. È importante evitare una situazione in cui l'intero progetto venga ritardato a causa del fatto che un compito critico non è stato completato.

Se un progetto è tecnicamente avanzato, le stime iniziali saranno quasi certamente ottimistiche anche quando si cercherà di considerare tutte le eventualità. Le pianificazioni, quindi, devono essere continuamente aggiornate man mano che diventano disponibili informazioni più precise sui progressi.

Stimare la difficoltà dei problemi e quindi il costo per sviluppare una soluzione è difficile. La produttività non è proporzionale al numero di persone che lavorano su un compito. Aggiungere persone a un progetto in ritardo lo rende ancora più in ritardo a causa dei costi di comunicazione. Quando si stima la pianificazione, è necessario tenere in considerazione la possibilità che qualcosa vada storto e prevedere una contingenza nella pianificazione.

8.3.1 Presentazione della pianificazione

Sono stati sviluppati dei grafici alternativi per visualizzare le pianificazioni del progetto, che sono spesso più facili da leggere e comprendere. Due tipi di visualizzazione sono comunemente utilizzati:

1. Grafici a barre basati sul calendario mostrano chi è responsabile di ciascuna attività, il tempo previsto, e quando l'attività è programmata per iniziare e finire.
2. Reti di attività mostrano le dipendenze tra le diverse attività che compongono un progetto.

Le attività di progetto sono l'elemento base della pianificazione. Ogni attività ha:

- una durata in giorni o mesi di calendario;
- una stima dell'impegno, che mostra il numero di giorni persona o mesi persona necessari per completare il lavoro;
- una scadenza entro la quale l'attività deve essere completata;
- un punto finale definito, che potrebbe essere un documento, la tenuta di una riunione di revisione, l'esecuzione con successo di tutti i test, e così via.

Quando si pianifica un progetto, si potrebbe decidere di definire delle tappe (milestones). Una tappa è una fine logica di una fase del progetto, dove il progresso del lavoro può essere rivisto. Le tappe possono essere associate a un singolo compito o a gruppi di attività correlate.

Alcune attività creano deliverables di progetto, ossia risultati che vengono consegnati al cliente del software. Di solito, i deliverables richiesti sono specificati nel contratto di progetto, e la visione del cliente sul progresso del progetto dipende da questi deliverables.

Task	Effort (person-days)	Duration (days)	Dependencies
T1	15	10	
T2	8	15	
T3	20	15	T1 (M1)
T4	5	10	
T5	5	10	T2, T4 (M3)
T6	10	5	T1, T2 (M4)
T7	25	20	T1 (M1)
T8	75	25	T4 (M2)
T9	10	15	T3, T6 (M5)
T10	20	15	T7, T8 (M6)
T11	10	10	T9 (M7)
T12	20	10	T10, T11 (M8)

Tabella 8.3: Attività, durate e dipendenze

Da questa tabella, si può vedere che il compito T3 dipende dal compito T1. Ciò significa che il compito T1 deve essere completato prima che T3 inizi.

Si noti che la durata stimata di alcuni compiti è maggiore rispetto all'impegno richiesto e viceversa. Se l'impegno è inferiore alla durata, le persone allocate a quel compito non stanno lavorando a tempo pieno su di esso. Se l'impegno è maggiore della durata, significa che più membri del team stanno lavorando contemporaneamente sul compito.

La Figura 8.3 prende le informazioni della Tabella 8.3 e presenta la pianificazione del progetto come un grafico a barre che mostra il calendario del progetto e le date di inizio e fine dei compiti. Leggendo da sinistra a destra, il grafico a barre mostra chiaramente quando i compiti iniziano e finiscono. Le tappe ($M1$, $M2$, ecc.) sono anche mostrate nel grafico a barre. Si noti che i compiti che sono indipendenti possono essere eseguiti in parallelo. Ad esempio, i compiti $T1$, $T2$ e $T4$ iniziano all'inizio del progetto.

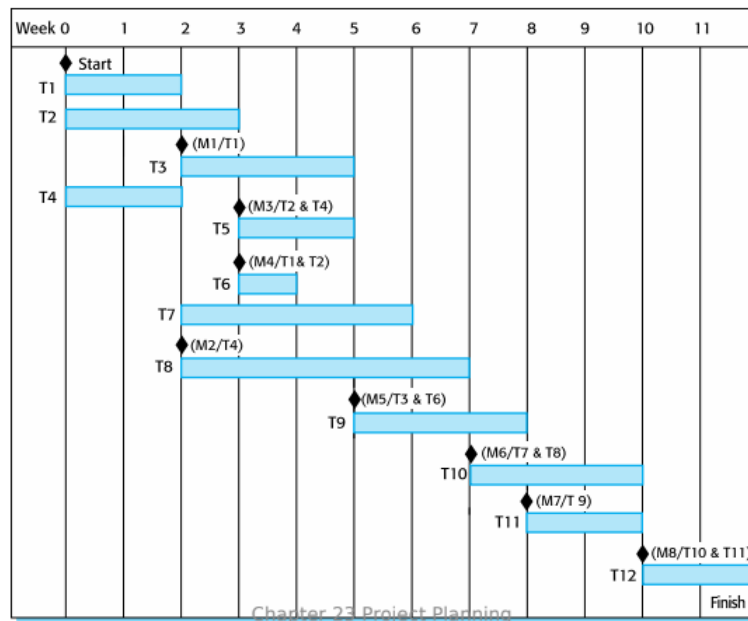


Figura 8.3: Grafico a barre delle attività

Oltre a pianificare la programmazione di consegna del software, i project manager devono allocare risorse ai compiti. La risorsa chiave è, ovviamente, l'ingegnere del software che eseguirà il lavoro. Devono essere assegnati alle attività di progetto. L'allocazione delle risorse può essere analizzata utilizzando strumenti di gestione del progetto, e un grafico a barre può essere generato per mostrare quando il personale sta lavorando sul progetto (Figura 8.4). Le persone possono lavorare su più di un compito contemporaneamente e, a volte, non lavorano sul progetto. Possono essere in ferie, lavorare su altri progetti o partecipare a corsi di formazione. Gli incarichi part-time vengono mostrati utilizzando una linea diagonale che attraversa la barra.

Se un compito subisce un ritardo, i compiti successivi che dipendono da esso possono essere influenzati. Non possono iniziare fino a quando il compito ritardato non viene completato.

8.4 Pianificazione agile

I metodi agili di sviluppo software sono approcci iterativi in cui il software viene sviluppato e consegnato ai clienti in incrementi. A differenza degli approcci plan-driven, la funzionalità di questi incrementi non viene pianificata in anticipo ma viene decisa durante lo sviluppo. La decisione su cosa includere in un incremento dipende dai progressi e dalle priorità del cliente. L'argomentazione a favore di questo approccio è che le priorità e i requisiti del cliente cambiano, quindi ha senso avere un piano flessibile che possa adattarsi a tali cambiamenti.

I metodi di sviluppo agile come Scrum e Extreme Programming adottano un approccio alla pianificazione in due fasi, corrispondenti alla fase di avvio e alla pianificazione dello sviluppo nei metodi tradizionali:

1. Pianificazione del rilascio, che guarda avanti per diversi mesi e decide quali funzionalità includere in un rilascio del sistema.

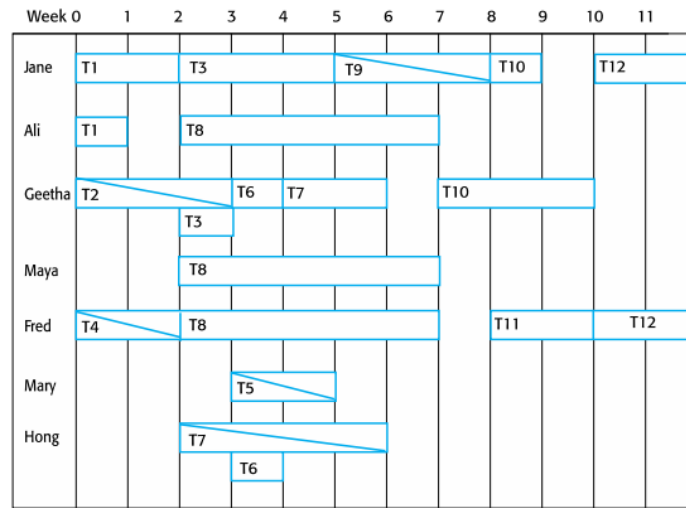


Figura 8.4: Tabella di allocazione del personale

2. Pianificazione dell'iterazione, che ha un orizzonte temporale più breve e si concentra sulla pianificazione del prossimo incremento del sistema, solitamente rappresentando 2-4 settimane di lavoro per il team.

L'approccio Scrum alla pianificazione, basato su backlog di progetto e revisioni quotidiane del lavoro da fare. Questo approccio è principalmente orientato alla pianificazione delle iterazioni. Un altro approccio alla pianificazione agile, sviluppato come parte di Extreme Programming, si basa sulle user stories. Il cosiddetto planning game può essere utilizzato sia per la pianificazione del rilascio sia per quella delle iterazioni.



Figura 8.5: Il planning game

La base del planning game (Figura 8.5) è un insieme di user stories che coprono tutte le funzionalità da includere nel sistema finale. Il team di sviluppo e il cliente lavorano insieme per sviluppare queste storie. I membri del team leggono e discutono le storie e le classificano in base al tempo necessario per implementarle. Alcune storie possono essere troppo grandi per essere implementate in una sola iterazione e vengono quindi suddivise in storie più piccole.

Poiché stimare lo sforzo o il tempo necessario può essere difficile, viene spesso utilizzata una classificazione relativa. Il team confronta le storie a coppie e decide quale richiederà più tempo ed effort, senza valutare esattamente quanto sarà necessario. Alla fine di questo processo, l'elenco delle storie è ordinato, con le storie più complesse in cima alla lista. Il team assegna quindi dei punti effort a tutte le storie: ad esempio, una storia complessa potrebbe ricevere 8 punti, una semplice 2 punti.

Una volta stimate le storie, l'effort relativo viene tradotto in una prima stima dello sforzo totale richiesto usando il concetto di velocità (velocity), ovvero il numero di punti effort implementati dal team al giorno. Questo dato può essere stimato dall'esperienza precedente o implementando una o due storie per verificare quanto tempo serve. Con una stima della velocità, si può calcolare lo sforzo totale in giorni/persona per implementare il sistema.

La pianificazione del rilascio implica la selezione e il perfezionamento delle storie che riflettono le funzionalità da implementare in un rilascio del sistema, nonché l'ordine di implementazione.

Le storie da implementare durante l'iterazione vengono scelte in base al tempo disponibile (solitamente 2-3 settimane) e alla velocità del team. Alla fine dell'iterazione, anche se non tutte le storie sono state implementate, l'incremento viene considerato completo. La velocità viene ricalcolata e utilizzata per pianificare la versione successiva.

All'inizio di ogni iterazione, c'è una fase di pianificazione dei compiti in cui gli sviluppatori suddividono le storie in task di sviluppo. Un task dovrebbe richiedere tra 4 e 16 ore. Gli sviluppatori

scelgono i task che implementeranno, sapendo la propria velocità individuale, così da non accettare più compiti di quelli che possono completare.

Questo approccio alla pianificazione ha due vantaggi principali:

1. Il team ha una panoramica dei task da completare, comprendendo chi sta facendo cosa e facilitando la collaborazione.
2. Gli sviluppatori scelgono i propri compiti, aumentando il senso di proprietà e la motivazione.

Un incremento di software viene sempre consegnato alla fine di ogni iterazione, e se le funzionalità non possono essere completate, il lavoro viene ridotto senza estendere la scadenza. Tuttavia, ciò può creare problemi ai clienti, poiché devono utilizzare un sistema incompleto o modificare i propri processi.

La pianificazione agile funziona bene con team piccoli e stabili. Tuttavia, nei grandi progetti, con team distribuiti geograficamente o in cui i membri cambiano spesso, è difficile attuare la collaborazione necessaria. In questi casi, si utilizzano approcci tradizionali alla gestione del progetto.

8.5 Tecniche di stima

Stimare i tempi di un progetto è un compito difficile. Ci sono così tante incertezze che è impossibile stimare accuratamente i costi di sviluppo del sistema nelle prime fasi di un progetto. Tuttavia, le organizzazioni devono comunque effettuare stime di sforzo e costo. Per farlo, possono essere utilizzate due tipi di tecniche:

1. *Tecniche basate sull'esperienza.* La stima delle future esigenze di sforzo si basa sull'esperienza del responsabile di progetto relativa a progetti passati e al dominio applicativo. Essenzialmente, il manager formula un giudizio informato sulle probabili necessità di sforzo.
2. *Modelli algoritmici di costo.* In questo approccio, si utilizza una formula per calcolare lo sforzo del progetto basandosi su stime di attributi del prodotto, come dimensioni, caratteristiche del processo e competenze del personale coinvolto.

In entrambi i casi, è necessario utilizzare il proprio giudizio per stimare direttamente lo sforzo oppure le caratteristiche del progetto e del prodotto. Nella fase iniziale di un progetto, queste stime hanno un ampio margine di errore. Se la stima iniziale dello sforzo richiesto è di x mesi di lavoro, il range effettivo potrebbe variare da $0,25x$ a $4x$ rispetto allo sforzo reale misurato al completamento del sistema. Durante la pianificazione dello sviluppo, le stime diventano progressivamente più accurate man mano che il progetto avanza (Figura 8.6).

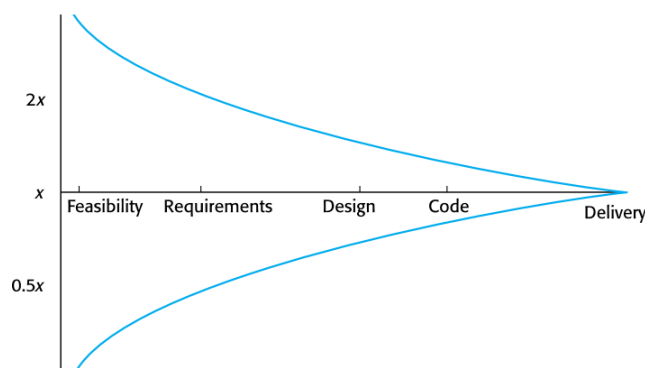


Figura 8.6: Stima dell'incertezza

8.5.1 Tecniche basate sull'esperienza

Le tecniche basate sull'esperienza si basano sull'esperienza del manager relativa a progetti passati e sullo sforzo effettivamente impiegato in tali progetti per attività correlate allo sviluppo software. Tipicamente, si identificano i deliverable da produrre in un progetto e i diversi componenti o sistemi software da sviluppare. Questi vengono documentati in un foglio di calcolo, stimati individualmente

e poi sommati per calcolare lo sforzo totale richiesto. Coinvolgere un gruppo di persone nell'attività di stima può essere utile: ogni membro del gruppo spiega la propria stima, rivelando spesso fattori che altri non avevano considerato, permettendo di iterare fino a ottenere una stima condivisa.

La difficoltà con le tecniche basate sull'esperienza è che un nuovo progetto software potrebbe non avere molto in comune con progetti precedenti. Lo sviluppo software cambia molto rapidamente, e un progetto potrebbe spesso utilizzare tecniche non familiari come i servizi web, la configurazione di sistemi applicativi o HTML5. Se non si ha esperienza con queste tecniche, l'esperienza passata potrebbe non aiutare a stimare lo sforzo richiesto, rendendo più difficile produrre stime accurate di costi e tempi.

8.5.2 Modelli algoritmici di costo

La modellazione algoritmica dei costi utilizza una formula matematica per prevedere i costi di progetto basandosi su stime delle dimensioni del progetto, del tipo di software da sviluppare e di altri fattori legati al team, al processo e al prodotto.

La maggior parte dei modelli algoritmici per stimare lo sforzo in un progetto software si basa su una formula semplice:

$$Effort = A \times Size^B \times M$$

A: un fattore costante che dipende dalle pratiche organizzative locali e dal tipo di software sviluppato. *Size*: una valutazione delle dimensioni del codice del software o una stima della funzionalità espressa in function points o application points. *B*: rappresenta la complessità del software e solitamente varia tra 1 e 1,5. *M*: un fattore che considera attributi di processo, prodotto e sviluppo, come i requisiti di affidabilità del software e l'esperienza del team di sviluppo.

Diversi fattori influenzano la dimensione finale: utilizzo di sistemi e componenti riutilizzati; linguaggio di programmazione; distribuzione del sistema. Man mano che il processo di sviluppo procede, la stima della dimensione diventa più accurata.

I modelli algoritmici dei costi rappresentano un modo sistematico per stimare lo sforzo necessario per sviluppare un sistema. Tuttavia, questi modelli sono complessi e difficili da utilizzare. Esistono molti attributi e un ampio margine di incertezza nella stima dei loro valori. Questa complessità limita l'applicazione pratica della modellazione algoritmica dei costi a un numero relativamente piccolo di grandi aziende, principalmente nei settori della difesa e dell'ingegneria aerospaziale.

8.6 Modellazione dei costi COCOMO

Questo modello empirico è stato sviluppato raccogliendo dati da un gran numero di progetti software di diverse dimensioni. I dati sono stati analizzati per scoprire le formule che meglio si adattavano alle osservazioni.

COCOMO II è un modello disponibile gratuitamente ed è supportato da strumenti open-source.

COCOMO II è stato sviluppato a partire dai precedenti modelli di stima dei costi COCOMO (Constructive Cost Modeling), basati principalmente sullo sviluppo di codice originale. Il modello COCOMO II tiene conto degli approcci moderni allo sviluppo software, come lo sviluppo rapido con linguaggi dinamici, lo sviluppo basato sul riuso e la programmazione di database. COCOMO II integra diversi sottomodelli basati su queste tecniche, che producono stime sempre più dettagliate.

I sottomodelli (Figura 8.7) che fanno parte del modello COCOMO II sono:

1. *Modello di composizione dell'applicazione*. Questo modello stima lo sforzo necessario per sviluppare sistemi creati da componenti riutilizzabili, scripting o programmazione di database. *Modello per la progettazione preliminare*. Utilizzato nelle fasi iniziali della progettazione del sistema, dopo che i requisiti sono stati stabiliti. *Modello di riuso*. Questo modello calcola lo sforzo necessario per integrare componenti riutilizzabili e/o codice generato automaticamente. *Modello post-architettura*. Una volta progettata l'architettura del sistema, è possibile effettuare una stima più accurata delle dimensioni del software.

8.6.1 Modello di composizione dell'applicazione

Il modello di composizione dell'applicazione è stato introdotto in COCOMO II per supportare la stima dello sforzo richiesto nei progetti di prototipazione e nei progetti in cui il software viene sviluppato componendo componenti esistenti. Si basa su una stima dei weighted application points (a volte

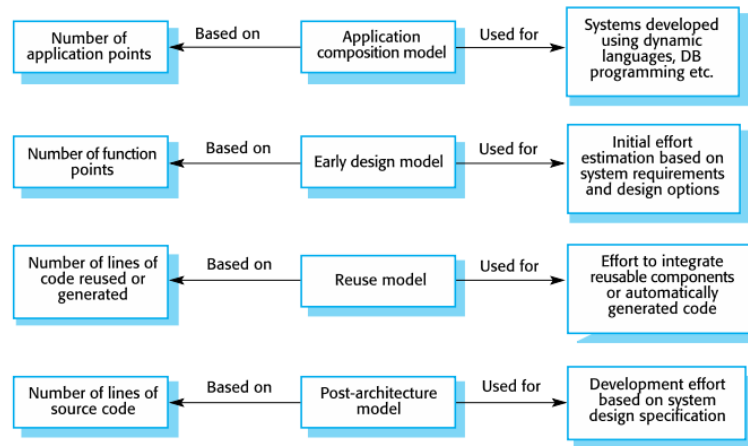


Figura 8.7: Modelli di stima COCOMO

chiamati object points), divisi per una stima standard della produttività in termini di application points.

La formula finale per calcolare lo sforzo richiesto per i prototipi di sistema è:

$$PM = (NAP \times (1 - \%reuse/100)) / PROD$$

Dove: PM stima dello sforzo in person-months (mesi/uomo); NAP è il numero totale di application points nel sistema consegnato; $\%reuse$ è la percentuale stimata di codice riutilizzato nello sviluppo; $PROD$ è la produttività degli application points, come mostrato nella Tabella 8.4.

Esperienza e capacità dello sviluppatore	Molto bassa	Bassa	Nominale	Alta	Molto alta
Maturità e capacità degli strumenti ICASE	Molto bassa	Bassa	Nominale	Alta	Molto alta
PROD (NAP/mese)	4	7	13	25	50

Tabella 8.4: Attività, durate e dipendenze

8.6.2 Modello di progettazione preliminare

Si presume che i requisiti degli utenti siano stati concordati e che le fasi iniziali del processo di progettazione del sistema siano in corso. L'obiettivo a questo stadio è fornire una stima approssimativa e rapida dei costi.

Le stime preliminari sono particolarmente utili per esplorare diverse opzioni, in modo da confrontare differenti modalità di implementazione dei requisiti dell'utente. Le stime prodotte in questa fase si basano sulla formula standard dei modelli algoritmici, ovvero:

$$PM = A \times Size^B \times M$$

$M = PERS \times RCPX \times RUSE \times PDIF \times PREX \times FCIL \times SCED$; A è pari a 2,94, la dimensione del sistema viene espressa in KSLOC (migliaia di linee di codice sorgente); l'esponente B riflette lo sforzo aggiuntivo necessario con l'aumentare della dimensione del progetto. Questo valore può variare da 1,1 a 1,24, a seconda della novità del progetto, della flessibilità dello sviluppo, dei processi di gestione dei rischi, della coesione del team di sviluppo e del livello di maturità dei processi dell'organizzazione.

Il moltiplicatore M si basa su sette attributi del progetto e del processo che possono aumentare o diminuire la stima.

- $PERS$: capacità del personale
- $PREX$: esperienza del personale
- $RCPX$: affidabilità e complessità del prodotto

- *RUSE*: riuso richiesto
- *PDIF*: difficoltà della piattaforma
- *SCED*: programmazione temporale
- *FSIL*: strutture di supporto

8.6.3 Modello di riuso

Il modello di riuso COCOMO viene utilizzato per stimare lo sforzo necessario per integrare codice riutilizzabile o generato automaticamente.

COCOMO II considera due tipi di codice riutilizzato:

1. *Codice a scatola nera (black-box)*: è codice riutilizzabile senza necessità di comprenderlo o modificarlo. Lo sforzo di sviluppo per il codice black-box è considerato pari a zero, e la sua dimensione non è inclusa nel calcolo complessivo dello sforzo.
2. *Codice a scatola bianca (white-box)*: è codice riutilizzabile che deve essere adattato per integrarsi con il nuovo codice o con altri componenti riutilizzati. Questo richiede uno sforzo aggiuntivo per comprendere e modificare il codice affinché funzioni correttamente nel sistema in sviluppo.

La formula per il codice generato è la seguente:

$$PM = (ASLOC \times AT/100) \times ATPROD$$

ASLOC: numero stimato di linee di codice nei componenti riutilizzati che devono essere modificate; *AT*: percentuale di codice riutilizzato modificabile automaticamente. *ATPROD*: produttività degli ingegneri nell'integrazione di questo codice.

Lo sforzo di sviluppo nel modello di riuso è calcolato utilizzando il modello di progettazione preliminare COCOMO ed è basato sul numero totale di linee di codice del sistema. La formula per calcolare l'equivalente delle linee di codice è:

$$ESLOC = ASLOC \times (1 - AT/100) \times AAM$$

ESLOC: numero equivalente di nuove linee di codice sorgente; *AAM*: moltiplicatore di adattamento, che riflette lo sforzo aggiuntivo richiesto per riutilizzare i componenti.

8.6.4 Livello post-architetturale

Il modello post-architetturale è il più dettagliato dei modelli COCOMO II. Viene utilizzato quando si dispone di un design architettonico iniziale per il sistema. Il punto di partenza per le stime prodotte a questo livello è la stessa formula base utilizzata per le stime di progettazione preliminare.

A questo stadio del processo, è possibile fare una stima più accurata della dimensione del progetto, poiché si sa come il sistema sarà scomposto in sottosistemi e componenti. La stima complessiva delle dimensioni del codice si ottiene sommando tre componenti:

1. Una stima del numero totale di nuove linee di codice da sviluppare.
2. Una stima dei costi di riutilizzo basata su un numero equivalente di linee di codice sorgente, calcolata utilizzando il modello di riuso.
3. Una stima del numero di linee di codice che potrebbero essere modificate a causa di cambiamenti nei requisiti del sistema.

L'esponente *B* nella formula per il calcolo dello sforzo è legato ai livelli di complessità del progetto. *B* si basa su cinque fattori, come mostrato nella Tabella 8.5.

Questi fattori sono valutati su una scala a sei punti da 0 a 5, dove 0 significa "estremamente alto" e 5 significa "molto basso". Per calcolare *B*, si sommano i punteggi, si dividono per 100 e si aggiunge il risultato a 1,01. Questo fornisce l'esponente da utilizzare.

Esempio: Supponiamo che un'organizzazione intraprenda un progetto in un dominio nel quale ha poca esperienza. Il cliente del progetto non ha definito il processo da utilizzare né ha previsto tempo per un'analisi significativa dei rischi. Un nuovo team di sviluppo deve essere formato per

Fattore di scala	Descrizione
Risoluzione architetturale/rischi	Riflette l'entità dell'analisi dei rischi effettuata. Molto bassa significa poca analisi; estremamente alta significa un'analisi completa e approfondita.
Flessibilità dello sviluppo	Riflette il grado di flessibilità nel processo di sviluppo. Molto bassa significa che si utilizza un processo prescritto; estremamente alta significa che il cliente definisce solo obiettivi generali.
Esperienza pregressa	Riflette l'esperienza precedente dell'organizzazione con questo tipo di progetto. Molto bassa significa nessuna esperienza precedente; estremamente alta significa che l'organizzazione è completamente familiare con questo dominio applicativo.
Maturità del processo	Riflette la maturità del processo dell'organizzazione. Il valore si calcola in base al CMM Maturity Questionnaire, ma si può stimare sottraendo il livello di maturità del processo CMM da 5.
Coesione del team	Riflette quanto bene il team di sviluppo si conosce e lavora insieme. Molto bassa significa interazioni molto difficili; estremamente alta significa un team integrato ed efficace senza problemi di comunicazione.

Tabella 8.5: Fattori di scala utilizzati nel calcolo dell'esponente nel modello post-architettura

implementare il sistema. L'organizzazione ha recentemente avviato un programma di miglioramento dei processi ed è stata valutata come un'organizzazione di Livello 2 secondo l'assessment di capacità SEI.

Le caratteristiche del progetto portano a stime dei punteggi per il calcolo di B come segue:

1. *Precedenza*: valutato come basso (4). È un progetto nuovo per l'organizzazione.
2. *Flessibilità dello sviluppo*: valutata come molto alta (1). Non vi è coinvolgimento del cliente nel processo di sviluppo, quindi ci sono pochi vincoli esterni.
3. *Risoluzione dei rischi architetturali*: valutata come molto bassa (5). Non è stata effettuata alcuna analisi dei rischi.
4. *Coesione del team*: valutata come nominale (3). È un team nuovo, quindi non ci sono informazioni disponibili sulla coesione.
5. *Maturità del processo*: valutata come nominale (3). Sono in atto alcune forme di controllo del processo.

La somma di questi valori è 16. Si calcola quindi il valore finale dell'esponente dividendo questa somma per 100 e aggiungendo 0,01. Il valore aggiustato di B è quindi 1,17.

La stima complessiva dello sforzo viene affinata utilizzando un insieme esteso di 17 attributi del prodotto, del processo e dell'organizzazione, invece dei sette attributi utilizzati nel modello di progettazione preliminare.

La Tabella 8.6 mostra come gli attributi dei cost driver influenzano le stime dello sforzo.

Valore dell'esponente	1,17
Dimensione del sistema (inclusi fattori per riuso e volatilità dei requisiti)	128.000 DSI
Stima iniziale COCOMO (senza driver di costo)	730 mesi-persona
Affidabilità	Molto alta, moltiplicatore = 1,39
Complessità	Molto alta, moltiplicatore = 1,3
Vincolo di memoria	Alta, moltiplicatore = 1,21
Utilizzo degli strumenti	Basso, moltiplicatore = 1,12
Pianificazione	Accelerata, moltiplicatore = 1,29
Stima COCOMO aggiustata	2.306 mesi-persona

Tabella 8.6: L'effetto dei fattori di costo sulle stime dello sforzo

8.6.5 Durata del progetto e gestione del personale

Oltre a stimare i costi complessivi di un progetto e lo sforzo necessario per sviluppare un sistema software, i project manager devono anche calcolare quanto tempo richiederà lo sviluppo del software e quando sarà necessario il personale per lavorare al progetto.

Il modello COCOMO include una formula per stimare il tempo complessivo (in mesi di calendario) necessario per completare un progetto:

$$TDEV = 3 \times (PM)^{(0.33+0.2 \times (B-1.01))}$$

TDEV: durata nominale del progetto, in mesi di calendario, ignorando eventuali moltiplicatori legati alla pianificazione; *PM*: lo sforzo calcolato dal modello COCOMO (in mesi-persona); *B*: un esponente legato alla complessità, come già discusso.

La pianificazione nominale del progetto, come prevista dal modello COCOMO, non corrisponde necessariamente alla pianificazione richiesta dal cliente.

Esiste una relazione complessa tra il numero di persone che lavorano a un progetto, lo sforzo dedicato al progetto e la durata della consegna. Ad esempio, se quattro persone possono completare un progetto in 13 mesi (cioè 52 mesi-persona di sforzo), si potrebbe pensare che aggiungendo una persona in più, si possa completare il lavoro in 11 mesi (55 mesi-persona di sforzo). Tuttavia, il modello COCOMO suggerisce che saranno necessarie sei persone per completare il lavoro in 11 mesi (66 mesi-persona di sforzo).

La ragione di ciò è che aggiungere persone a un progetto riduce la produttività dei membri del team esistenti. Man mano che il team aumenta di dimensioni, i membri del team trascorrono più tempo a comunicare e a definire le interfacce tra le parti del sistema sviluppate da altre persone. Raddoppiare il numero di personale, quindi, non significa dimezzare la durata del progetto.

Di conseguenza, quando si aggiunge una persona in più, l'incremento effettivo dello sforzo aggiunto è inferiore a quello di una persona intera, poiché gli altri membri del team diventano meno produttivi.

8.7 Punti chiave

- Il prezzo di un sistema non dipende solo dai costi stimati di sviluppo e dal profitto richiesto dall'azienda sviluppatrice. Fattori organizzativi possono portare ad un aumento del prezzo per compensare il rischio maggiore o ad una riduzione per ottenere un vantaggio competitivo.
- Spesso il software viene prezzato per ottenere un contratto, e la funzionalità del sistema viene poi adattata per rispettare il prezzo stimato.
- Lo sviluppo basato su un piano è organizzato attorno a un progetto completo che definisce le attività del progetto, lo sforzo pianificato, il programma delle attività e i responsabili di ciascuna attività.
- La pianificazione del progetto comporta la creazione di diverse rappresentazioni grafiche di parti del piano. I diagrammi a barre, che mostrano la durata delle attività e le tempistiche del personale, sono le rappresentazioni di pianificazione più comunemente utilizzate.
- Una milestone di progetto è un risultato prevedibile di un'attività o di un insieme di attività. A ogni milestone, deve essere presentato un rapporto formale sul progresso alla direzione. Un deliverable è un prodotto di lavoro consegnato al cliente del progetto.
- Il gioco di pianificazione agile coinvolge l'intero team nella pianificazione del progetto. Il piano viene sviluppato in modo incrementale e, se si presentano problemi, viene aggiustato riducendo la funzionalità del software invece di ritardare la consegna di un incremento.
- Le tecniche di stima per il software possono essere basate sull'esperienza, dove i manager valutano lo sforzo necessario, oppure algoritmiche, dove lo sforzo richiesto viene calcolato a partire da altri parametri stimati del progetto.
- Il modello di costo COCOMO II è un modello algoritmico maturo che considera attributi legati al progetto, al prodotto, all'hardware e al personale per formulare una stima dei costi.

Capitolo 9

Altri argomenti

9.1 Discrete Time Markov Chains

Una Catena di Markov a Tempo Discreto (DTMC) è una struttura matematica utilizzata per modellare sistemi che evolvono in modo probabilistico, passo dopo passo, in tempi discreti. Formalmente, una DTMC è definita come una tupla (U, X, Y, p, g) , in cui:

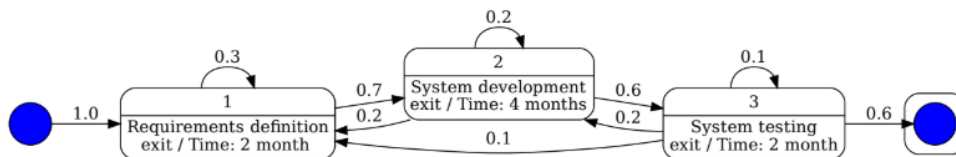
1. *Insieme di input (U)*: Può essere vuoto, finito o infinito. Rappresenta l'insieme degli ingressi che influenzano il sistema.
2. *Insieme di stati (X)*: Deve essere non vuoto e rappresenta tutte le configurazioni possibili del sistema.
3. *Insieme di output (Y)*: Deve anch'esso essere non vuoto e rappresenta i valori osservabili che il sistema può generare.
4. *Funzione di probabilità di transizione (p)*: È una funzione $p : X \times X \rightarrow [0, 1]$ che restituisce la probabilità che il sistema passi da uno stato $x \in X$ a uno stato $x' \in X'$ dato un input $u \in U$, ovvero $p(x'|x, u)$. La somma delle probabilità per ogni stato successivo deve essere pari a 1, ovvero $\int_{X'} p(x'|x, u) dx' = 1$.
5. *Funzione di output (g)*: È una funzione $g : X \rightarrow Y$ che associa ogni stato del sistema $x \in X$ a un valore di output $y \in Y$.

Per semplicità di notazione, si usa indicare i componenti con $M(U)$, $M(X)$, $M(Y)$ per riferirsi rispettivamente agli input, stati e output di una DTMC.

9.1.1 Esempi di utilizzo delle DTMC

Esempio 9.1.1 – Tempo di completamento

Consideriamo un processo software che attraversa tre fasi: definizione dei requisiti, sviluppo del sistema e test del sistema. Ogni fase ha una durata media e delle probabilità di transizione tra stati:



- *Definizione dei requisiti*: La fase iniziale ha una durata di 2 mesi. Se il processo non termina, ci sono due possibilità per proseguire: può andare direttamente alla fase di sviluppo o ripetere la fase di definizione dei requisiti. La probabilità di avanzare alla fase successiva è 0.7 e la probabilità di restare in questa fase è 0.3.
- *Sviluppo del sistema*: Questa fase ha una durata di 4 mesi. Le probabilità di avanzare o restare sono rispettivamente 0.6 e 0.2. Se il processo non termina in questa fase, può tornare alla fase precedente con probabilità 0.1.

- *Test del sistema:* Infine, il test del sistema ha una durata di 2 mesi. Le probabilità di passare alla fase finale, ovvero l'uscita, è 0.6. Se il processo non termina in questa fase, può tornare alle fasi precedenti o rimanere nella stessa fase con probabilità 0.2, 0.1 e 0.1.

L'obiettivo è calcolare la probabilità che il processo termini esattamente in 8 mesi. Il percorso possibile è: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$. La probabilità totale si ottiene moltiplicando le probabilità lungo questo percorso:

$$1.0 \times 0.7 \times 0.6 \times 0.6 = 0.252$$

Considerando lo stesso processo software, immaginiamo ora di voler calcolare la probabilità che termini in esattamente 10 mesi. Per calcolare questa probabilità, dobbiamo considerare i percorsi possibili che portano a una durata totale di 10 mesi. I percorsi sono:

Percorso1 : $0 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Percorso2 : $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3 \rightarrow 4$

La probabilità per ciascun percorso viene calcolata moltiplicando le probabilità di transizione lungo il percorso:

$$\begin{aligned} &1.0 \times 0.3 \times 0.7 \times 0.6 \times 0.6 + \\ &1.0 \times 0.7 \times 0.6 \times 0.1 \times 0.6 = \\ &0.07560 + 0.02520 = \\ &0.1008 \end{aligned}$$

Quindi, la probabilità che il processo termini esattamente in 10 mesi è 0.1008.

Immaginiamo ora di voler calcolare la probabilità che il processo software descritto in precedenza termini in esattamente 12 mesi. Come nel caso precedente, dobbiamo considerare tutti i percorsi che portano a una durata totale di 12 mesi.

Percorso1 : $0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Percorso2 : $0 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3 \rightarrow 4$

Percorso3 : $0 \rightarrow 1 \rightarrow 2 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Percorso4 : $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 4$

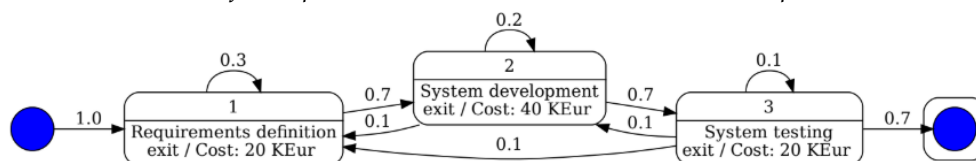
Le probabilità totali vengono calcolate come segue:

$$\begin{aligned} &1.0 \times 0.3 \times 0.3 \times 0.7 \times 0.6 \times 0.6 + \\ &1.0 \times 0.3 \times 0.7 \times 0.6 \times 0.1 \times 0.6 + \\ &1.0 \times 0.7 \times 0.2 \times 0.6 \times 0.6 + \\ &1.0 \times 0.7 \times 0.6 \times 0.1 \times 0.1 \times 0.6 = \\ &0.022680 + 0.007560 + 0.05040 + 0.002520 = \\ &0.08316 \end{aligned}$$

Pertanto, la probabilità che il processo termini esattamente in 12 mesi è 0.08316.

Esempio 9.1.2 – Costo di completamento

In questo esempio consideriamo un processo software composto da tre fasi principali, ciascuna caratterizzata da un costo fisso e probabilità di transizione che determinano il percorso del sistema.



- *Definizione dei requisiti:* La fase iniziale prevede un costo di 20.000 euro ed è obbligatoria.

La probabilità di passare alla fase successiva è 0.7, mentre quella di rimanere nella stessa fase è 0.3.

- *Sviluppo del sistema:* Questa fase ha un costo di 40.000 euro. La probabilità di avanzare alla fase successiva è 0.7, mentre la probabilità di restare nella fase corrente o tornare indietro è rispettivamente 0.2 e 0.1.
- *Test del sistema:* L'ultima fase prima della conclusione prevede un costo di 20.000 euro. La probabilità di completare il processo, ovvero passare alla fase finale di uscita, è 0.7. Altre probabilità includono restare nella fase o tornare indietro (con probabilità 0.1).

Supponiamo di voler calcolare la probabilità che il costo totale del processo sia esattamente 80k. Il percorso che soddisfa questa condizione è: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

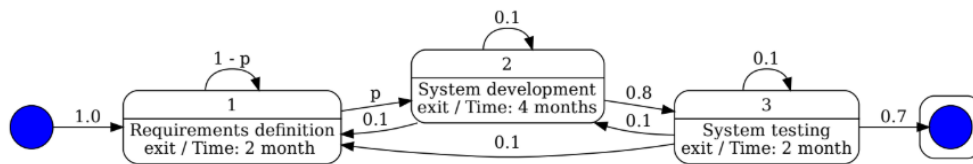
La probabilità totale di completare il processo seguendo il percorso specificato si ottiene moltiplicando le probabilità delle transizioni lungo il percorso:

$$1.0 \times 0.7 \times 0.7 \times 0.7 = 0.343$$

La probabilità che il processo termini con un costo complessivo di 80.000 è pari a 0.343.

Esempio 9.1.3 – Progettazione

Consideriamo un processo software che si sviluppa in tre fasi principali. Il sistema è caratterizzato da probabilità di transizione definite, con un parametro variabile p che influenza il comportamento del processo.



- *Definizione dei requisiti:* La fase iniziale ha una durata di 2 mesi. La probabilità di avanzare o restare nella fase corrente dipende dai parametri forniti, p e $1 - p$.
- *Sviluppo del sistema:* Questa fase ha una durata di 4 mesi. La probabilità di avanzare, restare nella fase corrente o tornare alla fase precedente sono rispettivamente 0.8, 0.1 e 0.1.
- *Test del sistema:* Ha una durata di 2 mesi. La probabilità di completare il processo è pari a 0.7.

L'obiettivo è determinare il valore minimo di p che garantisce che il processo termini esattamente in 8 mesi con una probabilità di almeno 0.5.

Il percorso che consente al processo di terminare esattamente in 8 mesi è $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$. Questo significa che il processo avanza senza ripetere alcuna fase.

La probabilità associata a questo percorso è data dalla moltiplicazione delle probabilità delle transizioni lungo il percorso:

$$1.0 \times p \times 0.8 \times 0.7 \geq 0.5$$

$$p \geq \frac{0.5}{0.8 \times 0.7} = 0.8928$$

Il valore minimo di p necessario per garantire che il processo termini esattamente in 8 mesi con una probabilità di almeno 0.5 è 0.8928.

Ora consideriamo lo stesso processo, ma il nostro obiettivo è garantire che il costo totale del processo sia esattamente 80k con una probabilità di almeno 0.5. Anche in questo caso, il valore p influenza le probabilità di transizione.

Il percorso che consente di ottenere un costo totale di 80k è $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

La probabilità associata a questo percorso è:

$$1.0 \times p \times 0.7 \times 0.7 \geq 0.5$$

$$p \geq \frac{0.5}{0.7 \times 0.7} = 1.02...$$

Non esiste un valore di p compreso tra 0 e 1 che soddisfi questa condizione, poiché il valore calcolato è maggiore di 1. Pertanto, non è possibile raggiungere una probabilità di almeno 0.5 che il costo totale del processo sia esattamente 80k.

9.1.2 Reti di DTMC

Le DTMC possono essere combinate in una rete, in cui ciascun componente è una DTMC autonoma ma interconnessa con le altre.

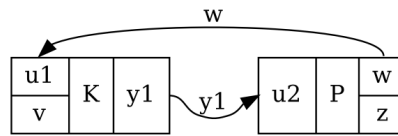


Figura 9.1: Una rete di DTMC

Ad esempio, considerando le due DTMC K e P con rispettivamente input $u1$ e v e $u2$ e output $y1$ e w e z (Figura 9.1). Le DTMC sono collegate tra loro attraverso gli output di una che diventano input dell'altra, in particolare l'output $y1$ di K diventa l'input $u2$ di P e l'output w di P diventa l'input $u1$ di K .

La rete risultante può essere definita come una nuova DTMC, con stati, probabilità di transizione e funzioni di output derivati dai componenti originali. Nel caso precedente la DTMC M risultante avrà input v e output z .

Formalmente, siano $K = (U_1 \times V, X_1, Y_1, p_1, g_1)$ e $P = (U_2, X_2, W \times Z, p_2, g_2)$ con $K(U_1) = P(W)$ e $P(U_2) = K(Y_1)$ allora

$$M = (V, X_1 \times X_2, Z, p, g)$$

dove

$$p((x'_1, x'_2)|(x_1, x_2), v) = p_1(x'_1|x_1, (g_{2,W}(x_2), v)) \times p_2(x'_2|x_2, g_1(x_1))$$

$$g(x_1, x_2) = g_{2,Z}(x_2)$$

Bibliografia

- [1] Ian Sommerville. *Software Engineering*. 9^a ed. Boston: Addison-Wesley, 2011.